

Large Language Models

building up from RNNs through Transformers

Paul Hongsuck Seo

phseo@korea.ac.kr

Korea University



KOREA
UNIVERSITY



Multimodal Interactive
Intelligence Laboratory

Processing Sequential Data

Examples of Sequence Data

- Speech Recognition:



"The quick brown fox jumped
over the lazy dog."

- Music Generation:

∅



- Sentiment Classification:

"There is nothing to like
in this movie."



- DNA Sequence Analysis:

AGCCCCTGTGAGGAACTAG



AG**CCCCTGTGAGGAACTAG**

- Machine Translation:

Voulez-vous chanter avec moi?



Do you want to sing with me?

- Video Activity Recognition:



Running

- Name Entity Recognition:

Yesterday, Harry Potter met
Hermione Granger.



Yesterday, **Harry Potter** met
Hermione Granger.

[slide courtesy: Stanford, CS230]

Sequential Data Representation

Input data: a set of vectors $\mathbf{x} = (\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(T)})^T$



ECG signal (3 channels)
100 samples/sec and acquire for 2 mins
→ 12,000 x 3 vector (T : 12,000)

$$\mathbf{x} = \left(\begin{pmatrix} 0.3 \\ 0.1 \\ 0.2 \end{pmatrix}, \begin{pmatrix} 0.5 \\ 0.6 \\ 0.4 \end{pmatrix}, \dots \dots \right)^T$$

Textual Data Representation

- Text sequential data representation
 - Text data is converted into a vector representation using a vocabulary.
 - Ex) The vocabulary is built with the most frequent 30,000 words in (Cho et. al., 2014).
- Text representation using dictionary
 - Bag-of-words (BoW)
 - One hot code
 - Word embedding

Textual Data Representation

- Bag-of-Words (BoW)

- Count the frequency of each word and express it as a vector of m dimensions.
- m : dictionary size

‘April is the cruelest month’ $\rightarrow (0, \dots, 0.2, \dots, 0.2, \dots)^T$

- It is good for image retrieval but is not suitable for sequential data representation.

- One Hot Code

- Each word is represented by $m \times 1$ vector $\rightarrow$ very inefficient.

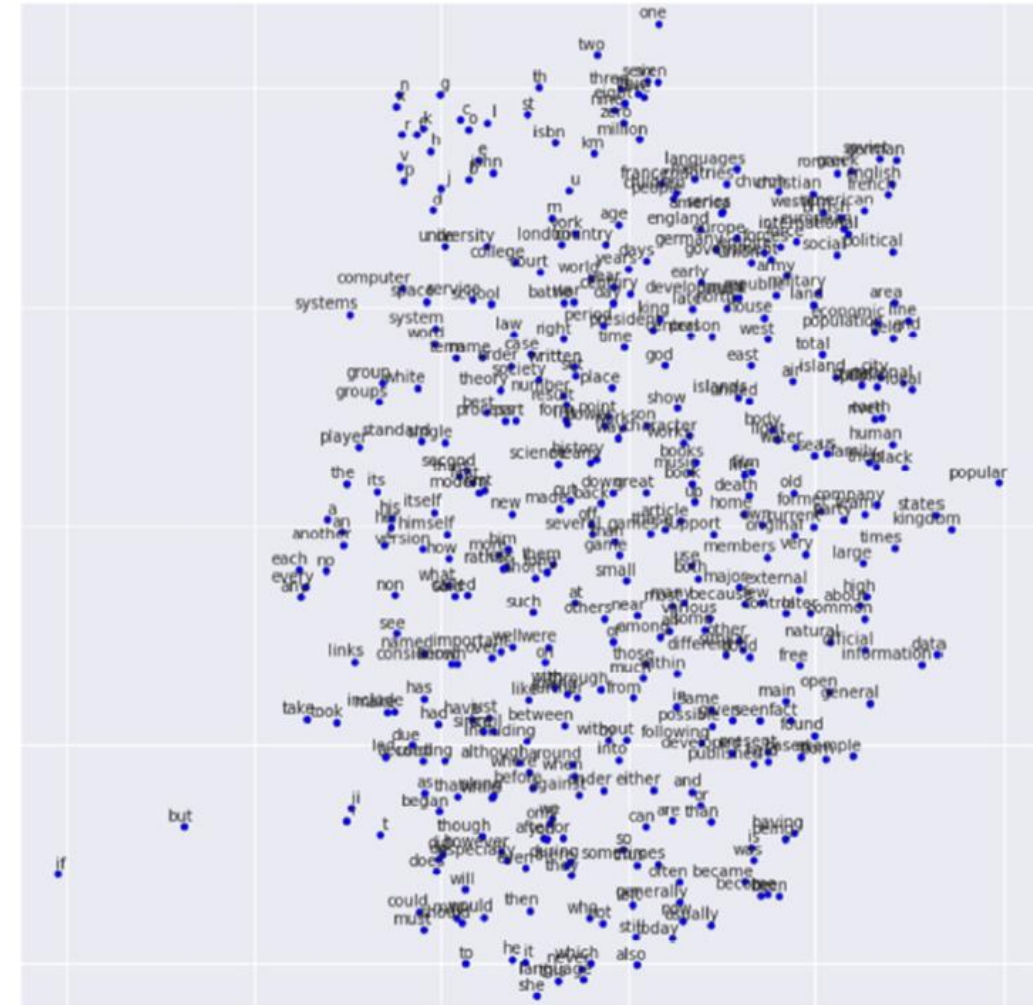
‘April is the cruelest month’ $\rightarrow \left((0,0,1,0,0,0, \dots)^T, (0,0,0,0,1,0, \dots)^T, \dots \right)^T$

Textual Data Representation

- Word Embedding

- By analyzing the correlation between words in dictionary, convert the word into lower dimensional space
- Ex) word in 30,000 dimensions → converted into 620 dimension

Word embedding using word2vec



Cho et. al., On the Properties of Neural Machine Translation: Encoder–Decoder Approaches, SSST-8 2014

Textual Data Representation

- The **order** in which the features (words) appear is important.
- Training samples may have different sizes
- Context dependency
 - In non-sequential data, covariance matrix indicates a correlation over features.
 - In sequential data, context dependency is important.

She got up at lunch and ate breakfast, and he came back later.



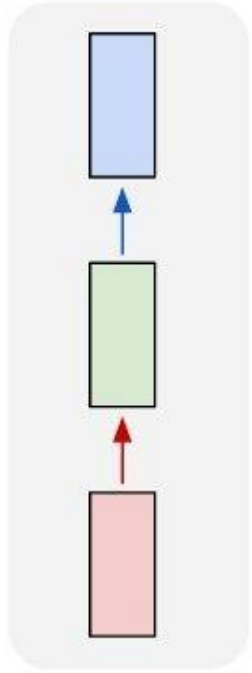
Long-term dependency
(Two distant words are correlated)

Use LSTM for handling this!

Recurrent Neural Networks

“Vanilla” Neural Networks

one to one

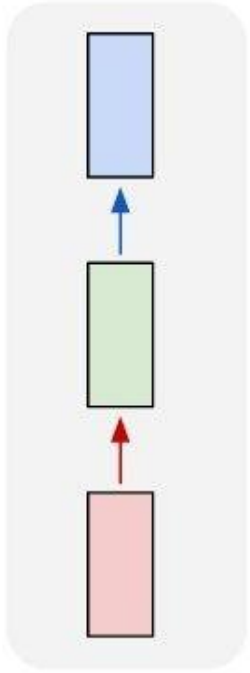


Vanilla Neural Networks

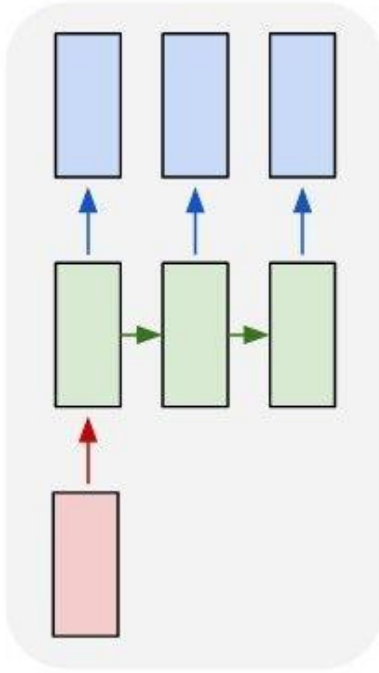
[slide courtesy: Stanford, CS231]

Recurrent Neural Networks (RNNs)

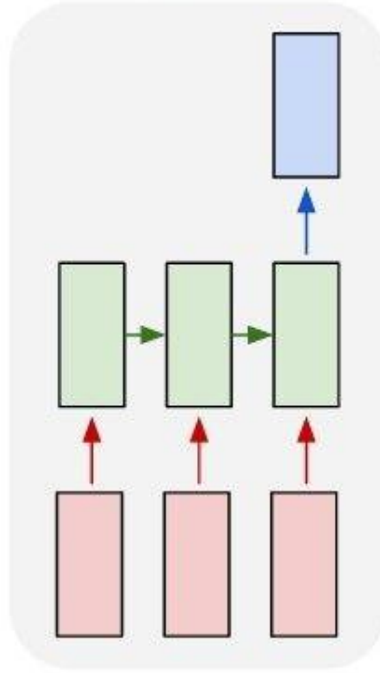
one to one



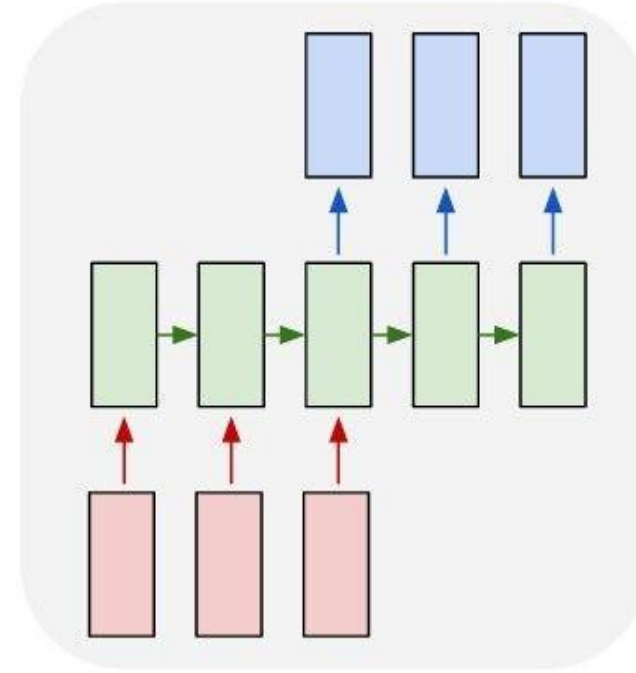
one to many



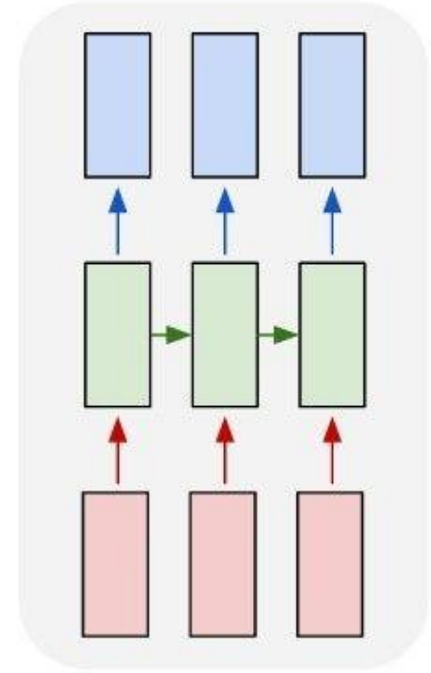
many to one



many to many



many to many

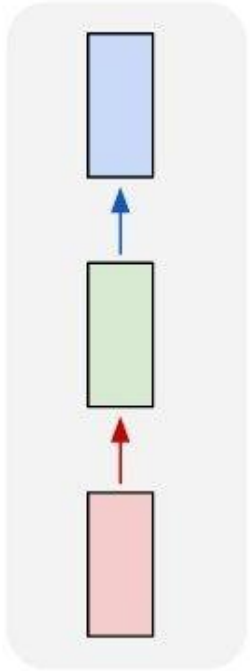


e.g., **Image Captioning**
Image → Sequence of words

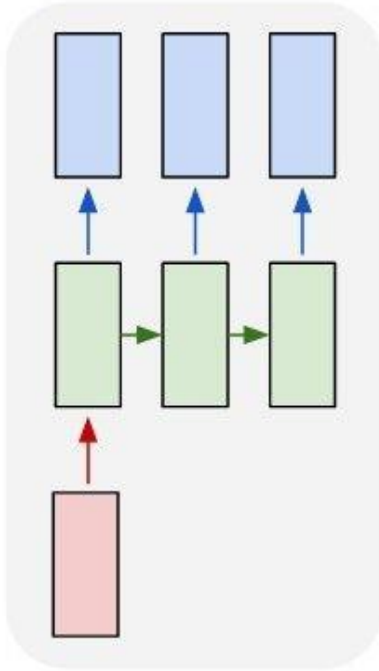
[slide courtesy: Stanford, CS231]

Recurrent Neural Networks (RNNs)

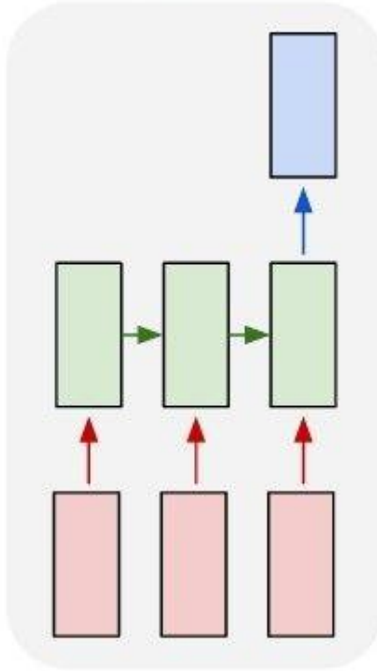
one to one



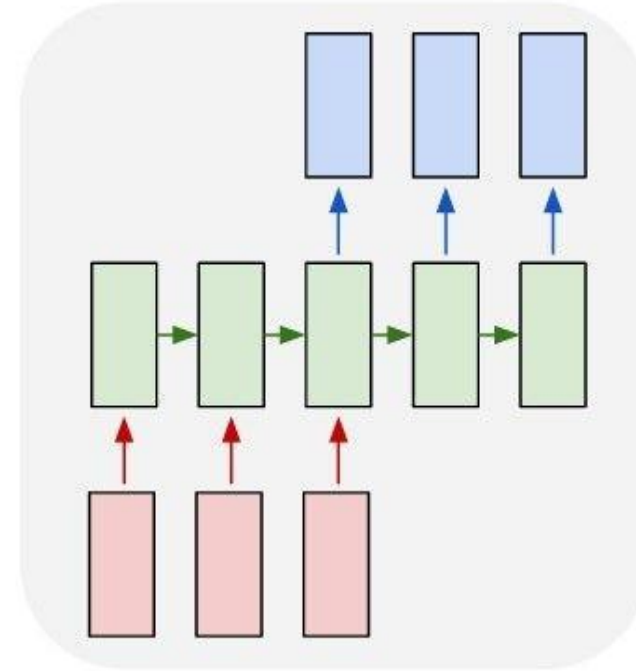
one to many



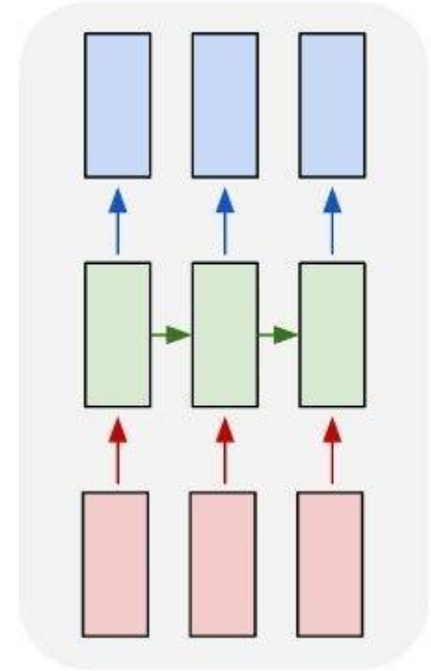
many to one



many to many



many to many

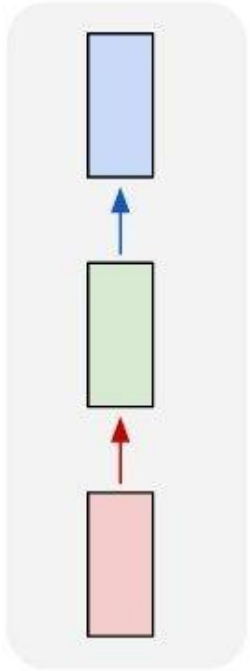


e.g., **Sentiment Analysis**
Sequence of words → Sentiment label

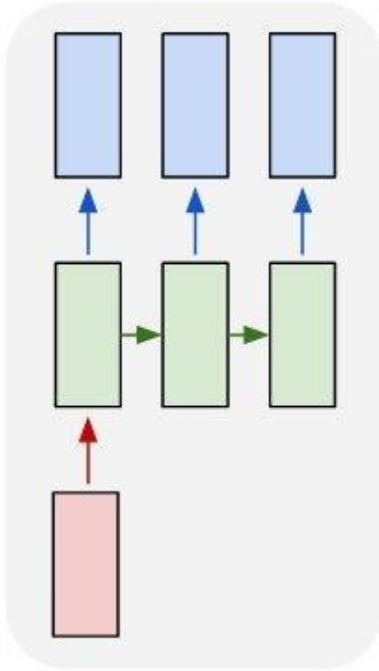
[slide courtesy: Stanford, CS231]

Recurrent Neural Networks (RNNs)

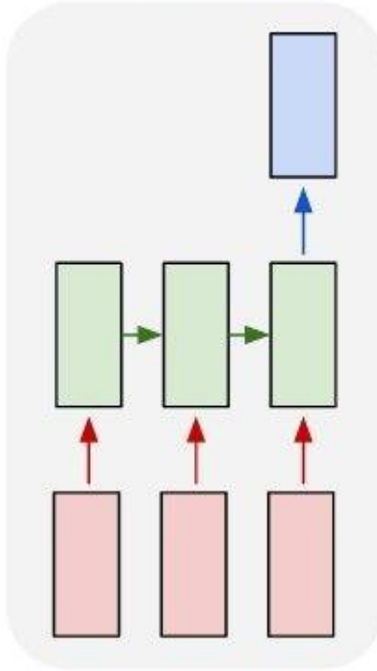
one to one



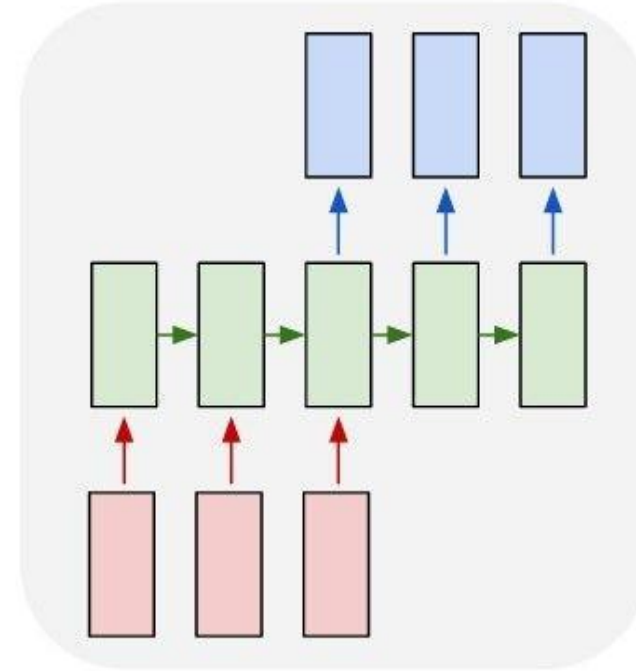
one to many



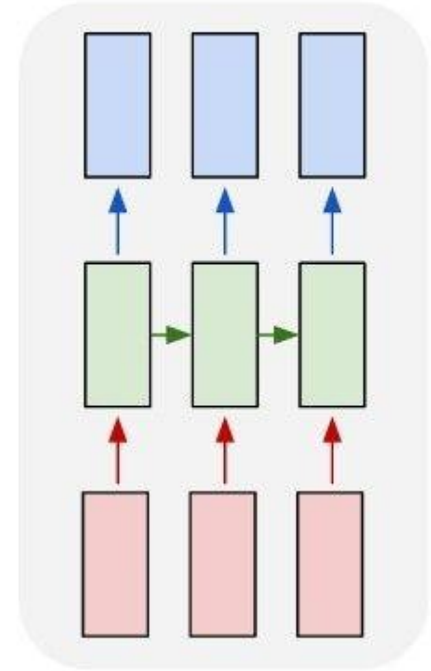
many to one



many to many



many to many

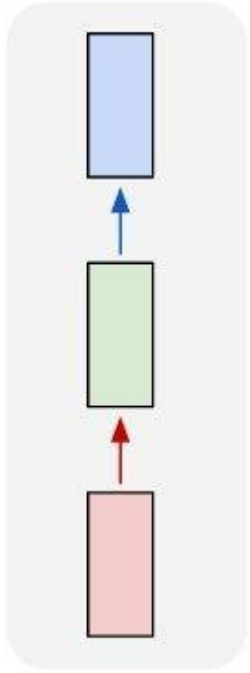


e.g., **Machine Translation**
Sequence of words → Sequence of words

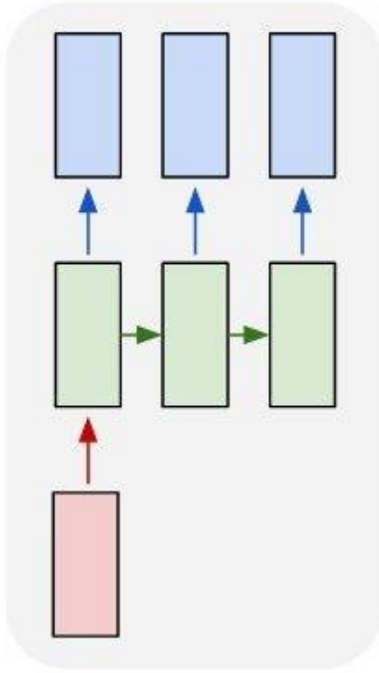
[slide courtesy: Stanford, CS231]

Recurrent Neural Networks (RNNs)

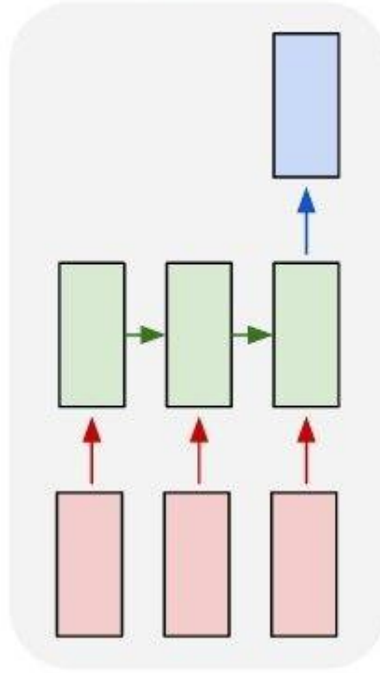
one to one



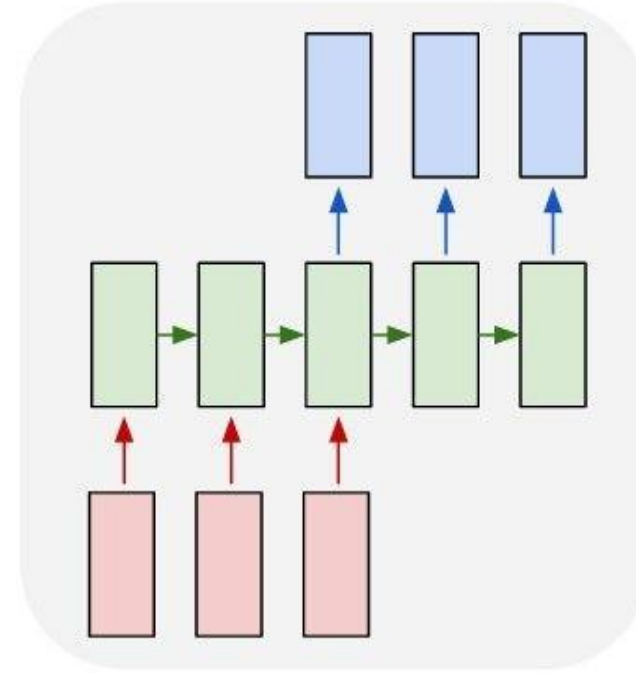
one to many



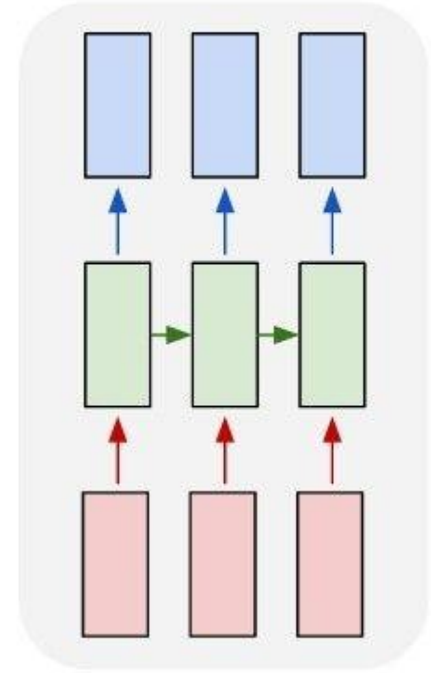
many to one



many to many



many to many



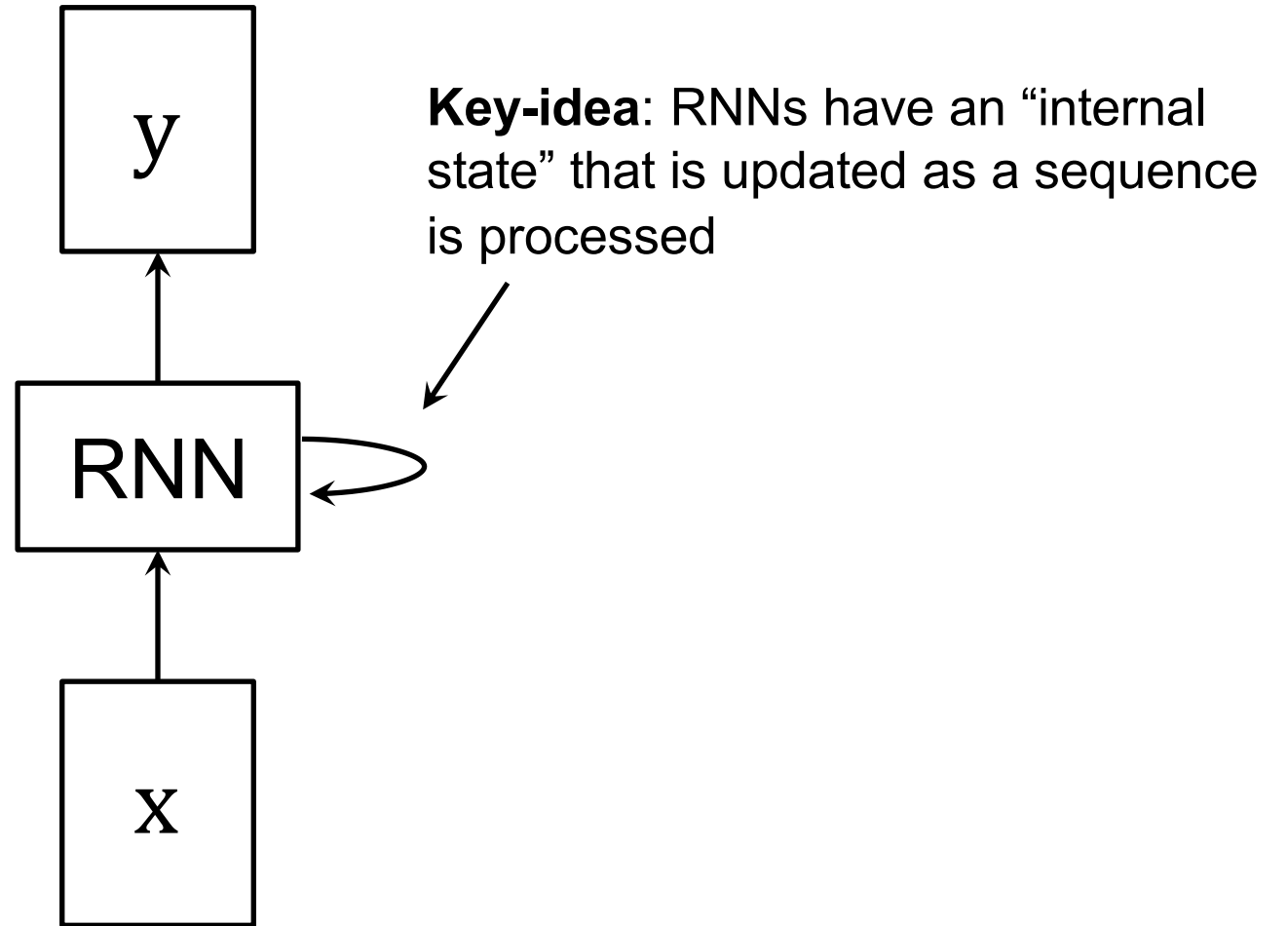
e.g., **Temporal Action Segmentation**

Sequence of video frames → Action class per frame



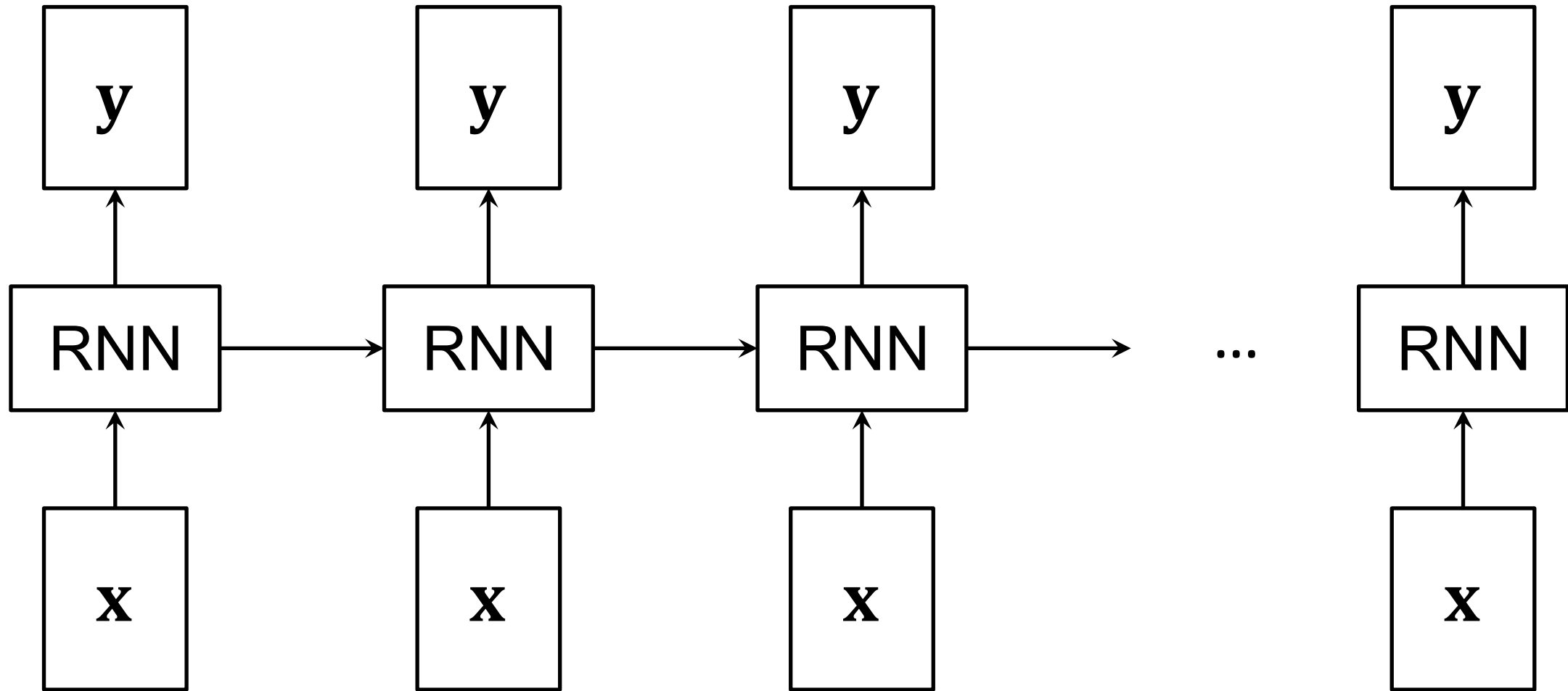
[slide courtesy: Stanford, CS231]

Recurrent Neural Networks (RNNs)



[slide courtesy: Stanford, CS231]

Recurrent Neural Networks (RNNs)



[slide courtesy: Stanford, CS231]

Recurrent Neural Networks (RNNs)

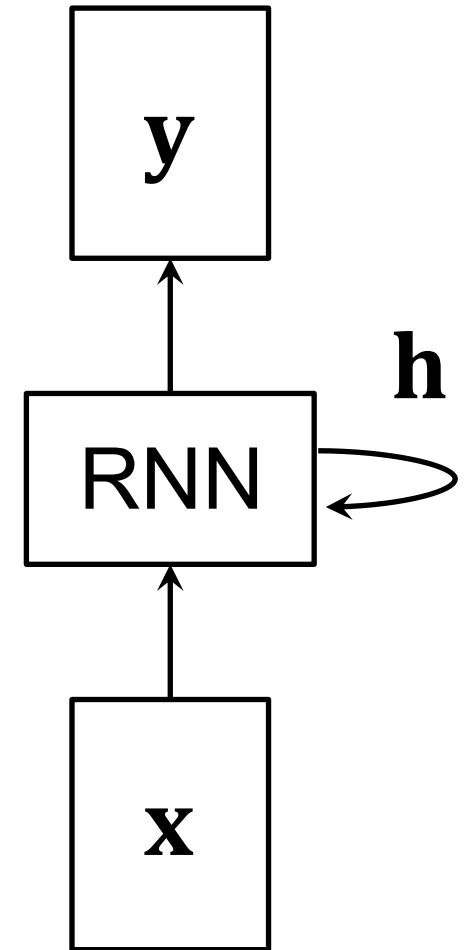
- We can process a sequence of vectors x by applying a recurrence formula at every time step:

$$\mathbf{h}_t = f_W(\mathbf{h}_{t-1}, \mathbf{x}_t)$$

New state Old state Input vector at some time step

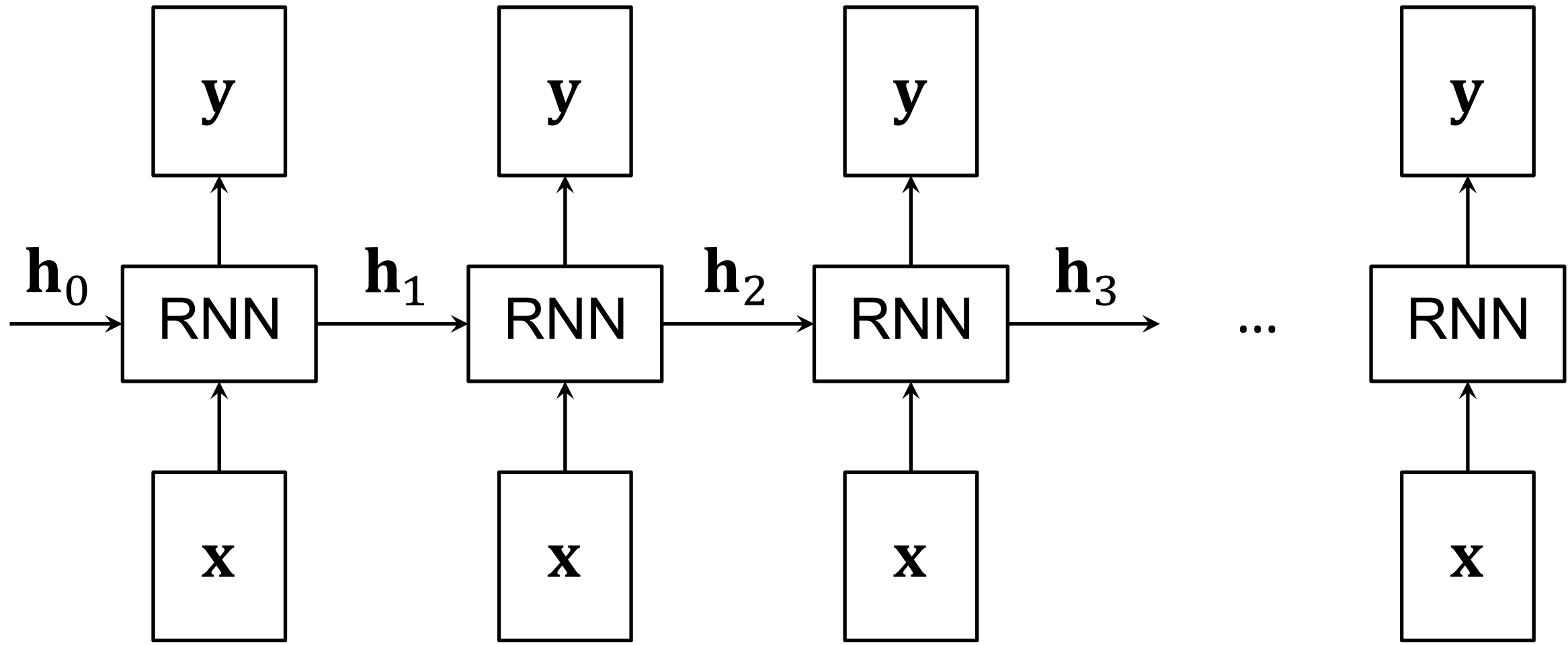
Some function with parameters W

- Notice: the same function and the same set of parameters are used at every time step.



[slide courtesy: Stanford, CS231]

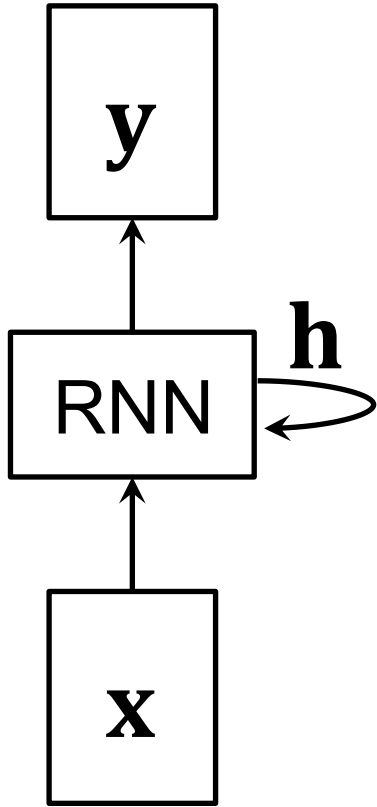
Recurrent Neural Networks (RNNs)



[slide courtesy: Stanford, CS231]

Simple Recurrent Neural Networks (RNNs)

- The state consists of a single “hidden” vector $\mathbf{h}$:



$$\mathbf{h}_t = f_W(\mathbf{h}_{t-1}, \mathbf{x}_t)$$

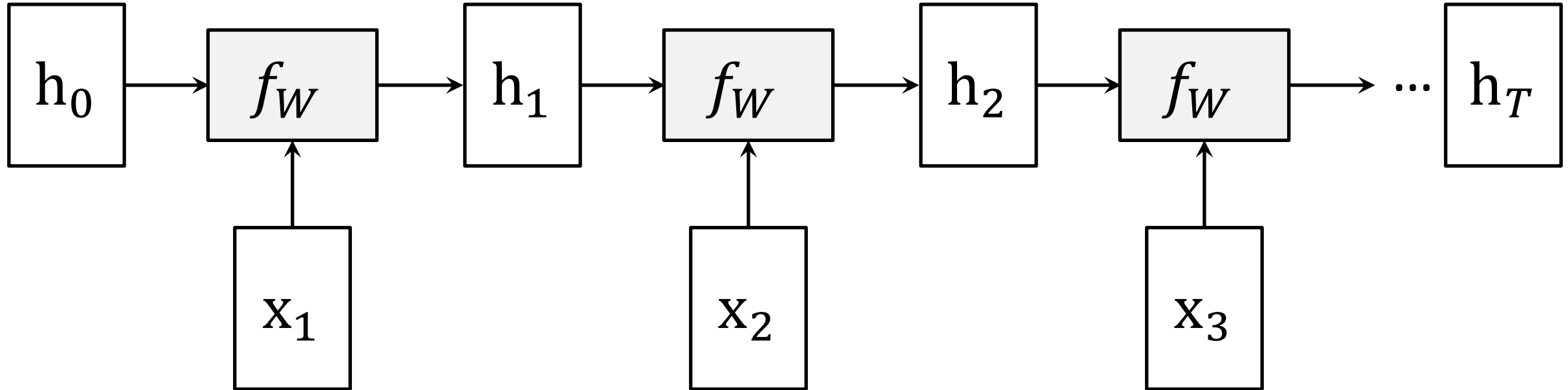
↓

$$\mathbf{h}_t = \tanh(W_{hh}\mathbf{h}_{t-1} + W_{xh}\mathbf{x}_t)$$

$$\mathbf{y}_t = W_{hy}\mathbf{h}_t$$

[slide courtesy: Stanford, CS231]

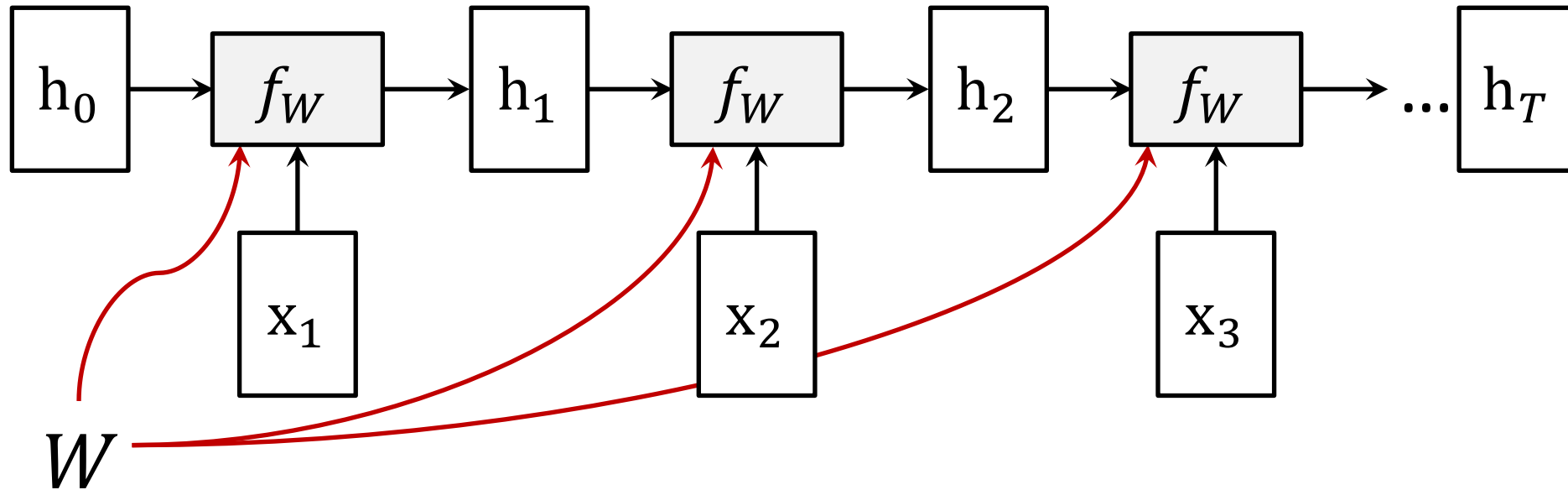
RNN: Computational Graph



[slide courtesy: Stanford, CS231]

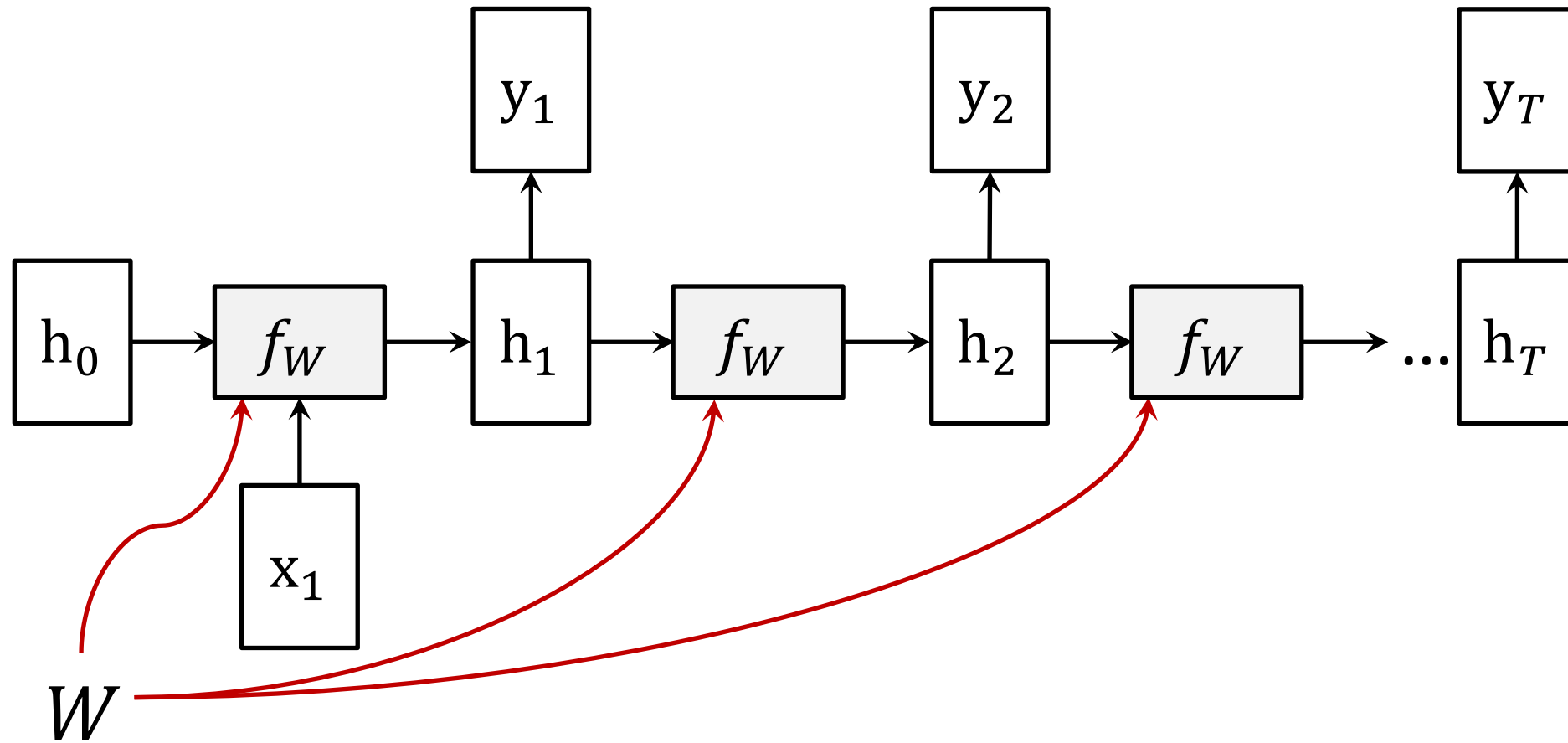
RNN: Computational Graph

- Re-use the same weight matrix at every time-step:



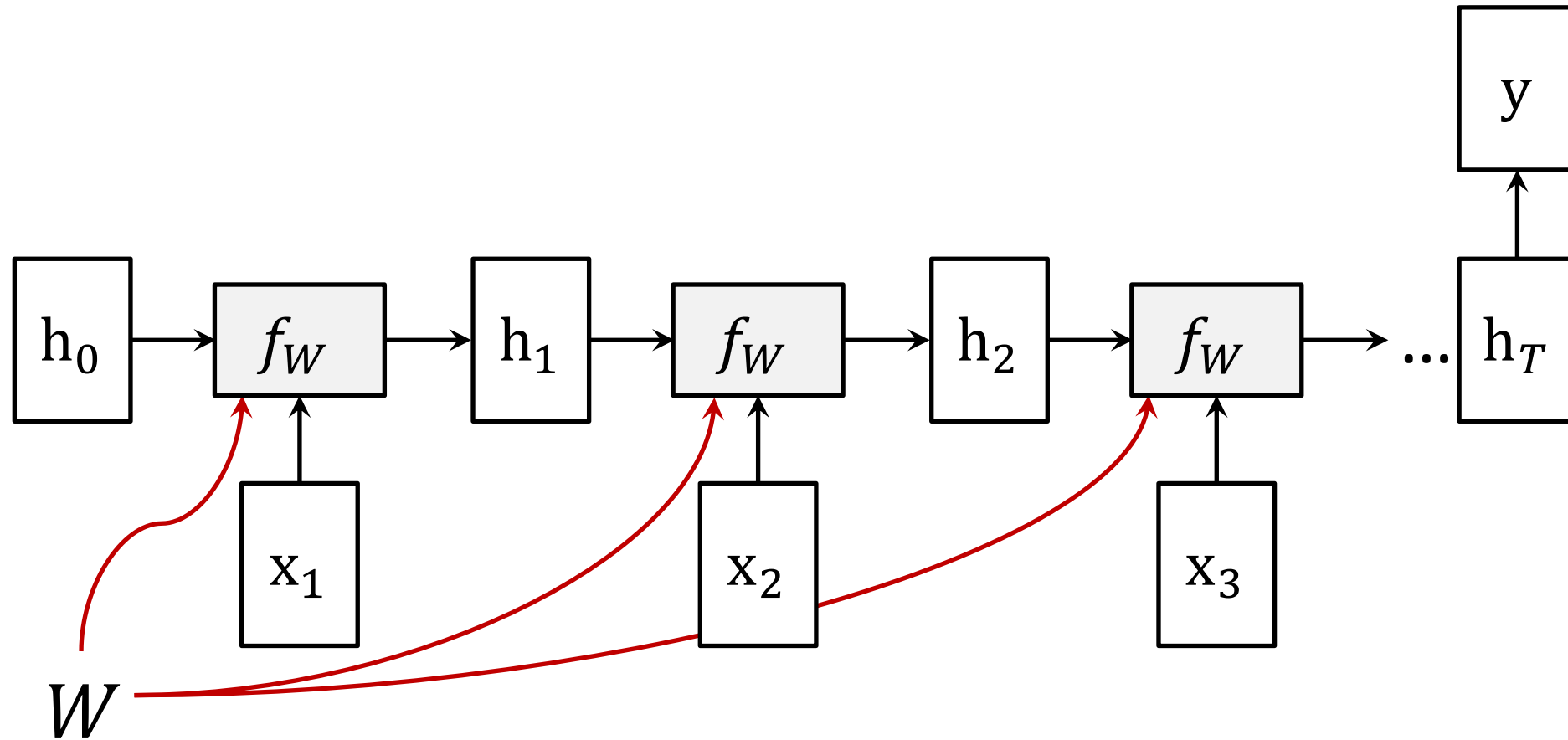
[slide courtesy: Stanford, CS231]

RNN: Computational Graph – One to Many



[slide courtesy: Stanford, CS231]

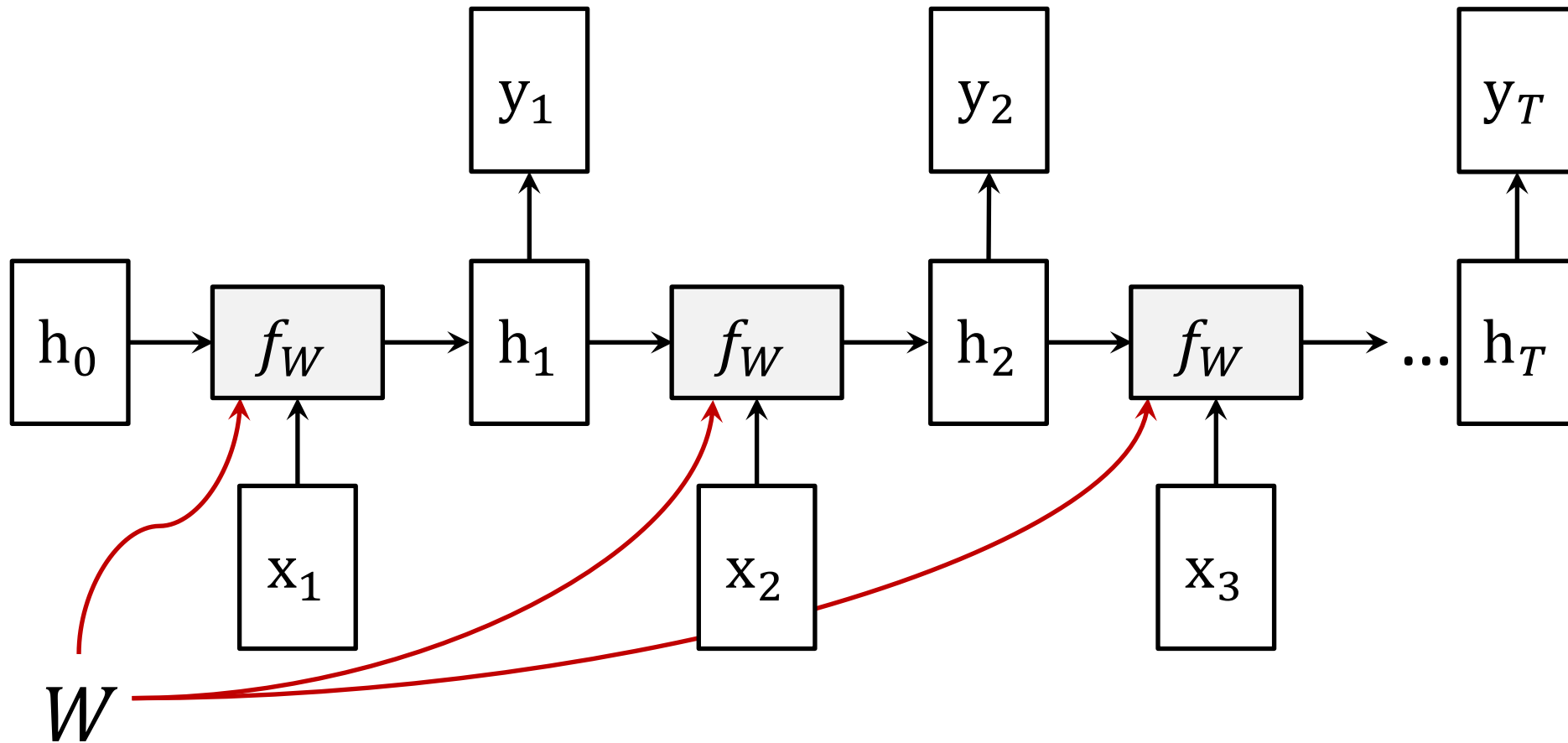
RNN: Computational Graph – Many to One



[slide courtesy: Stanford, CS231]

RNN: Computational Graph – Many to Many (1)

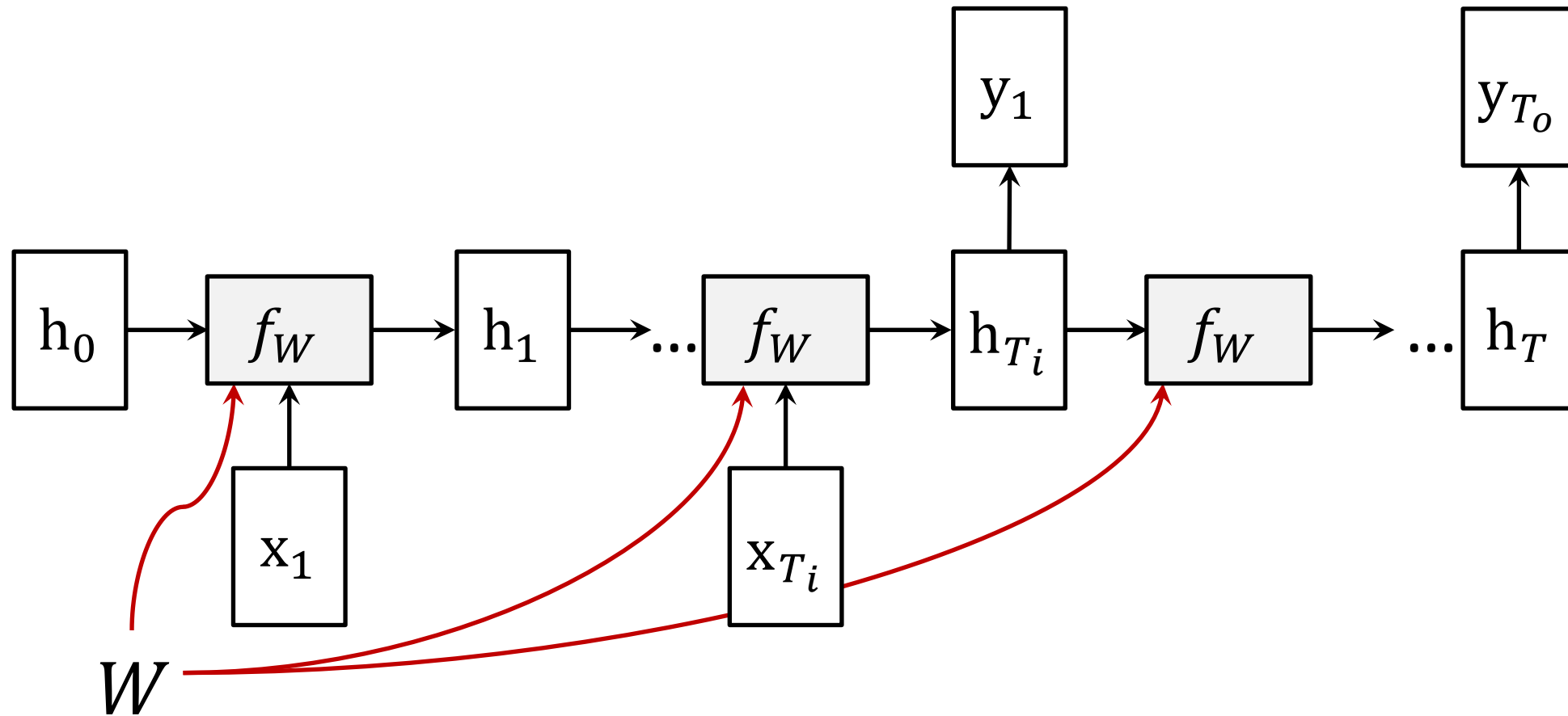
- Aligned prediction



[slide courtesy: Stanford, CS231]

RNN: Computational Graph – Many to Many (2)

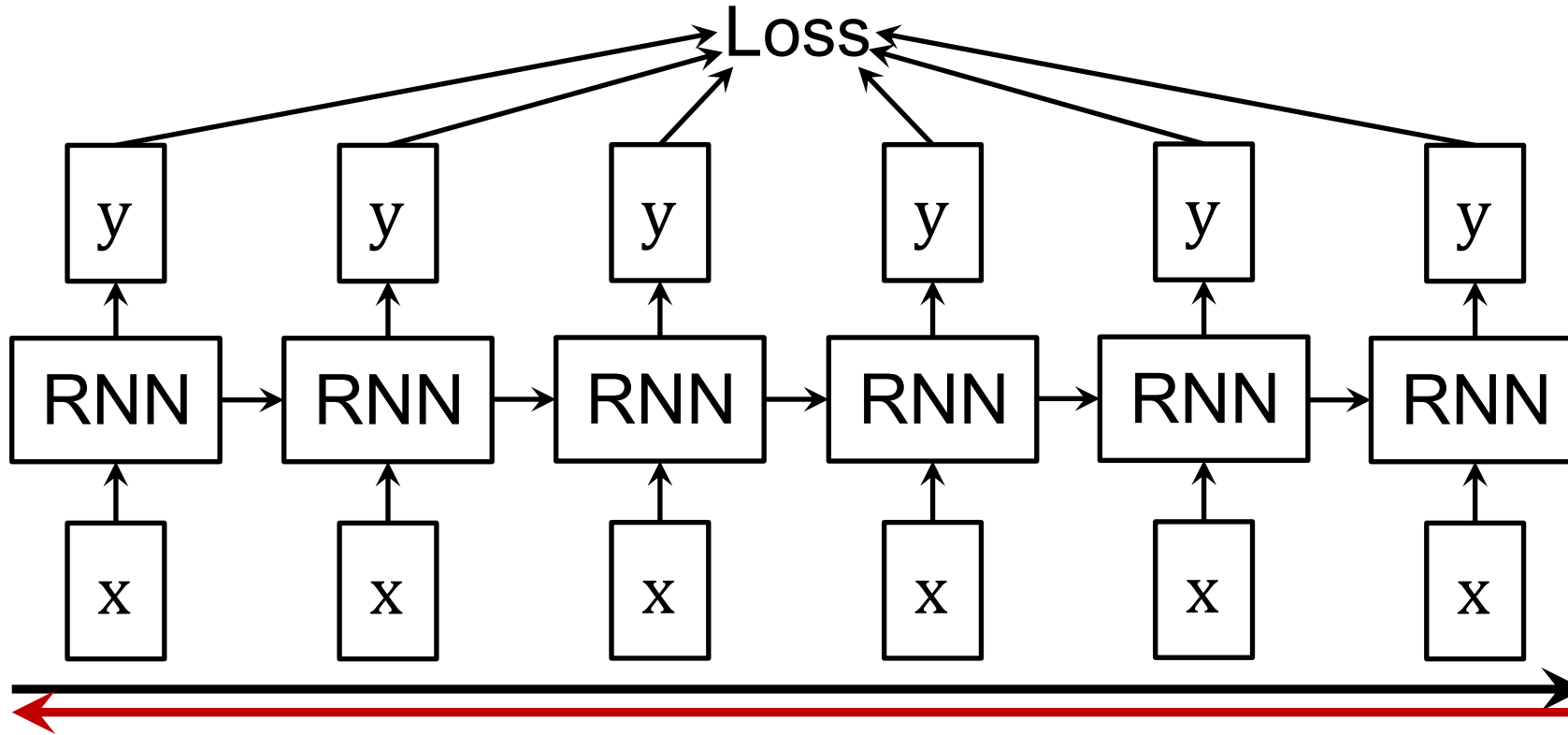
- Sequence to sequence



Sutskever et al., Sequence to Sequence Learning with Neural Networks, NIPS 2014

Backpropagation through Time

- Forward through entire sequence to compute loss, then backward through entire sequence to compute gradient:



[slide courtesy: Stanford, CS231]

Recurrent Neural Networks Properties

- RNN Advantage:
 - Can process any length input.
 - Computation for step t can (in theory) use information from many steps back.
 - Model size does not increase for longer input.
 - Same weights applied on every timestep, so there is symmetry in how inputs are processed.
- RNN Disadvantage:
 - Recurrent computation is slow.
 - In practice, difficult to access information from many steps back.
 - Unidirectional sequence modeling.

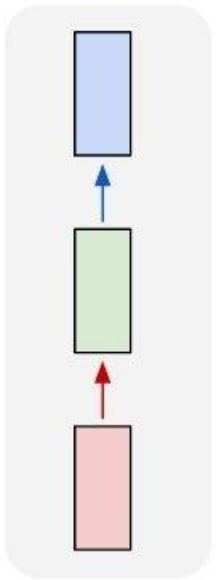
[slide courtesy: Stanford, CS231]

Language Processing with RNNs

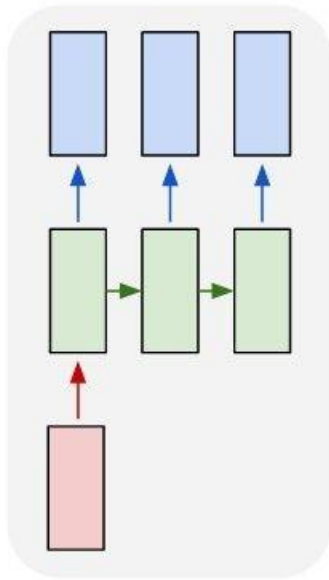
Language Processing with RNNs

- A text is a sequence of characters, words or *tokens*.
- In memory, it is represented as an array of characters.
- Language as a sequence input to RNNs.

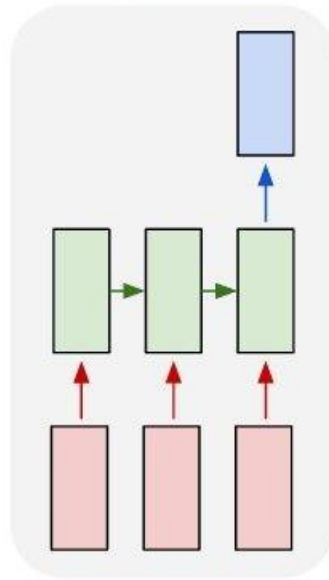
one to one



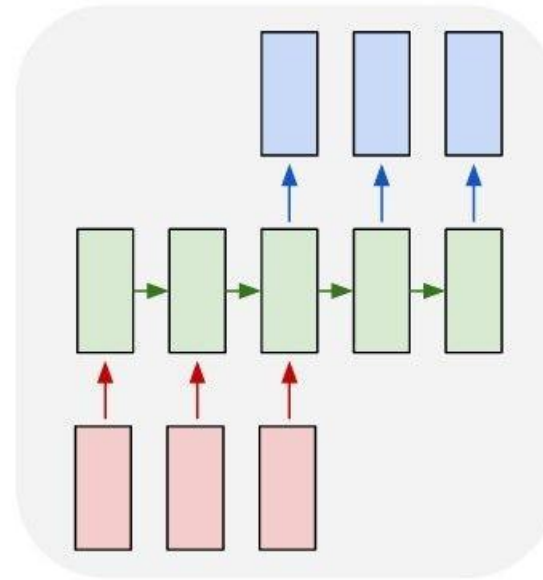
one to many



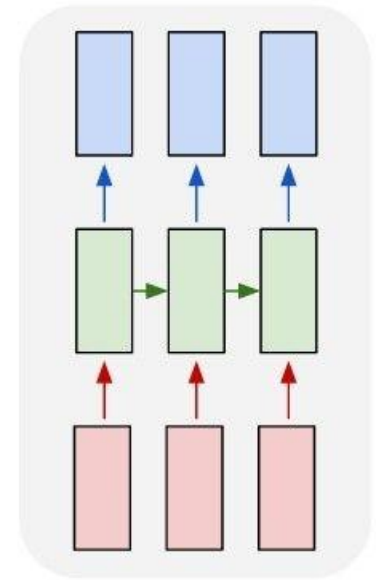
many to one



many to many

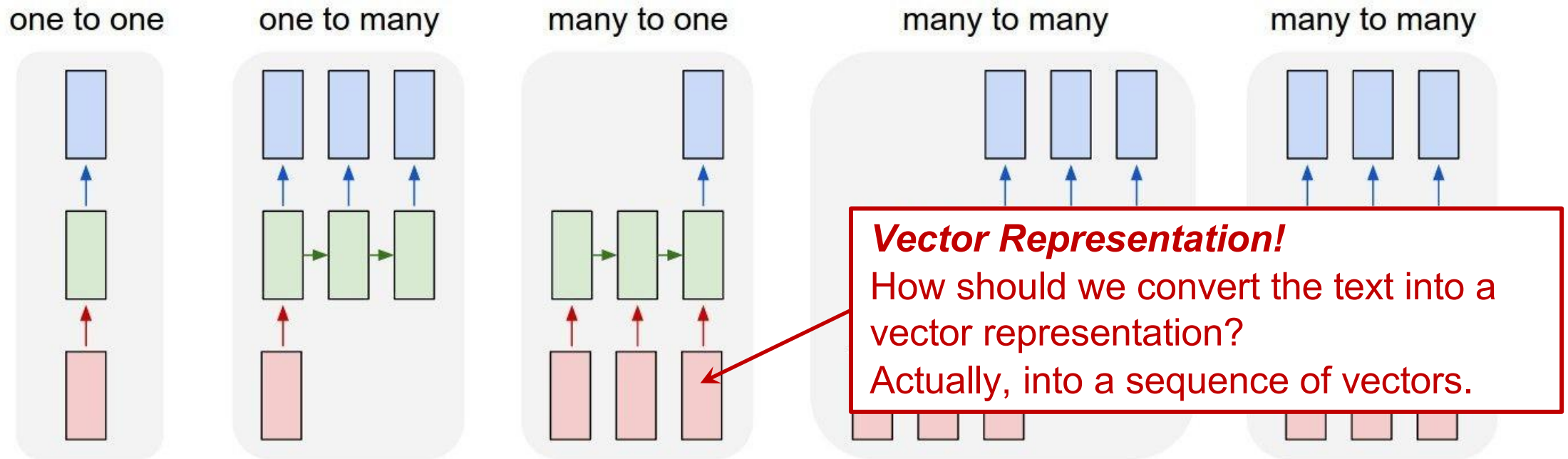


many to many



Language Processing with RNNs

- A text is a sequence of characters, words or *tokens*.
- In memory, it is represented as an array of characters.
- Language as a sequence input to RNNs.

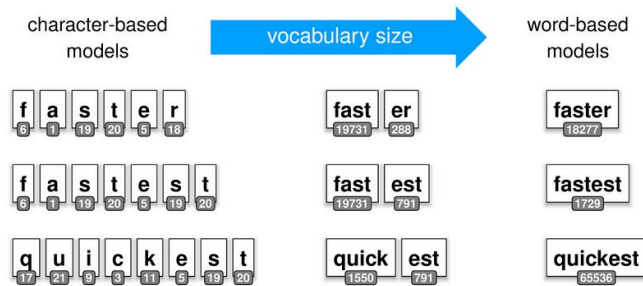


Text Preprocessing for Vector Representation

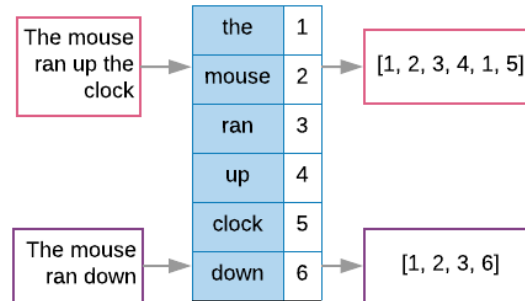
- ***Text tokenization*** is the process of converting a sequence of text into smaller parts, known as tokens.
- ***Vocabulary*** stores all possible tokens and maps each token to a unique index.
- Tokens are converted into a sequence of ***one-hot vectors***.
- Note: bag-of-words representation converts the entire text into a single vector whereas the one-hot vector representation outputs a sequence of vectors.

Text Tokenization

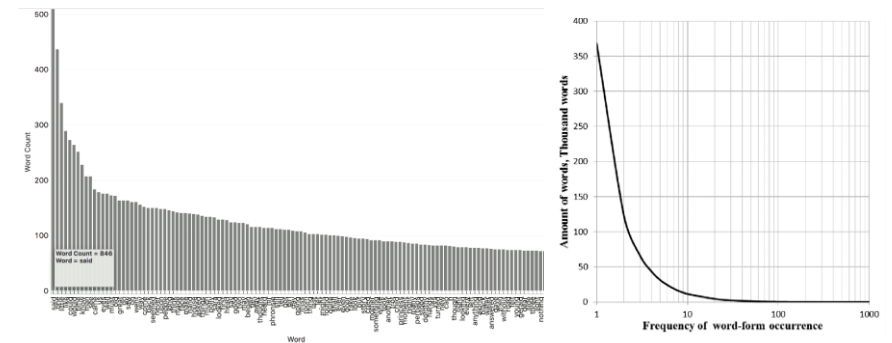
- Character tokenization: each character corresponds to a single token.
 - Lacks semantic encoding capability.
 - Results in long sequences.
- Word tokenization: each word is treated as a token.
 - Forms a big vocabulary with a long-tail distribution.
 - Needs a special token to deal with unknown words.
- Sub-word tokenization: the vocabulary consists of frequent sub-words.
 - Takes advantages from the both methods.



Tokenization at different levels



Example vocabulary for tokenization



Long-tail word distribution

[source: <https://www.innerdoc.com/periodic-table-of-nlp-tasks/14-tokenization/>]

[source: <https://orangedatamining.com/blog/observing-word-distribution/>]

[source: https://www.researchgate.net/figure/Word-frequency-distribution-for-the-text-corpus_fig1_259118530]

[source: <https://developers.google.com/machine-learning/guides/text-classification/step-3?hl=ko>]

Sub-word Tokenization

- Sub-word tokenization handles unknown words by merging semantics of sub-words
 - *preschool* → *pre* + *school*
 - *longest* → *long* + *est*
 - *tokenization* → *token* + *ization*
- Sub-word vocabulary construction methods
 - WordPiece
 - SentencePiece
 - Byte Pair Encoding (BPE)
 - Unigram Language Model

Sennrich et. al., Neural Machine Translation of Rare Words with Subword Units, ACL 2016

Taku, Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates, ACL 2018

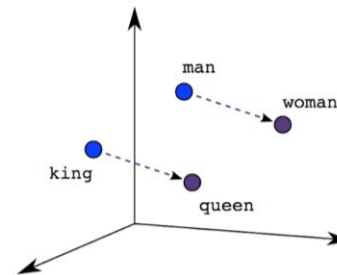
Taku et. al., SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing, EMNLP 2018

Schuster, Japanese and Korean voice search, ICASSP 2012

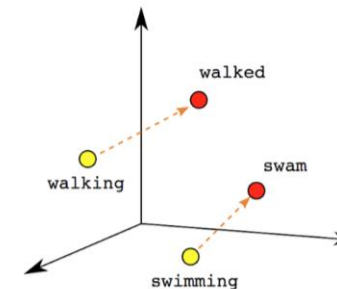
One-Hot Encoding vs. Word Embedding

- One-hot encoding distinguishes different words in a vector space.
- But it has some limitations
 - Highly sparse and inefficient use of memory with high-dimensional vectors.
 - The representation lacks semantic information resulting in identical pair-wise cosine similarities (0) between all words.
- Word embeddings in contrast map similar words close to each other.
 - It allows the semantic similarity between words to be captured by their vector distances or directions.

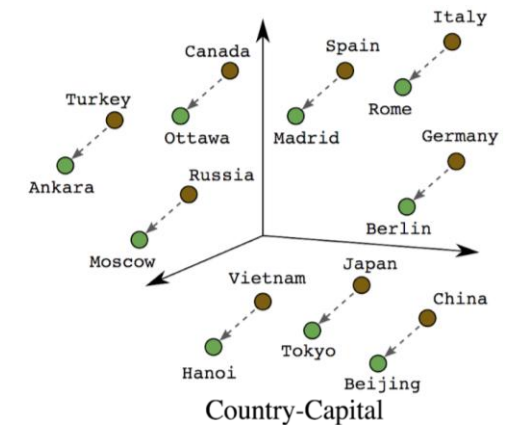
Man (5391)	Woman (9853)
0	0
0	0
0	0
0	0
⋮	⋮
1	1
⋮	⋮
0	0
0	0



Male-Female



Verb Tense



Country-Capital

[source: <https://developers.google.com/machine-learning/guides/text-classification/step-3?hl=ko>]

One-Hot Encoding vs. Word Embedding

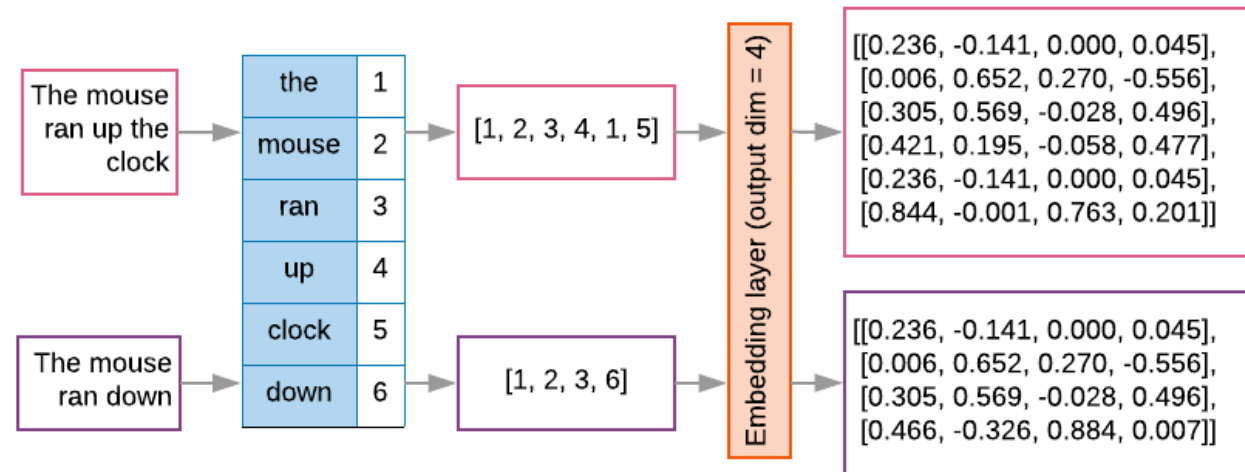
- Technically, one-hot encoding with an FC layer is identical to having a word embeddings
- In practice, word embeddings are implemented by indexing a column (or a row) vector from a weight matrix.

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 \end{bmatrix} \times \begin{bmatrix} 8 & 2 & 1 & 9 \\ 6 & 5 & 4 & 0 \\ 7 & 1 & 6 & 2 \\ 1 & 3 & 5 & 8 \\ 0 & 4 & 9 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 3 & 5 & 8 \end{bmatrix}$$

One-hot vector

Embedding Weight Matrix

Hidden layer output



[source: <https://medium.com/@gautam.e/what-is-nn-embedding-really-de038baadd24>]

[source: <https://developers.google.com/machine-learning/guides/text-classification/step-3?hl=ko>]

Example Task: Machine Translation

Machine Translation

- Translating a sentence (not a document) into another language.
- Input and output structures?
 - Both input and output are texts (sequences of tokens)
 - Note: there is no alignment information.
- What training data?
 - Europarl Parallel Corpus
 - A benchmark with multiple parallel corpora for machine translation
 - Extracted from the proceedings of the European Parliament.
 - French to English corpus contains 2M sentence pairs with 51M French and 50M English words.

Machine Translation

- Examples in a parallel corpus

French	English
Nous sommes arrivés à un moment où nous devrions reconnaître l'échec d'au moins ´.	We are actually at the point where we should recognise the failure of at least s ´.
Nous avons également essayé de rationaliser nos procédures par d'autres moyens.	We have tried to rationalise our procedures in other ways.
Ce qui a été observé dans la pratique, c'est que les demandeurs de visas dont la ´.	What has been observed in practice is that visa applicants who are rejected by o ´.
La violation flagrante des droits de l'homme et des libertés fondamentales dans ´.	The flagrant violation of human rights and freedoms in this region is a cause fo ´.
Cependant, cela montre également qu'il existe encore bien d'autres contraintes a ´.	However, it also shows that there is much more red tape that we need to get the ´.
Si de telles situations devaient se reproduire à l'avenir, il est impératif que ´.	Should similar situations arise in the future, reliable information must be made ´.
Si l'on traduit cela en mesures bien concrètes, il faudra prendre quelques décis ´.	If we convert that into concrete measures, then some painful decisions will have ´.
25.	25.
(EN) Monsieur le Président, force est de constater que la démocratie et les droi ´.	Mr President, the democratic and human rights situation in Venezuela is in const ´.
Une dérégulation trop forte a pour conséquence que les marchés financiers ne s'o ´.	Excessive deregulation makes expectation the main determinant of the finance mar ´.

[source: <https://datarepository.wolframcloud.com/resources/Europarl-English-French-Machine-Translation-Dataset-V7>]

Machine Translation

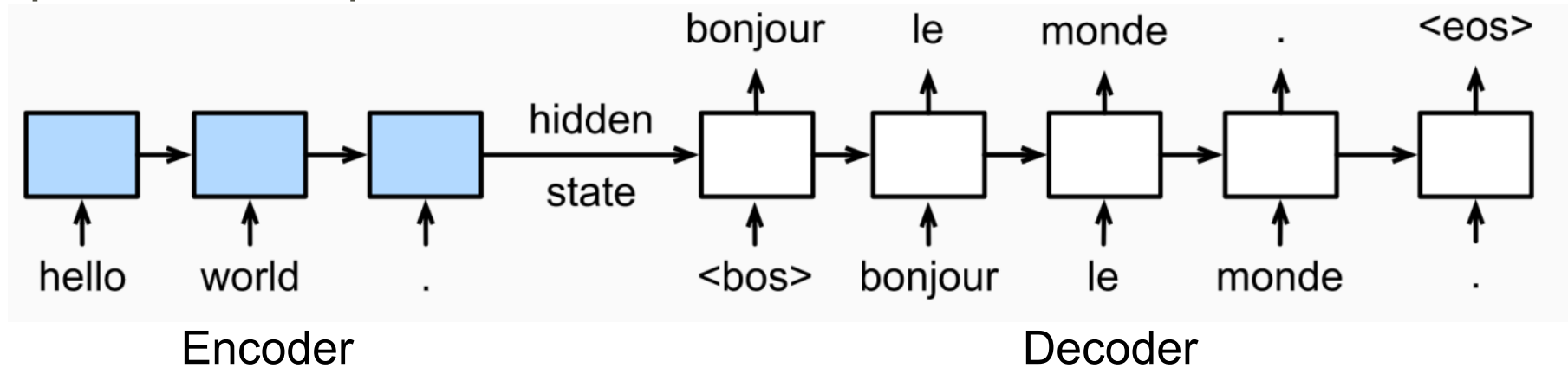
- What neural architecture?

[source: <https://pub.aimind.so/generating-text-with-lstm-networks-using-tensorflow-a-comprehensive-guide-79cc2eeda20a>]

[source: <https://www.linkedin.com/pulse/anatomy-sequence-to-sequence-machine-translation-rnn-gru-sobh-phd/>]

Machine Translation

- What neural architecture?
 - Sequence to sequence model!

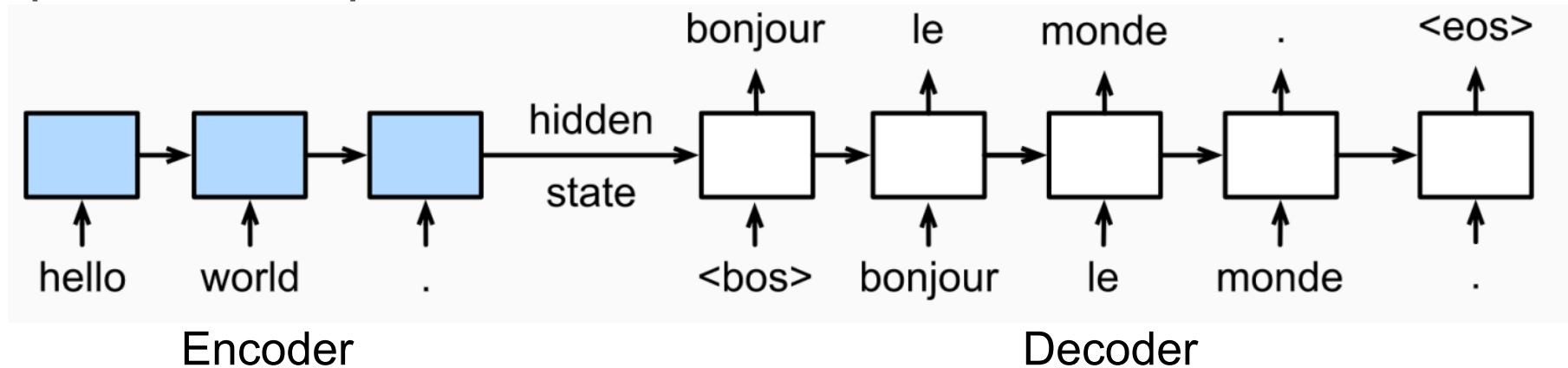


[source: <https://pub.aimind.so/generating-text-with-lstm-networks-using-tensorflow-a-comprehensive-guide-79cc2eeda20a>]

[source: <https://www.linkedin.com/pulse/anatomy-sequence-to-sequence-machine-translation-rnn-gru-sobh-phd/>]

Machine Translation

- What neural architecture?
 - Sequence to sequence model!



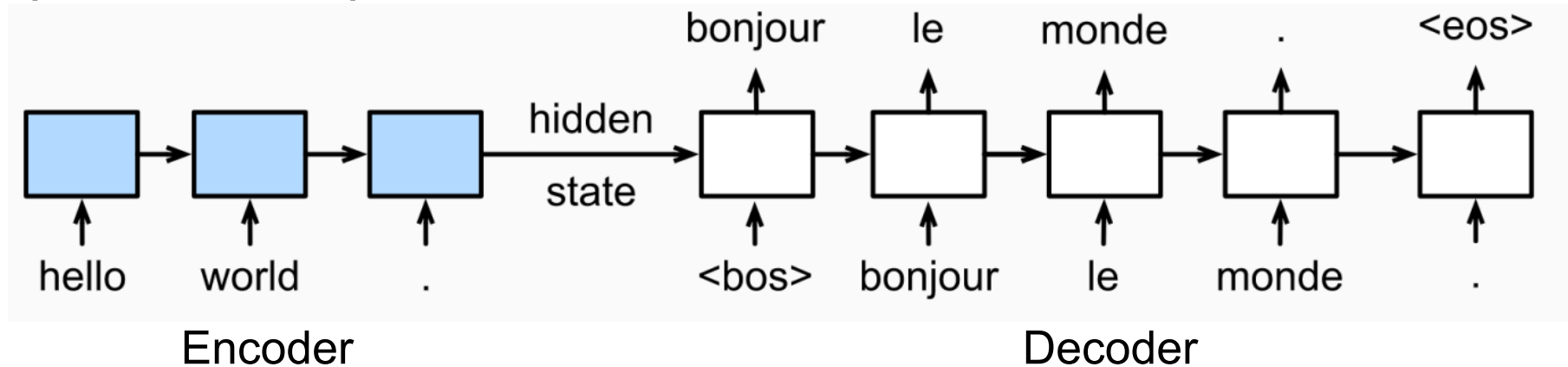
- Classifying a ***word (token) per timestep*** using softmax output function.
- The alignment is never explicitly learned.

[source: <https://pub.aimind.so/generating-text-with-lstm-networks-using-tensorflow-a-comprehensive-guide-79cc2eeda20a/>]

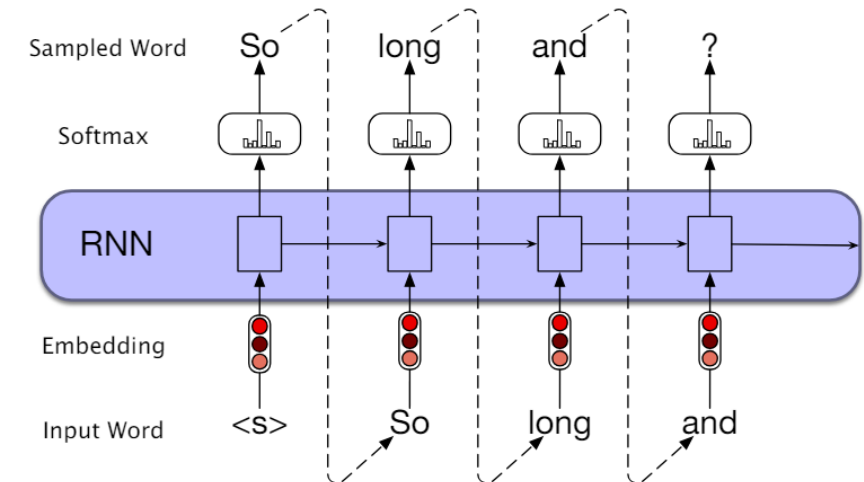
[source: <https://www.linkedin.com/pulse/anatomy-sequence-to-sequence-machine-translation-rnn-gru-sobh-phd/>]

Machine Translation

- What neural architecture?
 - Sequence to sequence model!



- Classifying a ***word (token) per timestep*** using softmax output function.
- The alignment is never explicitly learned.
- ***Autoregressive generation!***



[source: <https://pub.aimind.so/generating-text-with-lstm-networks-using-tensorflow-a-comprehensive-guide-79cc2eeda20a/>]
[source: <https://www.linkedin.com/pulse/anatomy-sequence-to-sequence-machine-translation-rnn-gru-sobh-phd/>]

Machine Translation

- What loss function?

Machine Translation

- What loss function?
 - Negative Log-Likelihood (NLL) loss
 - NLL loss maximizes the predicted probability of the GT label.

$$L(W) = -\frac{1}{N} \sum_{n=1}^N \log P_W(y = y_n | \mathbf{x}_n)$$

Machine Translation

- What loss function?
 - Negative Log-Likelihood (NLL) loss
 - NLL loss maximizes the predicted probability of the GT label.

$$L(W) = -\frac{1}{N} \sum_{n=1}^N \log P_W(y = y_n | \mathbf{x}_n)$$

GT translated sentence

Machine Translation

- What loss function?
 - Negative Log-Likelihood (NLL) loss
 - NLL loss maximizes the predicted probability of the GT label.

$$L(W) = -\frac{1}{N} \sum_{n=1}^N \log P_W(y = y_n | \mathbf{x}_n)$$

GT translated sentence

Probability of GT translated sentence

Machine Translation

- What loss function?
 - Negative Log-Likelihood (NLL) loss
 - NLL loss maximizes the predicted probability of the GT label.

$$L(W) = -\frac{1}{N} \sum_{n=1}^N \log P_W(y = y_n | \mathbf{x}_n)$$

GT translated sentence

Probability of GT translated sentence

- The probability of a generated sentence can be decomposed using the chain rule.

Chain Rule:

$$\begin{aligned} P(A_1, A_2, \dots, A_n) &= P(A_n | A_1, \dots, A_{n-1}) P(A_1, \dots, A_{n-1}) \\ &= P(A_n | A_1, \dots, A_{n-1}) P(A_{n-1} | A_1, \dots, A_{n-2}) P(A_1, \dots, A_{n-2}) \\ &= P(A_n | A_1, \dots, A_{n-1}) P(A_{n-1} | A_1, \dots, A_{n-2}) P(A_{n-2} | A_1, \dots, A_{n-3}) P(A_1, \dots, A_{n-3}) \\ &= P(A_n | A_1, \dots, A_{n-1}) P(A_{n-1} | A_1, \dots, A_{n-2}) \cdot \dots \cdot P(A_2 | A_1) P(A_1) \\ &= P(A_n | A_{1:n-1}) P(A_{n-1} | A_{1:n-2}) \cdot \dots \cdot P(A_2 | A_1) P(A_1) \\ &= \prod_{i=1}^n P(A_i | A_{1:i-1}) \end{aligned}$$

Machine Translation

- What loss function?
 - Negative Log-Likelihood (NLL) loss for sentence prediction $y = (t_1, t_2, \dots, t_L)$.

$$L(W) = -\frac{1}{N} \sum_{n=1}^N \log P_W \left(t_1 = t_1^{(n)}, t_2 = t_2^{(n)}, \dots, t_{L_n} = t_{L_n}^{(n)} \mid \mathbf{x}_n \right)$$

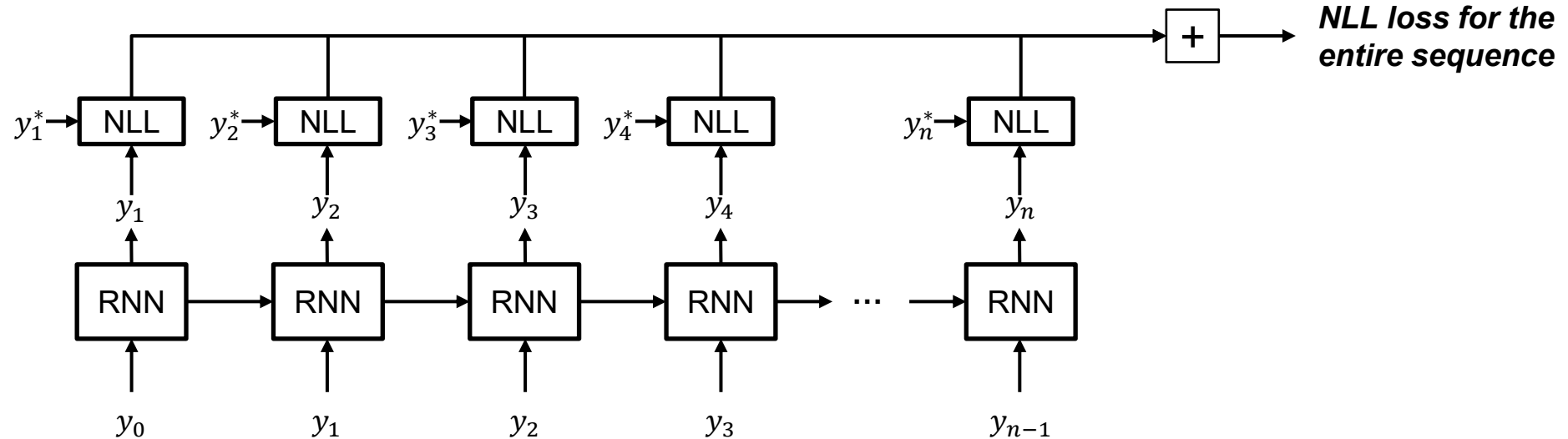
$$L(W) = -\frac{1}{N} \sum_{n=1}^N \log \prod_{i=1}^{L_n} P_W \left(t_i = t_i^{(n)} \mid t_{1:i-1}, \mathbf{x}_n \right)$$

$$L(W) = -\frac{1}{N} \sum_{n=1}^N \sum_{i=1}^{L_n} \log P_W \left(t_i = t_i^{(n)} \mid t_{1:i-1}, \mathbf{x}_n \right)$$

Estimated probability of
GT token at each timestep.

Machine Translation

- NLL loss computation with RNNs



Attention for Relevant Context Modeling

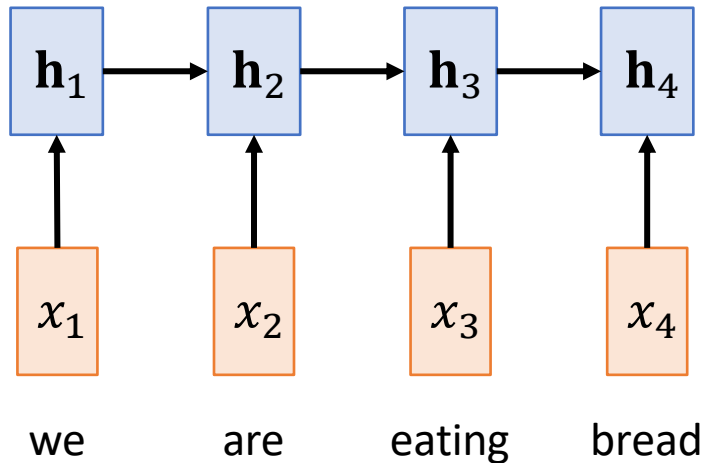
Sequence-to-Sequence with RNNs

- Recall machine translation with seq2seq modeling

Input: sequence $x_1, \dots, x_T$

Output: sequence $y_1, \dots, y_{T'}$

Encoder: $\mathbf{h}_t = f_W(x_t, \mathbf{h}_{t-1})$



Sutskever et al, Sequence to sequence learning with neural networks, NeurIPS 2014

Sequence-to-Sequence with RNNs

- Recall machine translation with seq2seq modeling

Input: sequence $x_1, \dots, x_T$

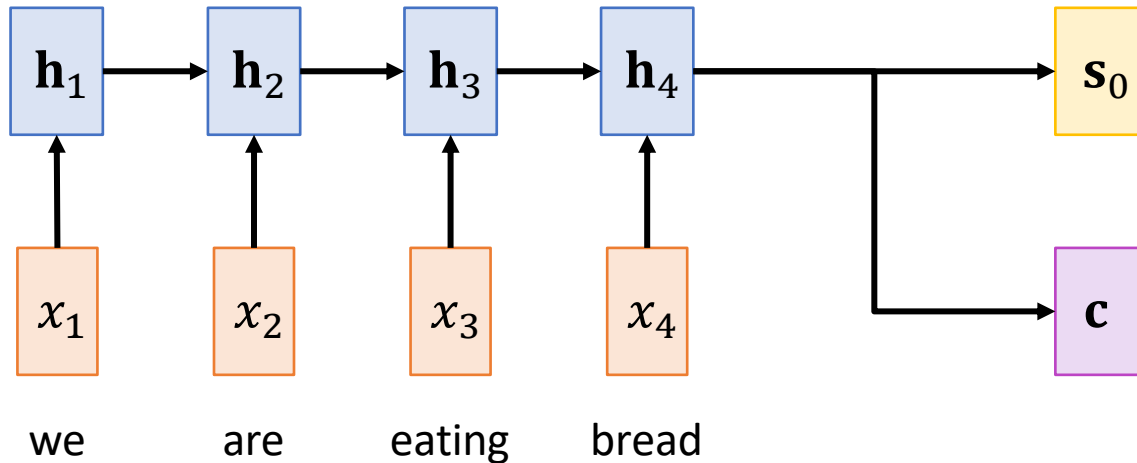
Output: sequence $y_1, \dots, y_{T'}$

Encoder: $\mathbf{h}_t = f_W(x_t, \mathbf{h}_{t-1})$

From final hidden state predict:

Initial decoder state s_0

Context vector c (often $c = \mathbf{h}_T$)



Sutskever et al, Sequence to sequence learning with neural networks, NeurIPS 2014

Sequence-to-Sequence with RNNs

- Recall machine translation with seq2seq modeling

Input: sequence $x_1, \dots, x_T$

Output: sequence $y_1, \dots, y_{T'}$

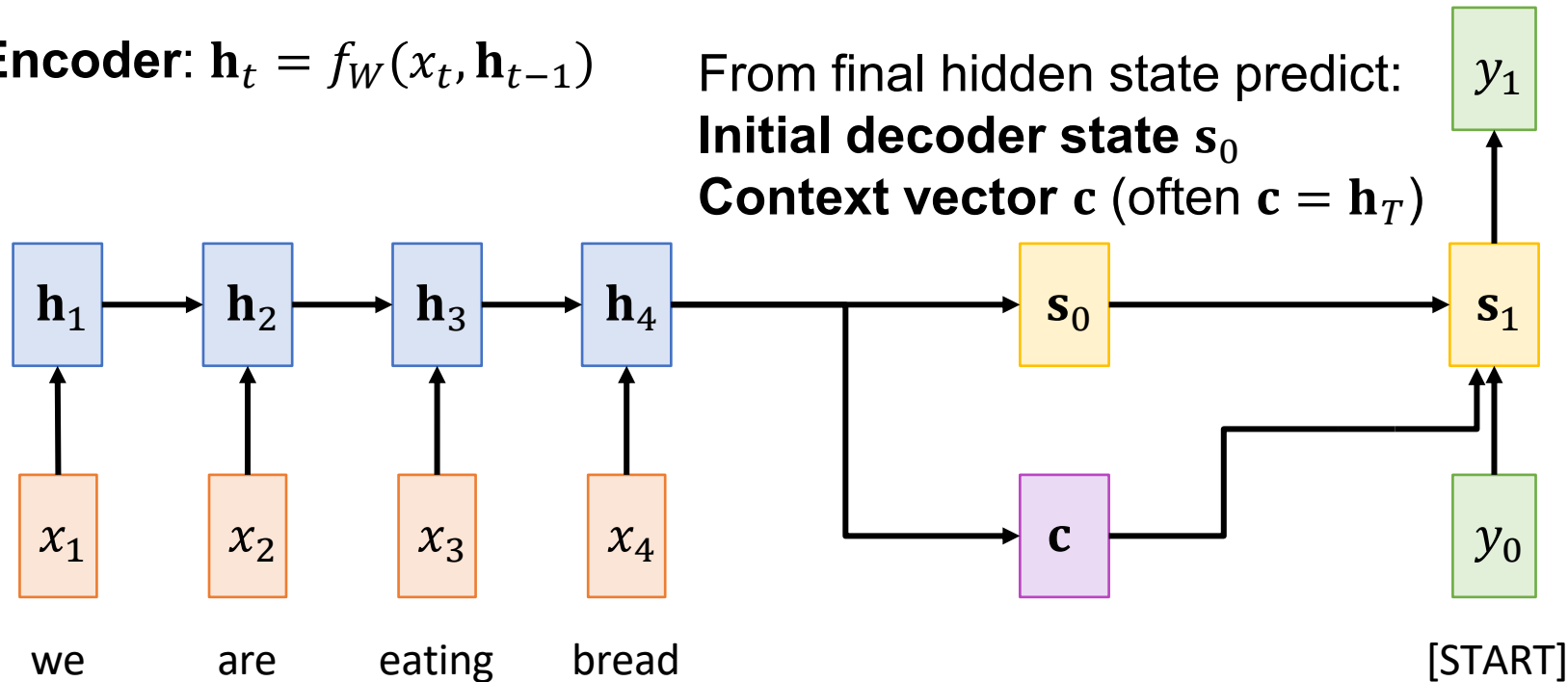
Decoder: $s_t = g_U(y_{t-1}, s_{t-1}, c)$ estamos

Encoder: $h_t = f_W(x_t, h_{t-1})$

From final hidden state predict:

Initial decoder state s_0

Context vector c (often $c = h_T$)



Sutskever et al, Sequence to sequence learning with neural networks, NeurIPS 2014

Sequence-to-Sequence with RNNs

- Recall machine translation with seq2seq modeling

Input: sequence $x_1, \dots, x_T$

Output: sequence $y_1, \dots, y_{T'}$

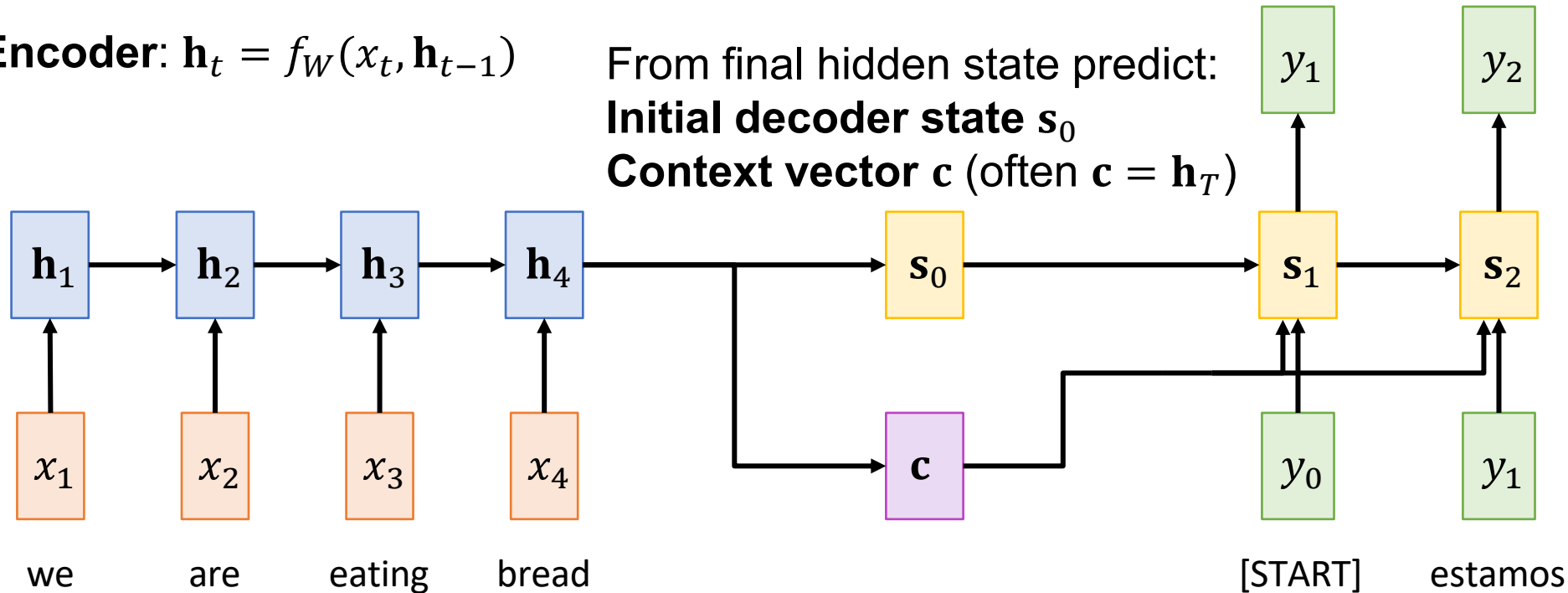
Decoder: $s_t = g_U(y_{t-1}, s_{t-1}, c)$ estamos comiendo

Encoder: $h_t = f_W(x_t, h_{t-1})$

From final hidden state predict:

Initial decoder state s_0

Context vector c (often $c = h_T$)



Sutskever et al, Sequence to sequence learning with neural networks, NeurIPS 2014

Sequence-to-Sequence with RNNs

- Recall machine translation with seq2seq modeling

Input: sequence $x_1, \dots, x_T$

Output: sequence $y_1, \dots, y_{T'}$

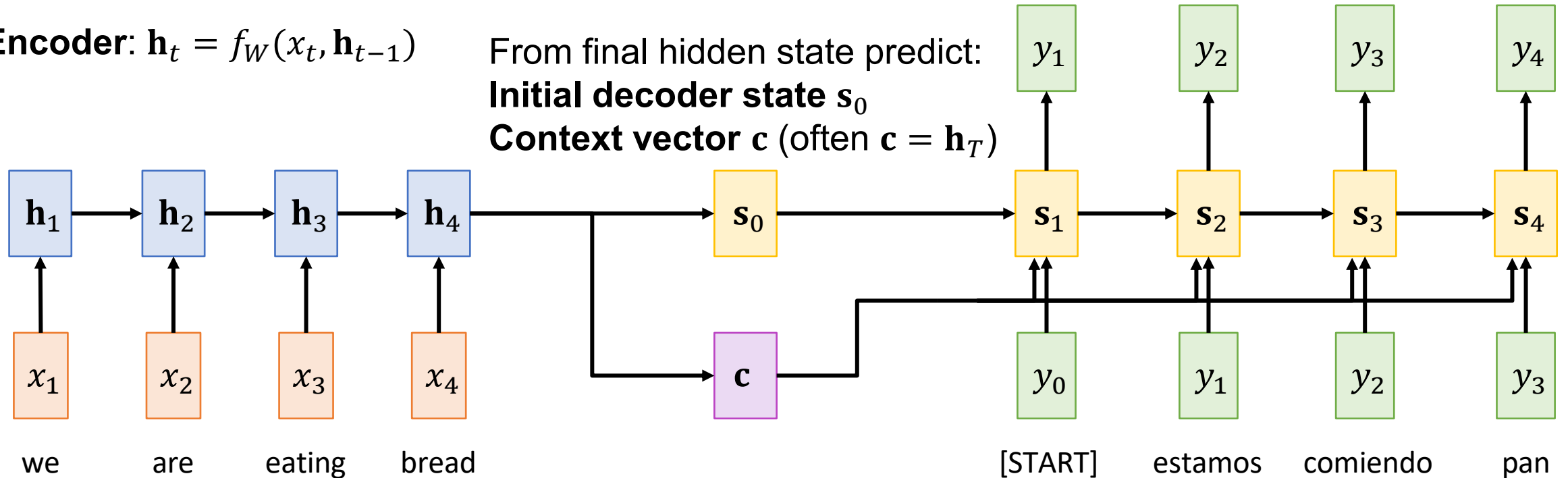
Decoder: $s_t = g_U(y_{t-1}, s_{t-1}, c)$ estamos comiendo pan [STOP]

Encoder: $h_t = f_W(x_t, h_{t-1})$

From final hidden state predict:

Initial decoder state s_0

Context vector c (often $c = h_T$)



Sutskever et al, Sequence to sequence learning with neural networks, NeurIPS 2014

Sequence-to-Sequence with RNNs

- Recall machine translation with seq2seq modeling

Input: sequence $x_1, \dots, x_T$

Output: sequence $y_1, \dots, y_{T'}$

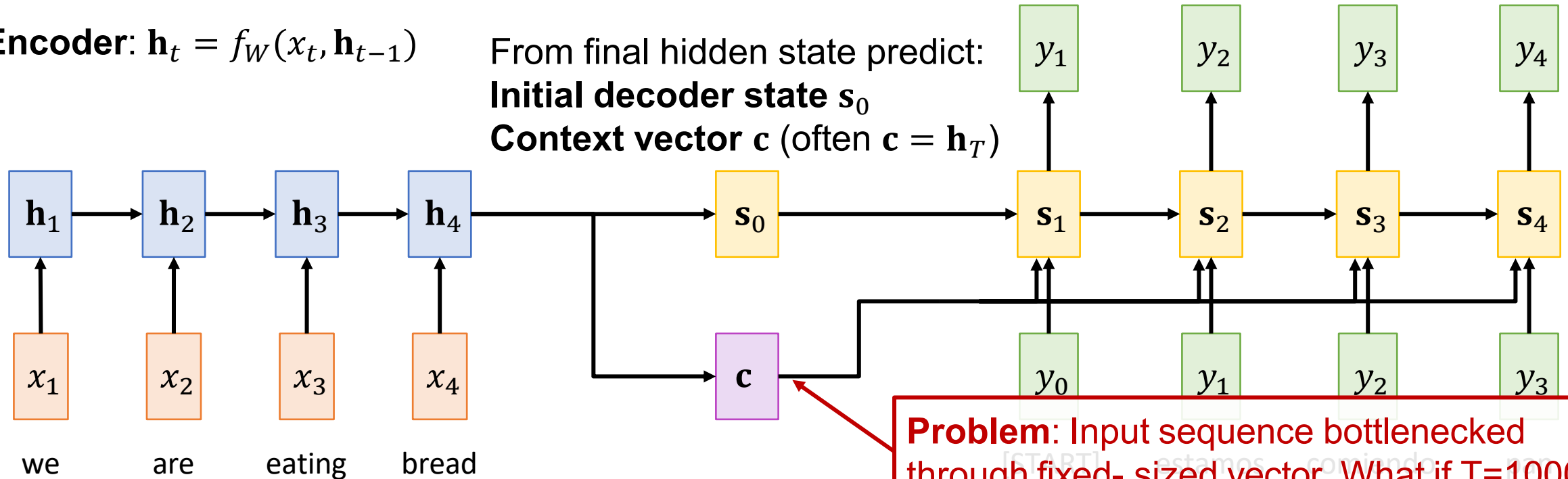
Decoder: $s_t = g_U(y_{t-1}, s_{t-1}, c)$ estamos comiendo pan [STOP]

Encoder: $h_t = f_W(x_t, h_{t-1})$

From final hidden state predict:

Initial decoder state s_0

Context vector c (often $c = h_T$)



Problem: Input sequence bottlenecked through fixed-sized vector. What if $T=1000$?

Sutskever et al, Sequence to sequence learning with neural networks, NeurIPS 2014

Sequence-to-Sequence with RNNs

- Recall machine translation with seq2seq modeling

Input: sequence $x_1, \dots, x_T$

Output: sequence $y_1, \dots, y_{T'}$

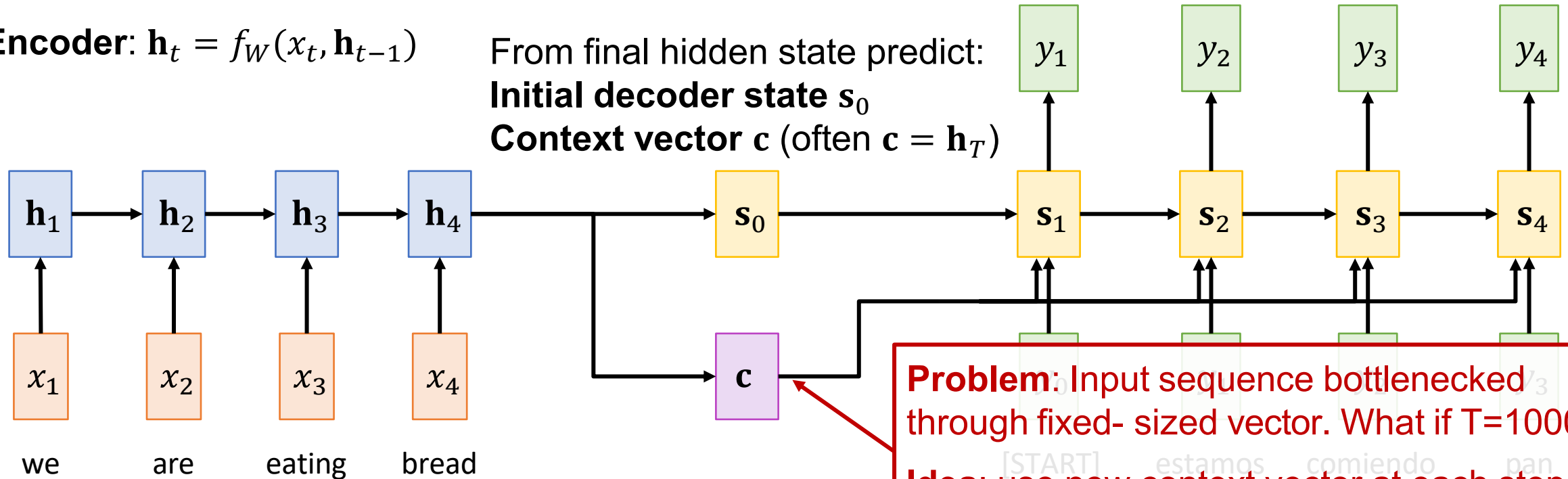
Decoder: $s_t = g_U(y_{t-1}, s_{t-1}, c)$ estamos comiendo pan [STOP]

Encoder: $h_t = f_W(x_t, h_{t-1})$

From final hidden state predict:

Initial decoder state s_0

Context vector c (often $c = h_T$)



Sutskever et al, Sequence to sequence learning with neural networks, NeurIPS 2014

Sequence-to-Sequence with RNNs and Attention

- Recall machine translation with seq2seq modeling

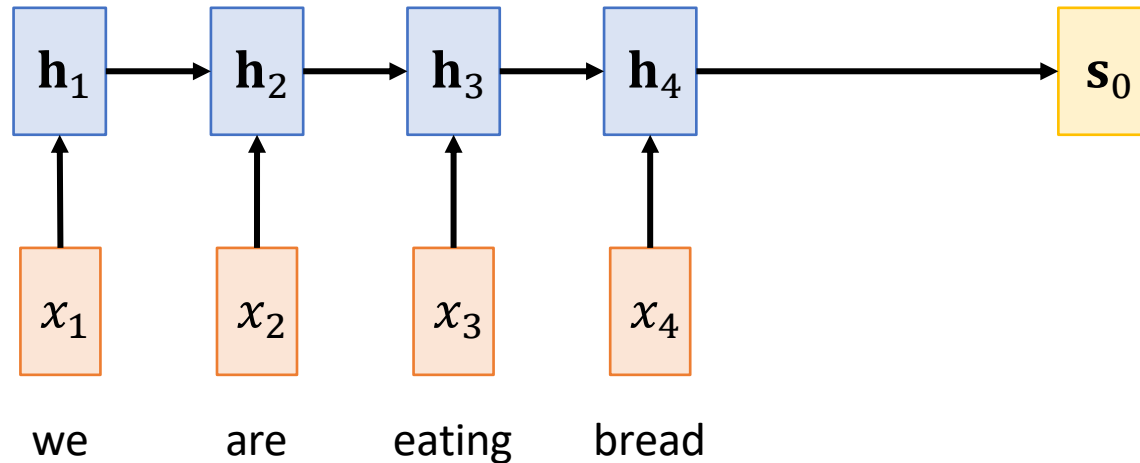
Input: sequence $x_1, \dots, x_T$

Output: sequence $y_1, \dots, y_{T'}$

Encoder: $\mathbf{h}_t = f_W(x_t, \mathbf{h}_{t-1})$

From final hidden state predict:

Initial decoder state s_0



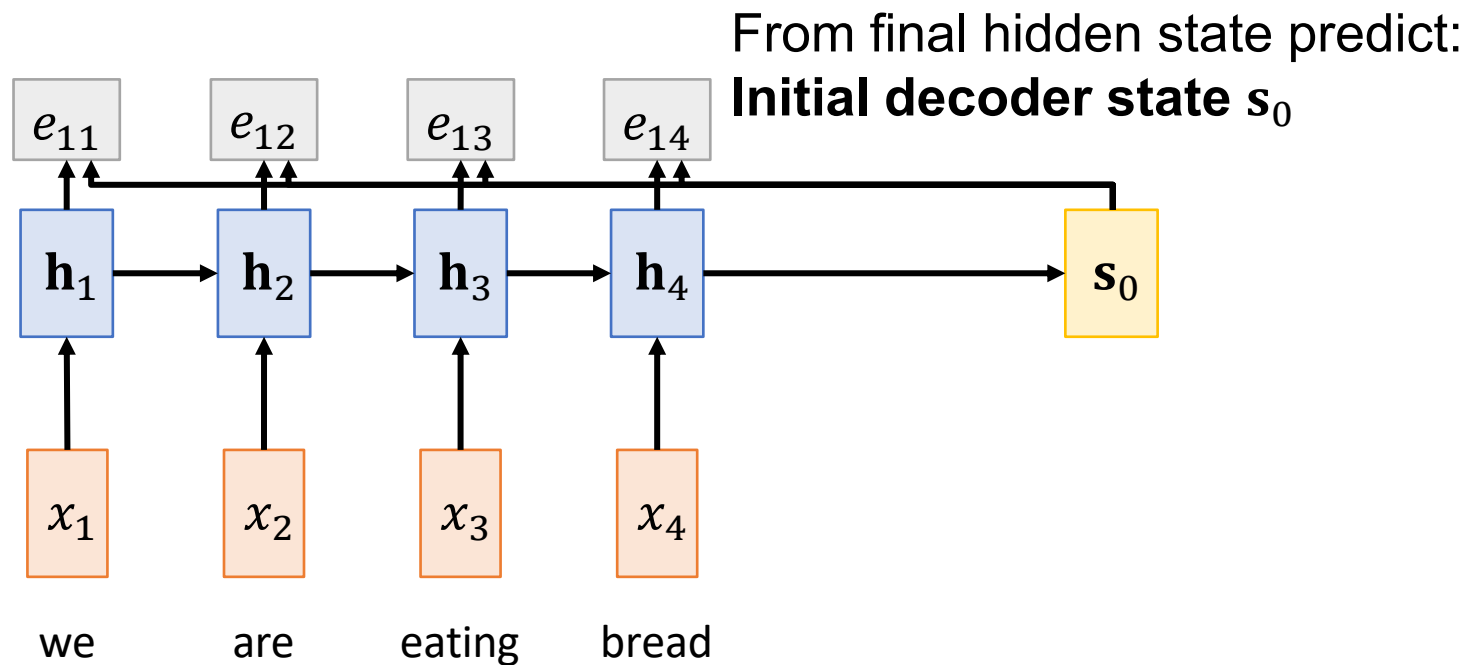
Bahdanau et. al., Neural machine translation by jointly learning to align and translate, ICLR 2015

Sequence-to-Sequence with RNNs and Attention

- Recall machine translation with seq2seq modeling

Compute (scalar) **alignment scores**

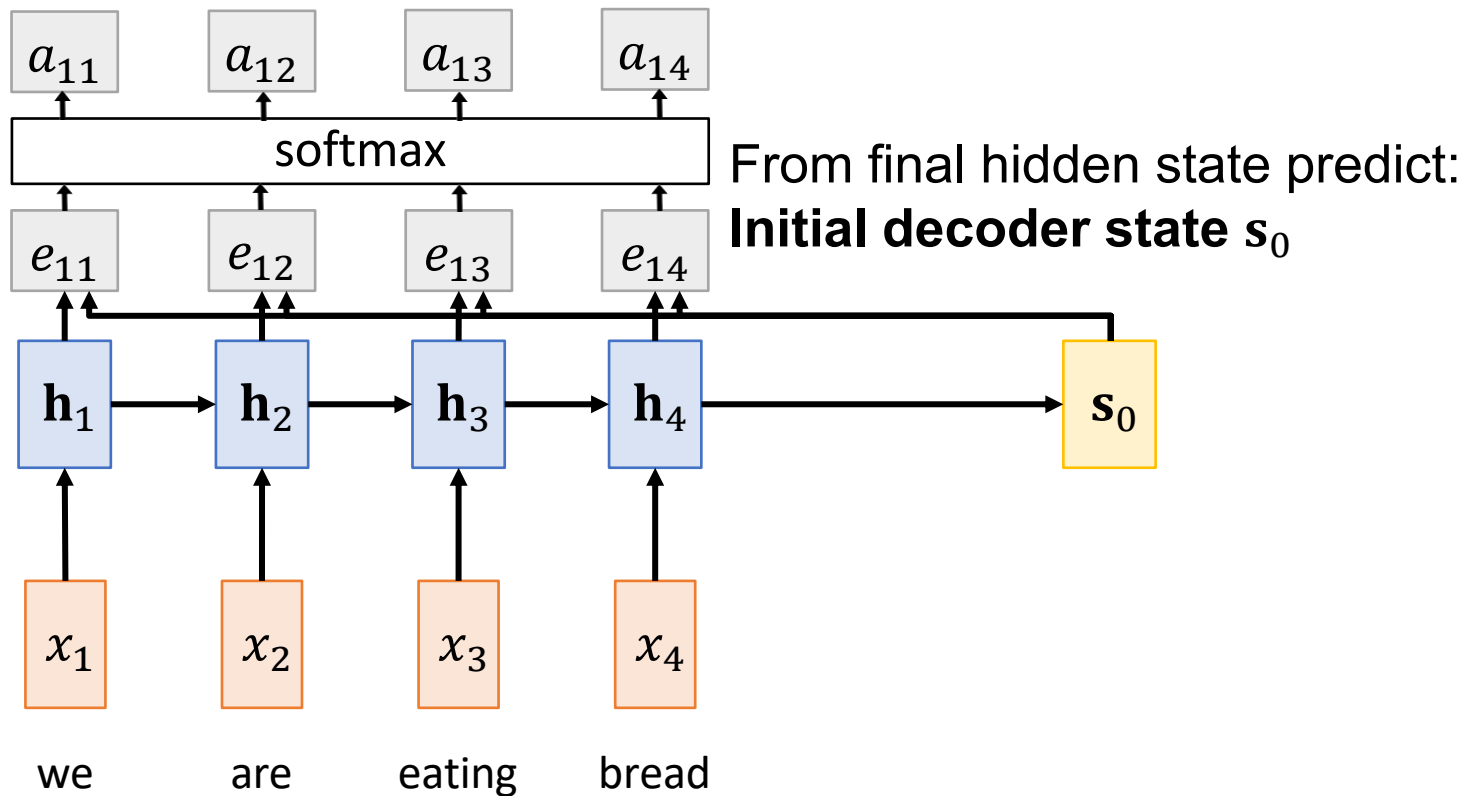
$$e_{ti} = f_{\text{att}}(\mathbf{s}_{t-1}, \mathbf{h}_i)$$



Bahdanau et. al., Neural machine translation by jointly learning to align and translate, ICLR 2015

Sequence-to-Sequence with RNNs and Attention

- Recall machine translation with seq2seq modeling



Compute (scalar) **alignment scores**

$$e_{ti} = f_{\text{att}}(s_{t-1}, h_i)$$

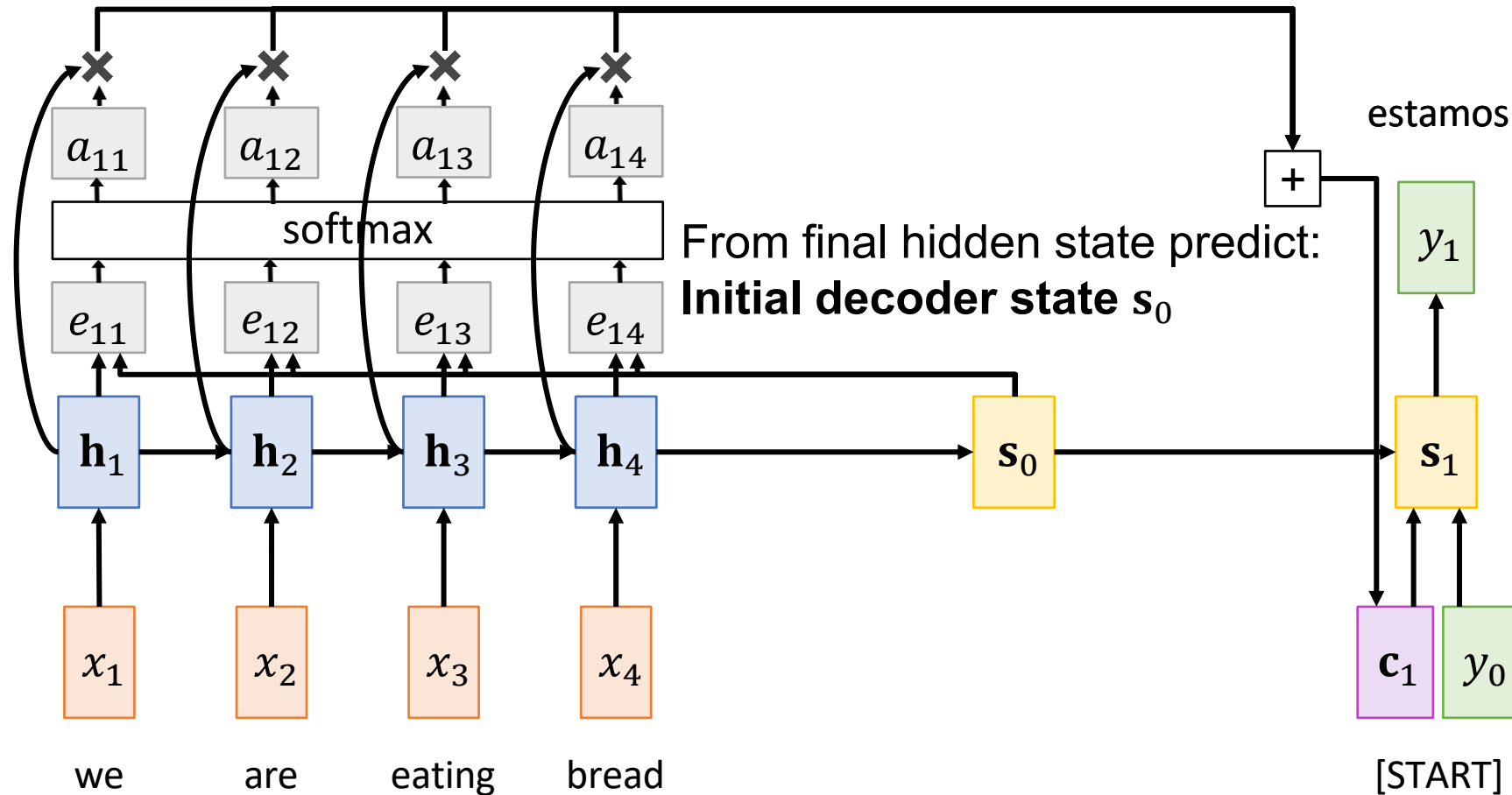
Normalize alignment scores to get **attention weights**

$$0 \leq \alpha_{ti} \leq 1, \quad \sum_i \alpha_{ti} = 1$$

Bahdanau et. al., Neural machine translation by jointly learning to align and translate, ICLR 2015

Sequence-to-Sequence with RNNs and Attention

- Recall machine translation with seq2seq modeling



Compute (scalar) **alignment scores**

$$e_{ti} = f_{\text{att}}(s_{t-1}, h_i)$$

Normalize alignment scores to get **attention weights**

$$0 \leq \alpha_{ti} \leq 1, \quad \sum_i \alpha_{ti} = 1$$

Compute context vector as **linear combination of hidden states**

$$c_t = \sum_i \alpha_{ti} h_i$$

Use context vector in decoder

$$s_t = g_U(y_{t-1}, s_{t-1}, c_t)$$

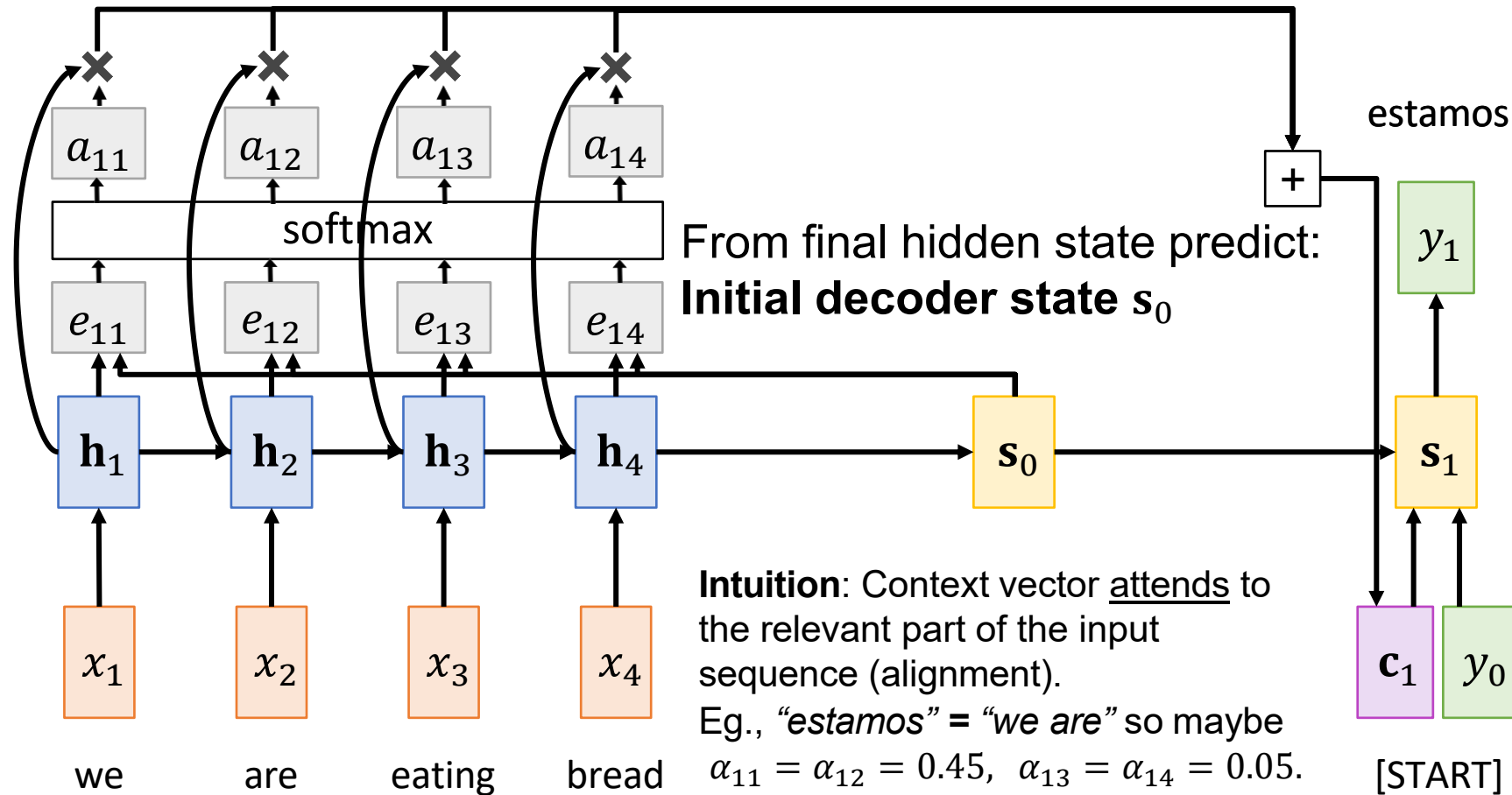
This is all **differentiable!**

Do not supervise attention weights
– backprop through everything.

Bahdanau et. al., Neural machine translation by jointly learning to align and translate, ICLR 2015

Sequence-to-Sequence with RNNs and Attention

- Recall machine translation with seq2seq modeling



Compute (scalar) **alignment scores**

$$e_{ti} = f_{\text{att}}(s_{t-1}, h_i)$$

Normalize alignment scores to get **attention weights**

$$0 \leq \alpha_{ti} \leq 1, \quad \sum_i \alpha_{ti} = 1$$

Compute context vector as **linear combination of hidden states**

$$c_t = \sum_i \alpha_{ti} h_i$$

Use context vector in decoder

$$s_t = g_U(y_{t-1}, s_{t-1}, c_t)$$

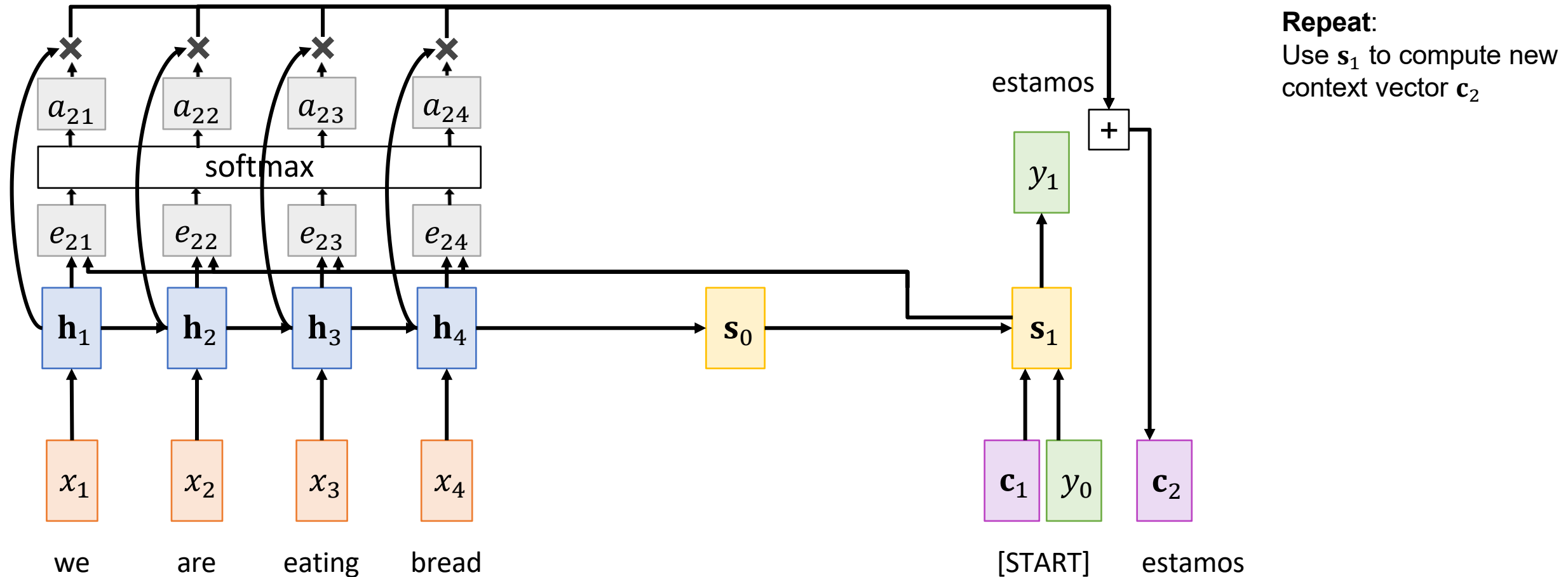
This is all **differentiable!**

Do not supervise attention weights
– backprop through everything.

Bahdanau et. al., Neural machine translation by jointly learning to align and translate, ICLR 2015

Sequence-to-Sequence with RNNs and Attention

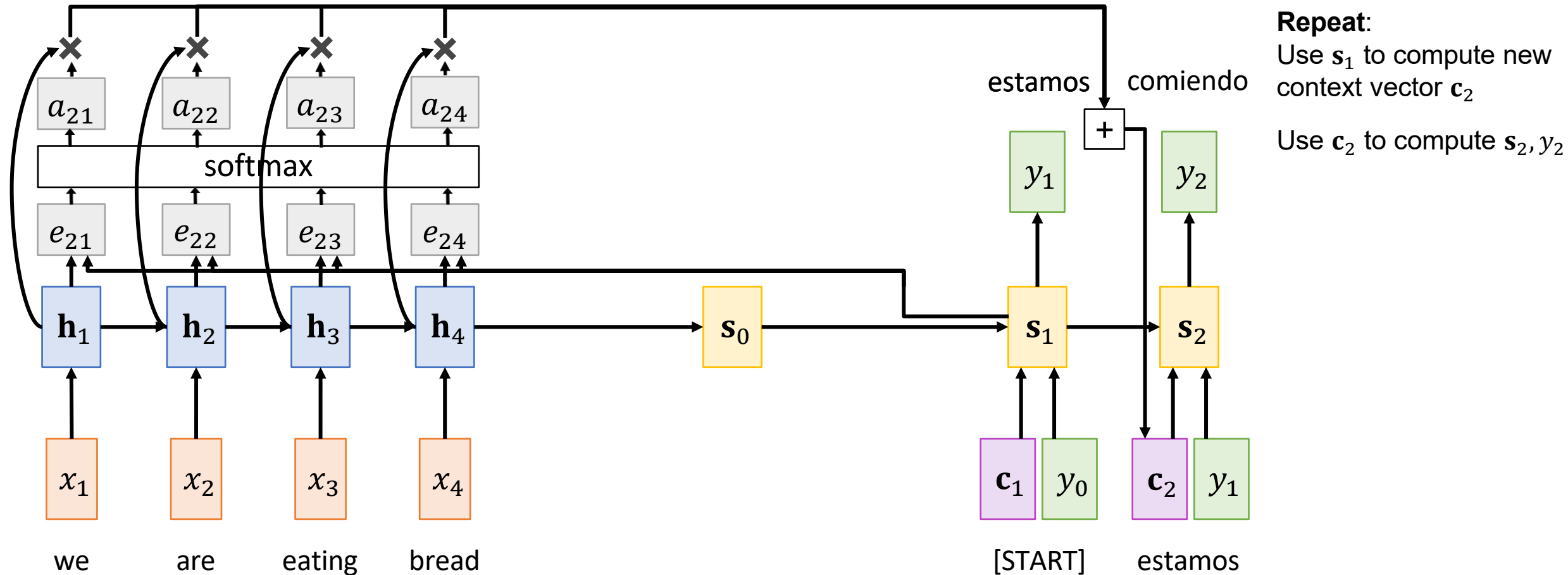
- Recall machine translation with seq2seq modeling



Bahdanau et. al., Neural machine translation by jointly learning to align and translate, ICLR 2015

Sequence-to-Sequence with RNNs and Attention

- Recall machine translation with seq2seq modeling



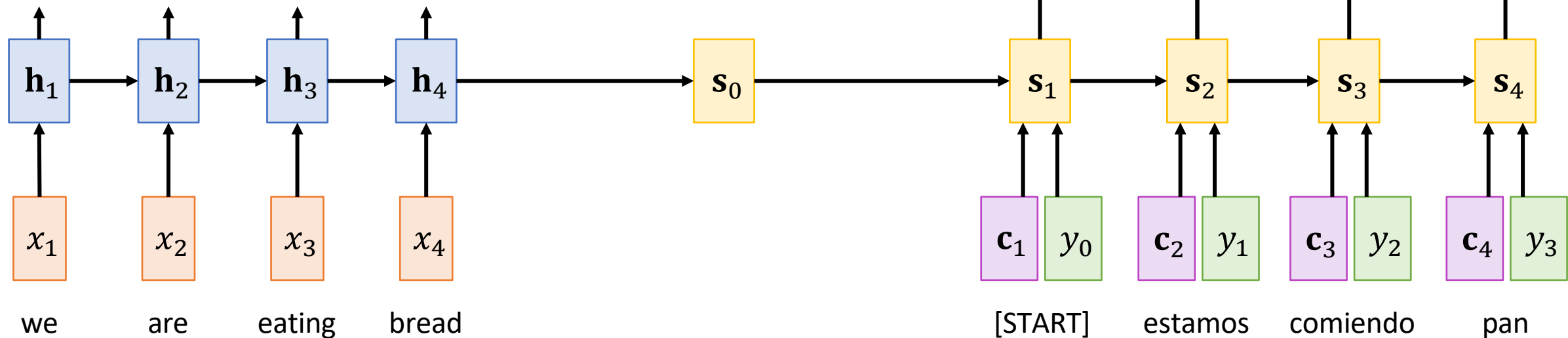
Bahdanau et. al., Neural machine translation by jointly learning to align and translate, ICLR 2015

Sequence-to-Sequence with RNNs and Attention

- Recall machine translation with seq2seq modeling

Use a different context vector in each timestep of decoder

- Input sequence not bottlenecked through a single vector
- At each timestep of decoder, context vector “looks at” different parts of the input sequence



Bahdanau et. al., Neural machine translation by jointly learning to align and translate, ICLR 2015

Sequence-to-Sequence with RNNs and Attention

Example:

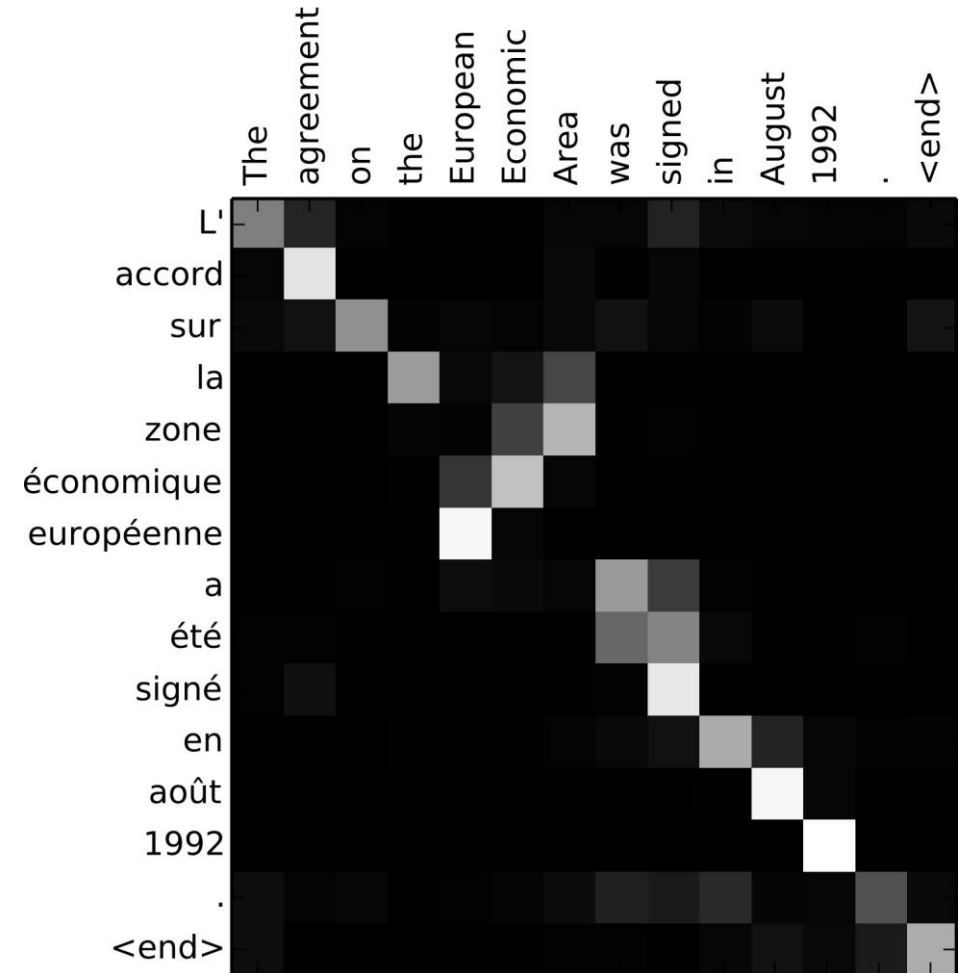
English to French translation

Input:

“The agreement on the European Economic Area was signed in August 1992.”

Output: “L’accord sur la zone économique européenne a été signé en août 1992.”

Visualization of attention weights



Bahdanau et. al., Neural machine translation by jointly learning to align and translate, ICLR 2015

Sequence-to-Sequence with RNNs and Attention

Example:

English to French translation

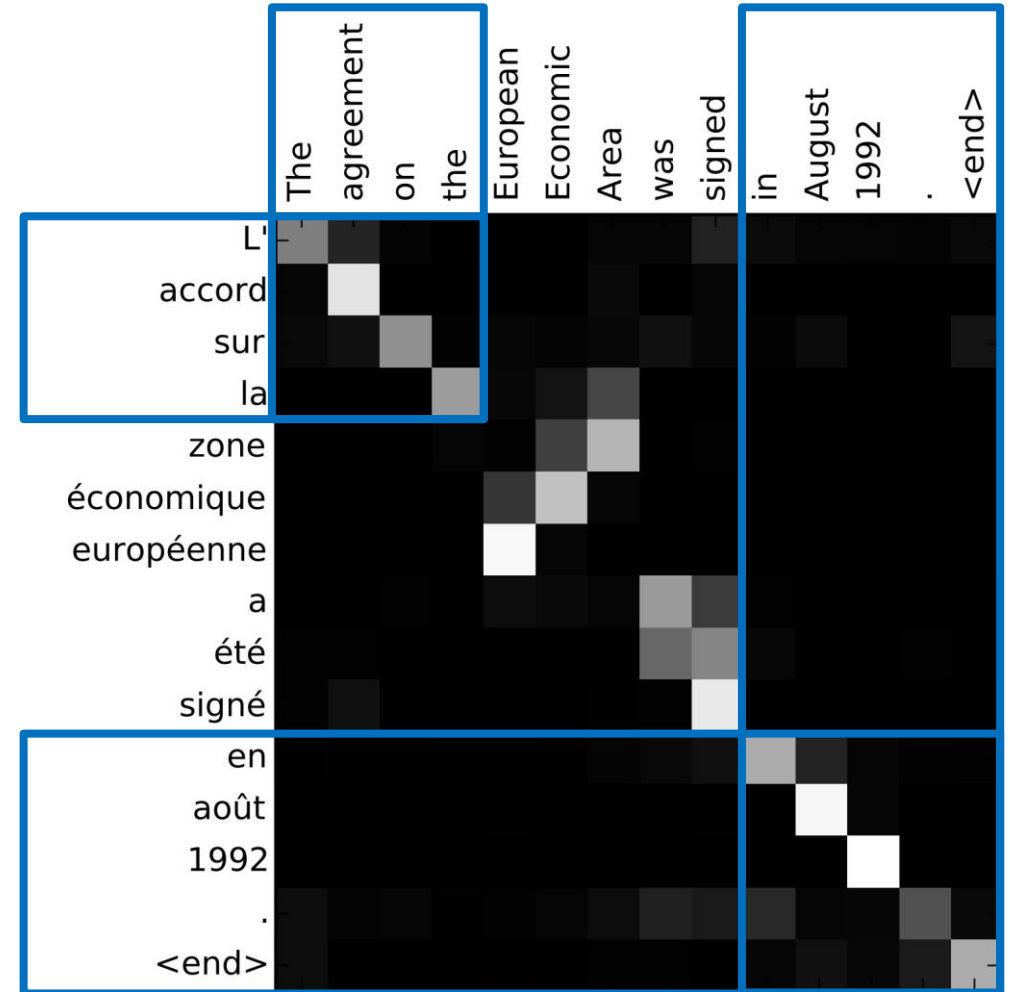
Input:

“The agreement on the European Economic Area was signed in August 1992.”

Output: “L’accord sur la zone économique européenne a été signé en août 1992.”

Diagonal attention means words correspond in order

Visualization of attention weights



Bahdanau et. al., Neural machine translation by jointly learning to align and translate, ICLR 2015

Sequence-to-Sequence with RNNs and Attention

Example:

English to French translation

Input:

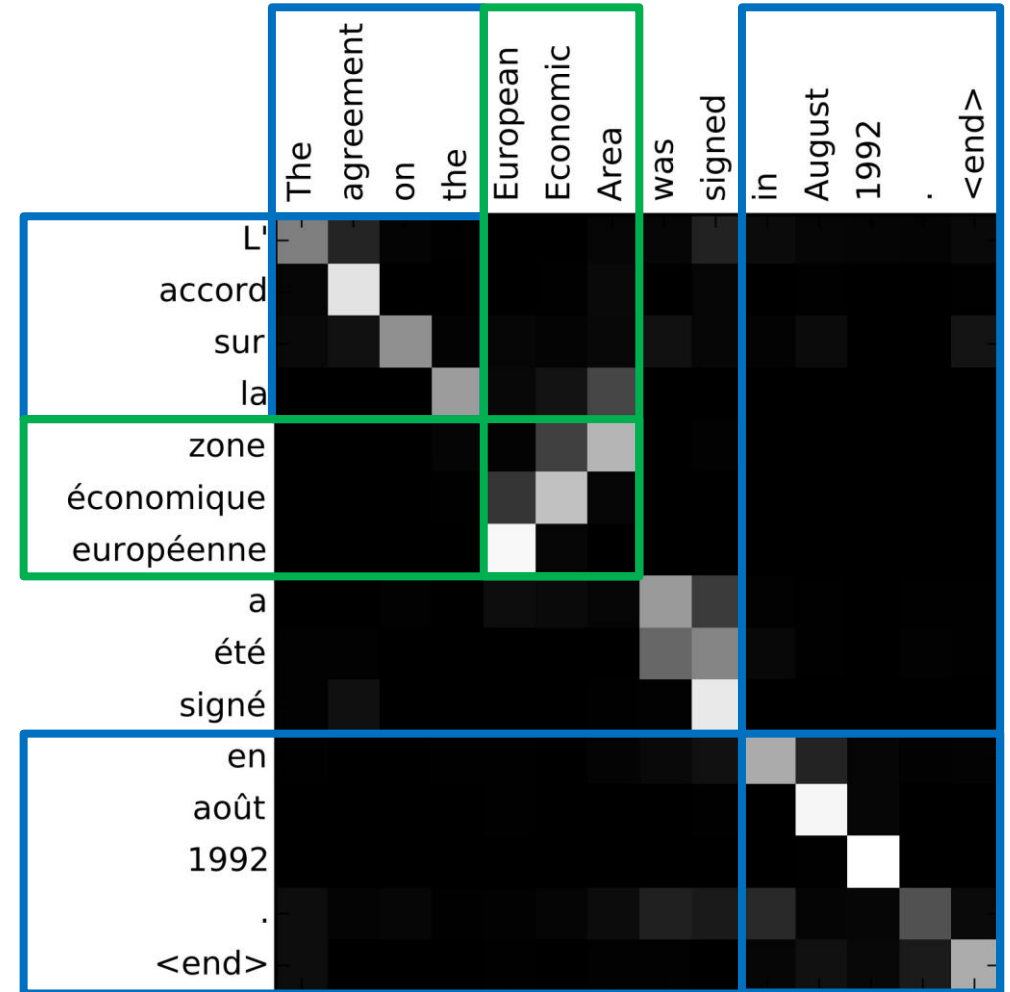
“The agreement on the European Economic Area was signed in August 1992.”

Output: “L’accord sur la zone économique européenne a été signé en août 1992.”

Diagonal attention means words correspond in order

Attention figures out different word orders

Visualization of attention weights



Bahdanau et. al., Neural machine translation by jointly learning to align and translate, ICLR 2015

Sequence-to-Sequence with RNNs and Attention

Example:

English to French translation

Input:

“The agreement on the European Economic Area was signed in August 1992.”

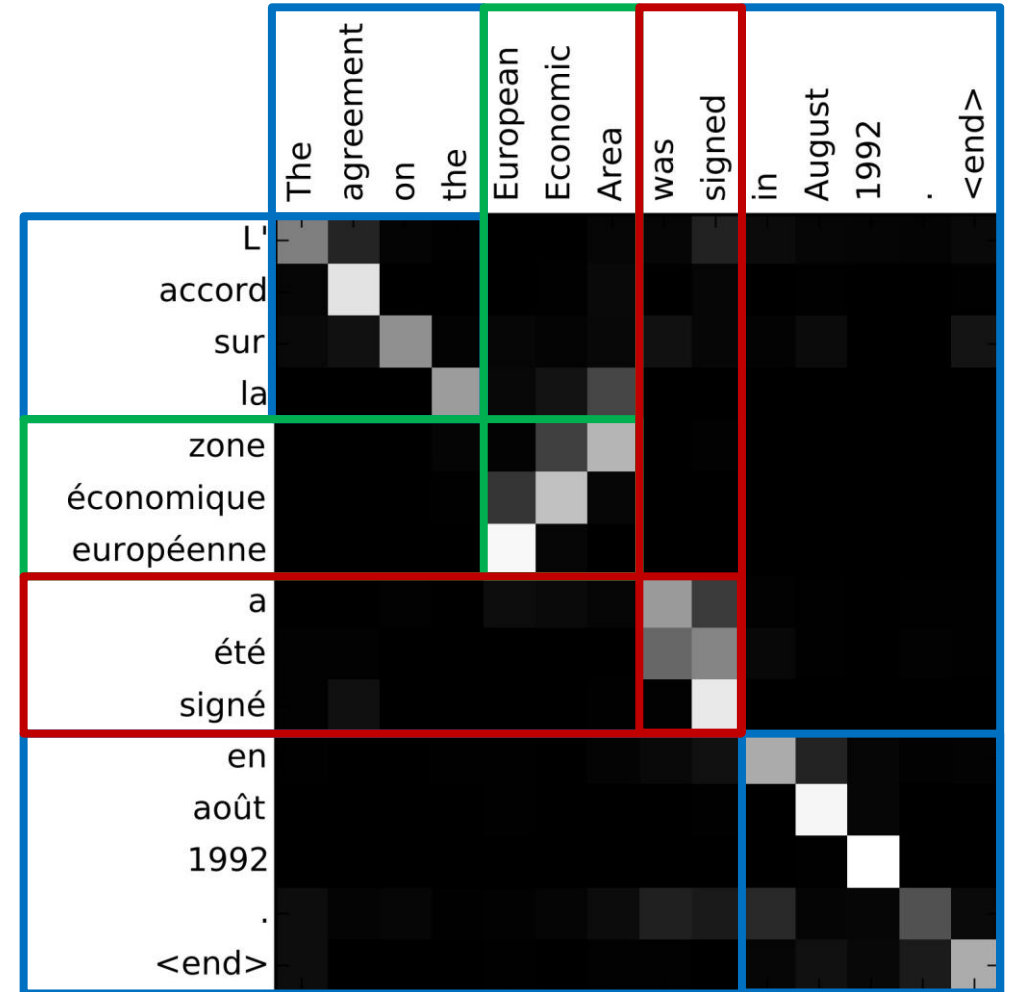
Output: “L’accord sur la zone économique européenne a été signé en août 1992.”

Diagonal attention means words correspond in order

Attention figures out different word orders

Verb conjugation

Visualization of attention weights



Bahdanau et. al., Neural machine translation by jointly learning to align and translate, ICLR 2015

Transformers

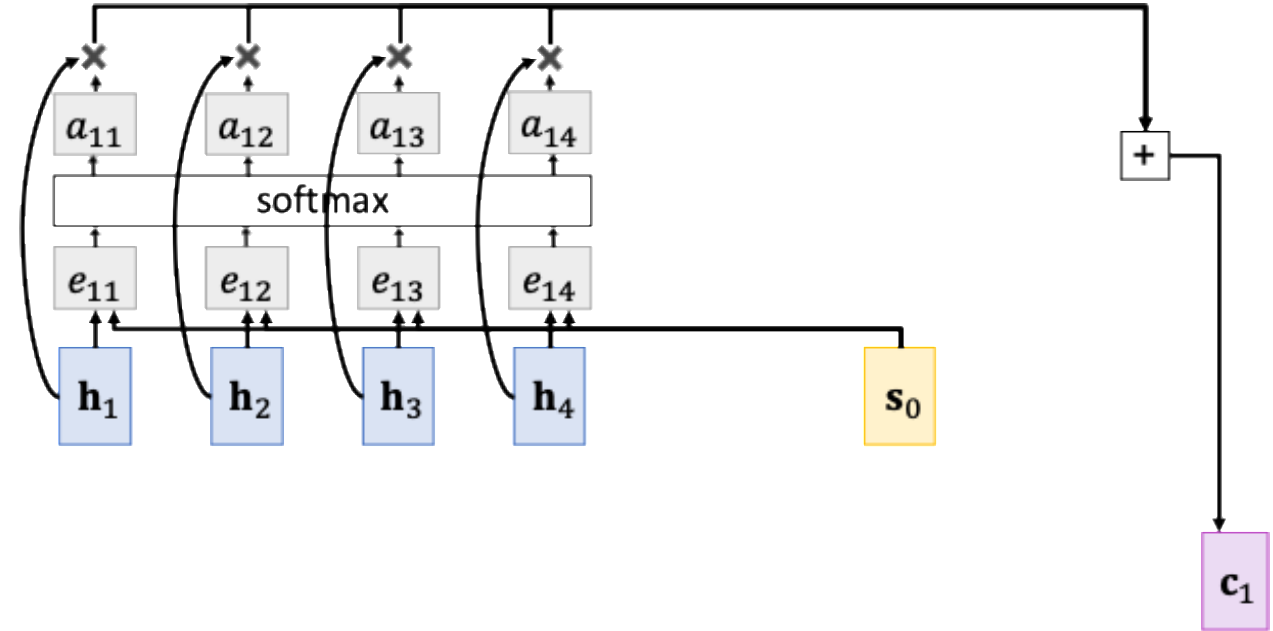
Attention Layer

Inputs:

Query vector: $\mathbf{q}$ (shape: D_Q)

Input vectors: $\mathbf{X}$ (shape: $N_X \times D_Q$)

Similarity function: f_{att}



Computation:

Similarities: $\mathbf{e}$ (shape: N_X), $e_i = f_{\text{att}}(\mathbf{q}, \mathbf{x}_i)$

Attention weights: $\mathbf{a} = \text{softmax}(\mathbf{e})$ (shape: N_X)

Output vector: $\mathbf{y} = \sum_i a_i \mathbf{x}_i$ (shape: D_Q)

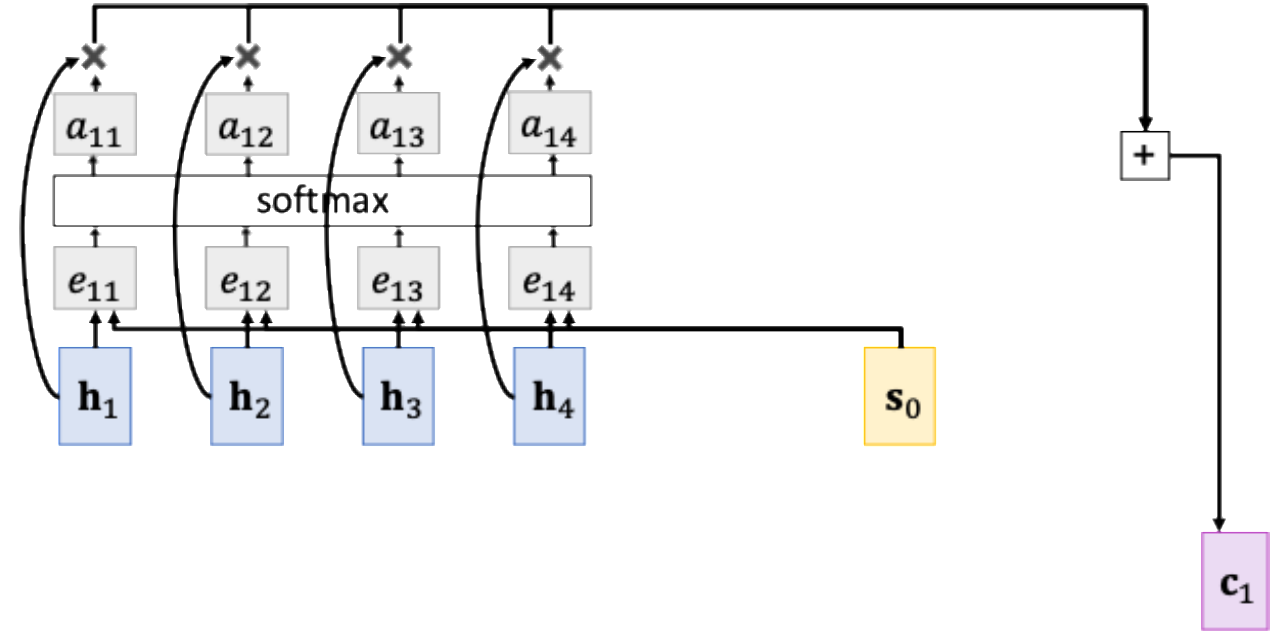
Attention Layer

Inputs:

Query vector: $\mathbf{q}$ (shape: D_Q)

Input vectors: $\mathbf{X}$ (shape: $N_X \times D_Q$)

Similarity function: dot product



Computation:

Similarities: $\mathbf{e}$ (shape: N_X) $e_i = \mathbf{q} \cdot \mathbf{x}_i$

Attention weights: $\mathbf{a} = \text{softmax}(\mathbf{e})$ (shape: N_X)

Output vector: $\mathbf{y} = \sum_i a_i \mathbf{x}_i$ (shape: D_Q)

Changes:

- Use dot product for similarity

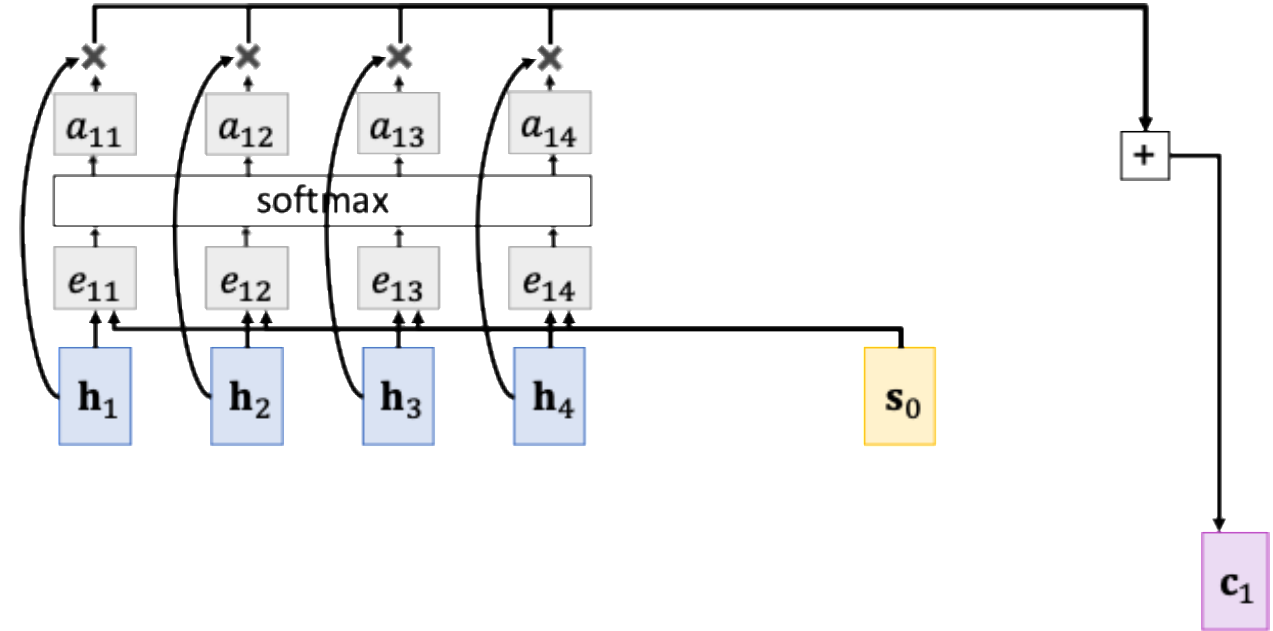
Attention Layer

Inputs:

Query vector: $\mathbf{q}$ (shape: D_Q)

Input vectors: $\mathbf{X}$ (shape: $N_X \times D_Q$)

Similarity function: scaled dot product



Computation:

Similarities: $\mathbf{e}$ (shape: N_X) $e_i = \mathbf{q} \cdot \mathbf{x}_i / \sqrt{D_Q}$

Attention weights: $\mathbf{a} = \text{softmax}(\mathbf{e})$ (shape: N_X)

Output vector: $\mathbf{y} = \sum_i a_i \mathbf{x}_i$ (shape: D_Q)

Changes:

- Use **scaled** dot product for similarity

Attention Layer

Inputs:

Query vector: $\mathbf{q}$ (shape: D_Q)

Input vectors: $\mathbf{X}$ (shape: $N_X \times D_Q$)

Similarity function: scaled dot product

Large similarities will cause softmax to saturate and give vanishing gradients

Recall $\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos(\angle \mathbf{ab})$ for $\mathbf{a}, \mathbf{b} \in D$ and thus $\mathbf{a} \cdot \mathbf{b}$ is large when D is large causing vanishing gradient.

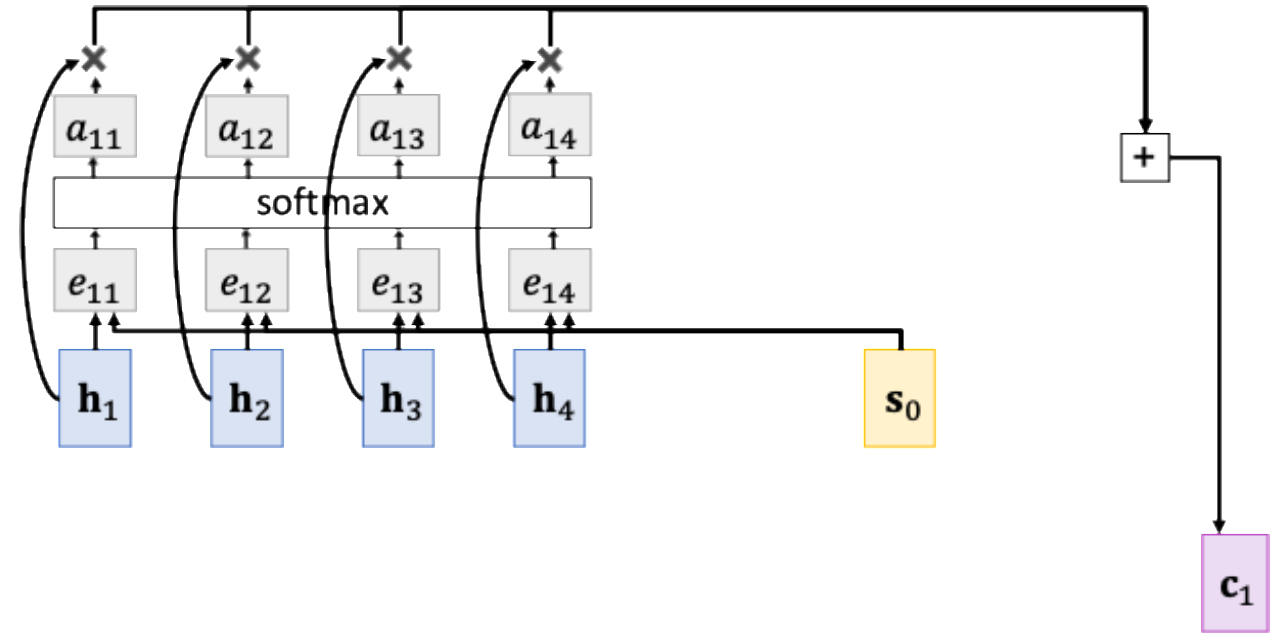
Scaling with $1/\sqrt{D}$ will result in outputs with 0 mean and unit variance given normalized inputs.

Computation:

Similarities: $\mathbf{e}$ (shape: N_X) $e_i = \mathbf{q} \cdot \mathbf{x}_i / \sqrt{D_Q}$

Attention weights: $\mathbf{a} = \text{softmax}(\mathbf{e})$ (shape: N_X)

Output vector: $\mathbf{y} = \sum_i a_i \mathbf{x}_i$ (shape: D_Q)



Changes:

- Use **scaled** dot product for similarity

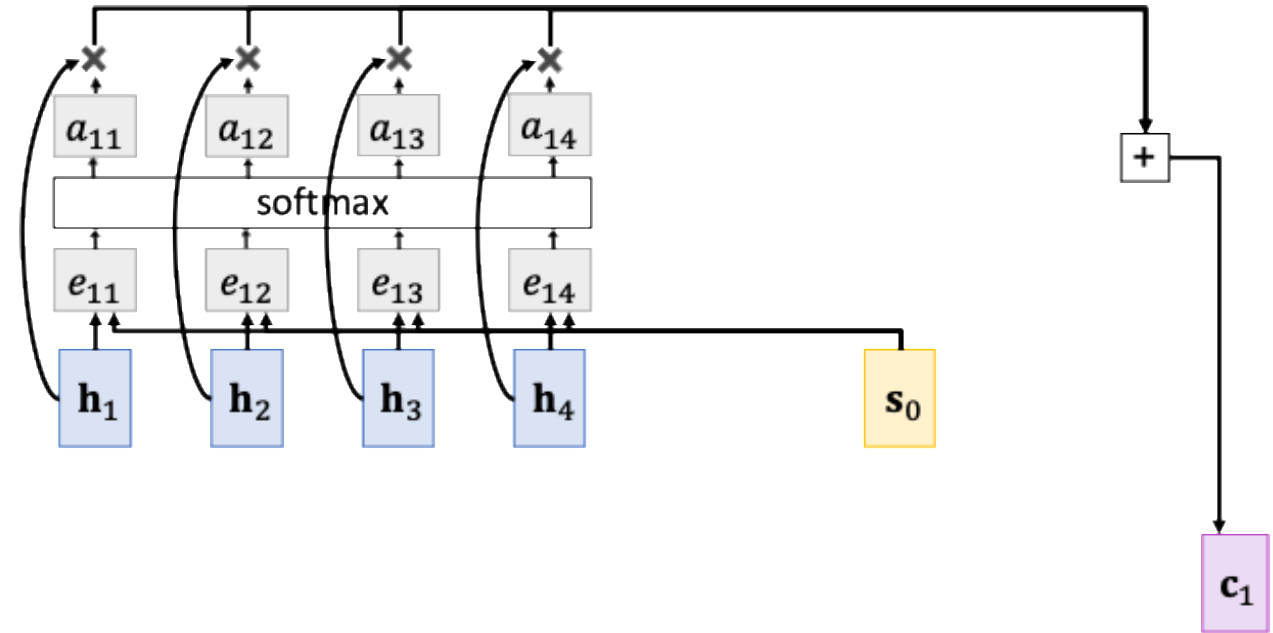
Attention Layer

Inputs:

Query vectors Q (shape: $N_Q \times D_Q$)

Input vectors: X (shape: $N_X \times D_Q$)

Similarity function: scaled dot product



Computation:

Similarities: $E = QX^T / \sqrt{D_Q}$ (shape: $N_Q \times N_X$), $e_{ij} = \mathbf{q}_i \cdot \mathbf{x}_j / \sqrt{D_Q}$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ (shape: $N_Q \times N_X$)

Output vector: $Y = AX$ (shape: $N_Q \times D_Q$), $\mathbf{y}_i = \sum_j a_{ij} \mathbf{x}_j$

Changes:

- Use scaled dot product for similarity
- Multiple query vectors

Attention Layer

Inputs:

Query vectors: Q (shape: $N_Q \times D_Q$)

Input vectors: X (shape: $N_X \times D_X$)

Similarity function: scaled dot product

Key matrix: W_K (shape: $D_X \times D_Q$)

Value matrix: W_V (shape: $D_X \times D_V$)

Computation:

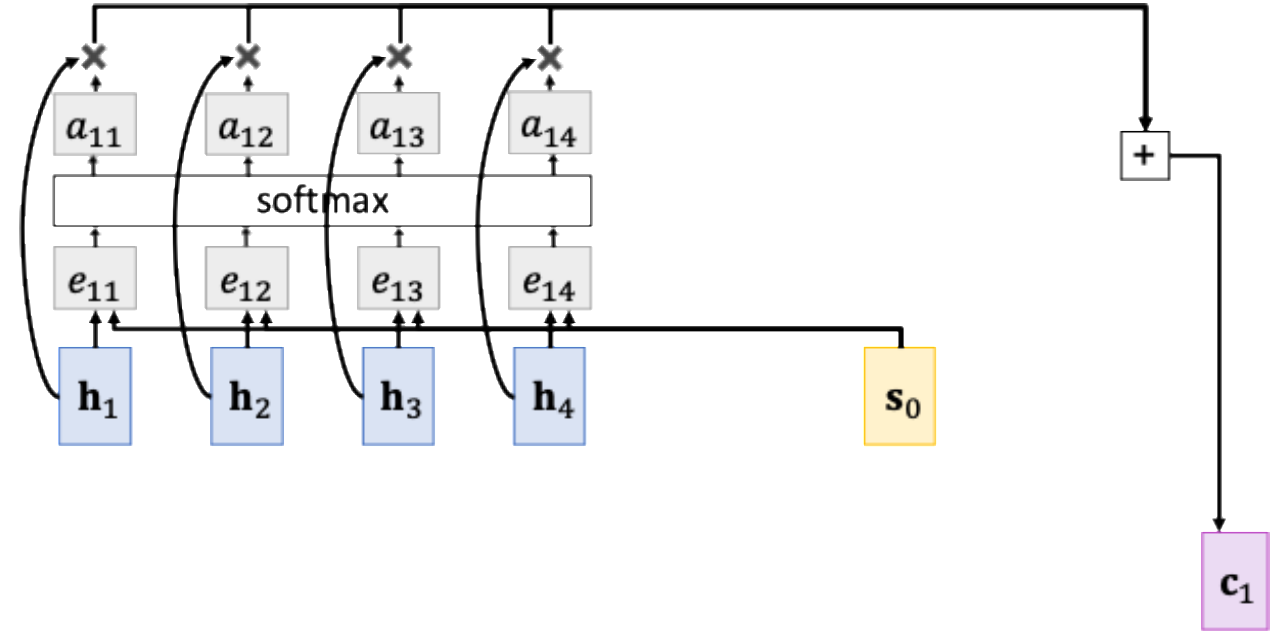
Key vectors: $K = XW_K$ (shape: $N_X \times D_Q$)

Value vectors: $V = XW_V$ (shape: $N_X \times D_V$)

Similarities: $E = QK / \sqrt{D_Q}$ (shape: $N_Q \times N_X$), $e_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D_Q}$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ (shape: $N_Q \times N_X$)

Output vector: $Y = AV$ (shape: $N_Q \times D_V$), $\mathbf{y}_i = \sum_j a_{ij} \mathbf{v}_j$



Changes:

- Use scaled dot product for similarity
- Multiple query vectors
- **Separate keys and values**

Attention Layer

Inputs:

Query vectors: Q (shape: $N_Q \times D_Q$)

Input vectors: X (shape: $N_X \times D_X$)

Similarity function: scaled dot product

Key matrix: W_K (shape: $D_X \times D_Q$)

Value matrix: W_V (shape: $D_X \times D_V$)

Computation:

Key vectors: $K = XW_K$ (shape: $N_X \times D_Q$)

Value vectors: $V = XW_V$ (shape: $N_X \times D_V$)

Similarities: $E = QK / \sqrt{D_Q}$ (shape: $N_Q \times N_X$), $e_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D_Q}$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ (shape: $N_Q \times N_X$)

Output vector: $Y = AV$ (shape: $N_Q \times D_V$), $\mathbf{y}_i = \sum_j a_{ij} \mathbf{v}_j$

$\mathbf{x}_3$
 $\mathbf{x}_2$
 $\mathbf{x}_1$

$\mathbf{q}_1$ $\mathbf{q}_2$ $\mathbf{q}_3$

Attention Layer

Inputs:

Query vectors: Q (shape: $N_Q \times D_Q$)

Input vectors: X (shape: $N_X \times D_X$)

Similarity function: scaled dot product

Key matrix: W_K (shape: $D_X \times D_Q$)

Value matrix: W_V (shape: $D_X \times D_V$)

Computation:

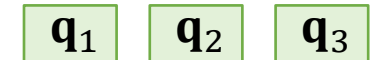
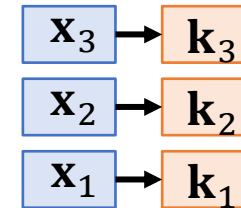
Key vectors: $K = XW_K$ (shape: $N_X \times D_Q$)

Value vectors: $V = XW_V$ (shape: $N_X \times D_V$)

Similarities: $E = QK / \sqrt{D_Q}$ (shape: $N_Q \times N_X$), $e_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D_Q}$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ (shape: $N_Q \times N_X$)

Output vector: $Y = AV$ (shape: $N_Q \times D_V$), $\mathbf{y}_i = \sum_j a_{ij} \mathbf{v}_j$



Attention Layer

Inputs:

Query vectors: Q (shape: $N_Q \times D_Q$)

Input vectors: X (shape: $N_X \times D_X$)

Similarity function: scaled dot product

Key matrix: W_K (shape: $D_X \times D_Q$)

Value matrix: W_V (shape: $D_X \times D_V$)

Computation:

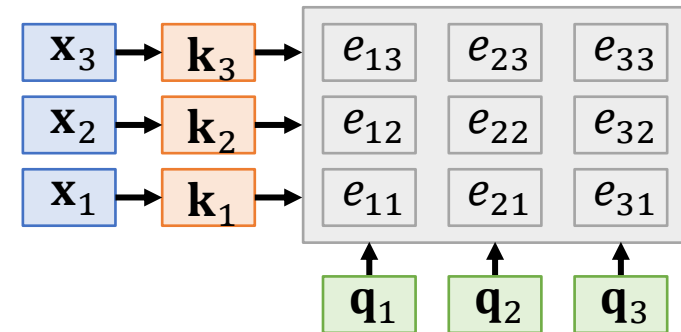
Key vectors: $K = XW_K$ (shape: $N_X \times D_Q$)

Value vectors: $V = XW_V$ (shape: $N_X \times D_V$)

Similarities: $E = QK / \sqrt{D_Q}$ (shape: $N_Q \times N_X$), $e_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D_Q}$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ (shape: $N_Q \times N_X$)

Output vector: $Y = AV$ (shape: $N_Q \times D_V$), $\mathbf{y}_i = \sum_j a_{ij} \mathbf{v}_j$



Attention Layer

Inputs:

Query vectors: Q (shape: $N_Q \times D_Q$)

Input vectors: X (shape: $N_X \times D_X$)

Similarity function: scaled dot product

Key matrix: W_K (shape: $D_X \times D_Q$)

Value matrix: W_V (shape: $D_X \times D_V$)

Computation:

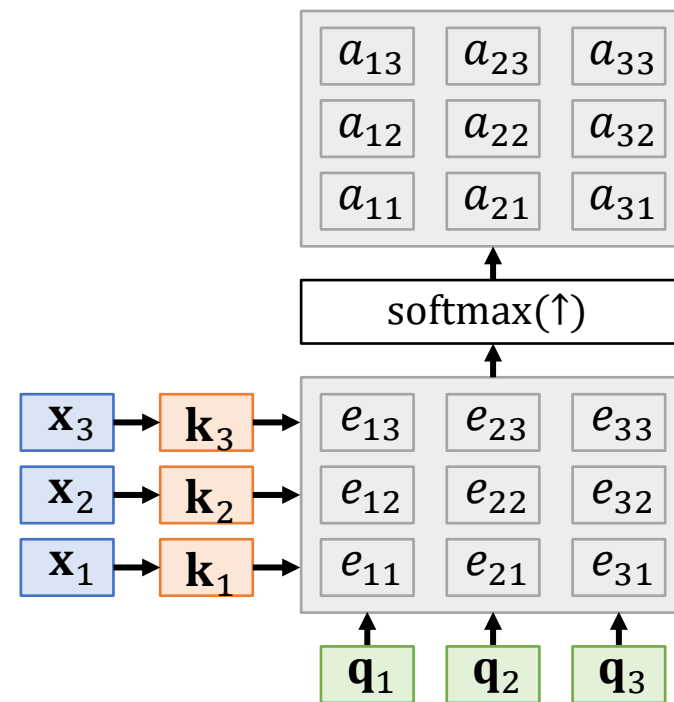
Key vectors: $K = XW_K$ (shape: $N_X \times D_Q$)

Value vectors: $V = XW_V$ (shape: $N_X \times D_V$)

Similarities: $E = QK / \sqrt{D_Q}$ (shape: $N_Q \times N_X$), $e_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D_Q}$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ (shape: $N_Q \times N_X$)

Output vector: $Y = AV$ (shape: $N_Q \times D_V$), $\mathbf{y}_i = \sum_j a_{ij} \mathbf{v}_j$



Attention Layer

Inputs:

Query vectors: Q (shape: $N_Q \times D_Q$)

Input vectors: X (shape: $N_X \times D_X$)

Similarity function: scaled dot product

Key matrix: W_K (shape: $D_X \times D_Q$)

Value matrix: W_V (shape: $D_X \times D_V$)

Computation:

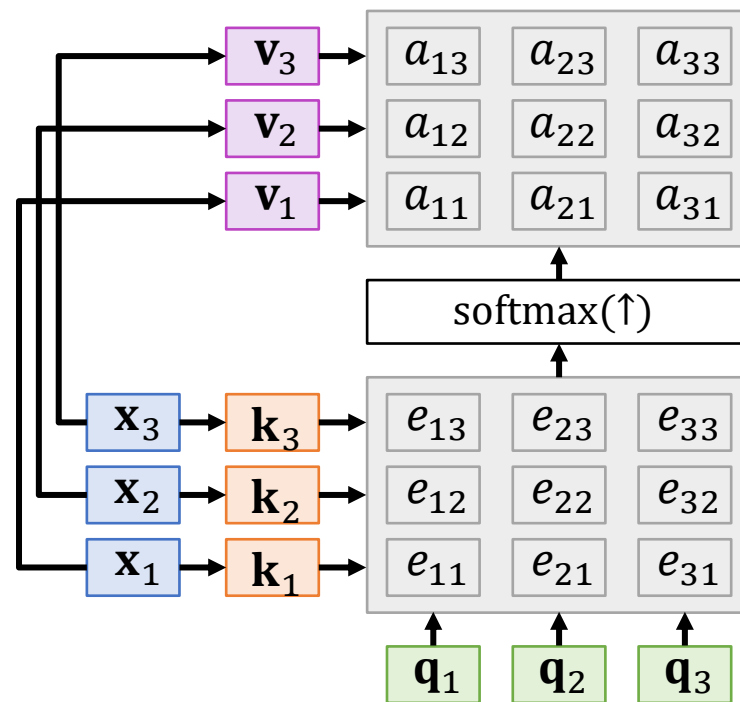
Key vectors: $K = XW_K$ (shape: $N_X \times D_Q$)

Value vectors: $V = XW_V$ (shape: $N_X \times D_V$)

Similarities: $E = QK / \sqrt{D_Q}$ (shape: $N_Q \times N_X$), $e_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D_Q}$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ (shape: $N_Q \times N_X$)

Output vector: $Y = AV$ (shape: $N_Q \times D_V$), $\mathbf{y}_i = \sum_j a_{ij} \mathbf{v}_j$



Attention Layer

Inputs:

Query vectors: Q (shape: $N_Q \times D_Q$)

Input vectors: X (shape: $N_X \times D_X$)

Similarity function: scaled dot product

Key matrix: W_K (shape: $D_X \times D_Q$)

Value matrix: W_V (shape: $D_X \times D_V$)

Computation:

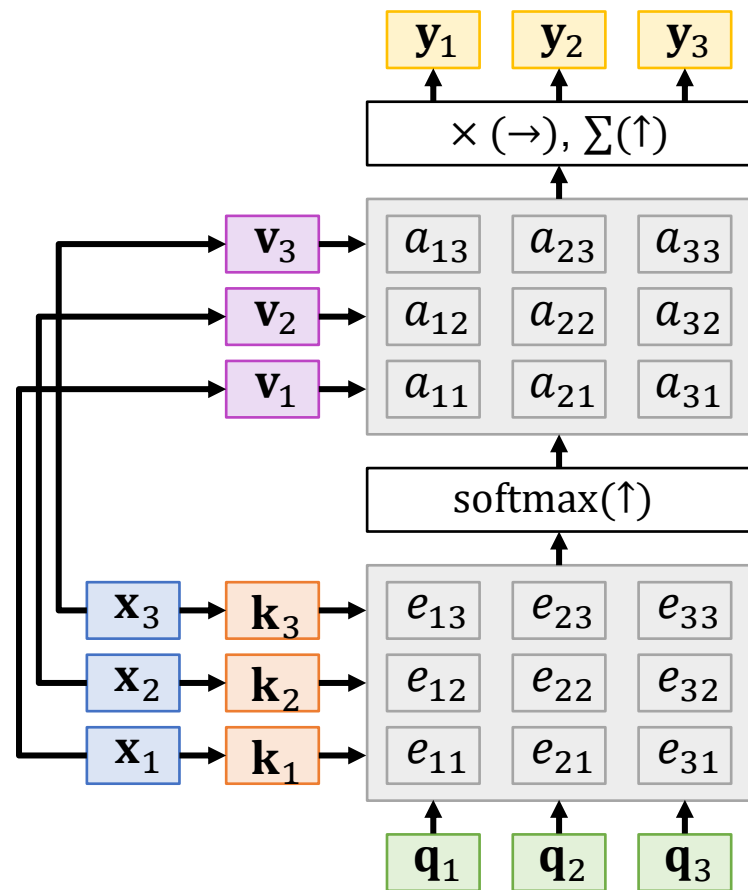
Key vectors: $K = XW_K$ (shape: $N_X \times D_Q$)

Value vectors: $V = XW_V$ (shape: $N_X \times D_V$)

Similarities: $E = QK / \sqrt{D_Q}$ (shape: $N_Q \times N_X$), $e_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D_Q}$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ (shape: $N_Q \times N_X$)

Output vector: $Y = AV$ (shape: $N_Q \times D_V$), $\mathbf{y}_i = \sum_j a_{ij} \mathbf{v}_j$



Attention Layer

Inputs:

Query vectors: Q (shape: $N_Q \times D_Q$)

Input vectors: X (shape: $N_X \times D_X$)

Similarity function: scaled dot product

Key matrix: W_K (shape: $D_X \times D_Q$)

Value matrix: W_V (shape: $D_X \times D_V$)

Can we use attention layer to cross-contextualize within a set of input vectors?

Computation:

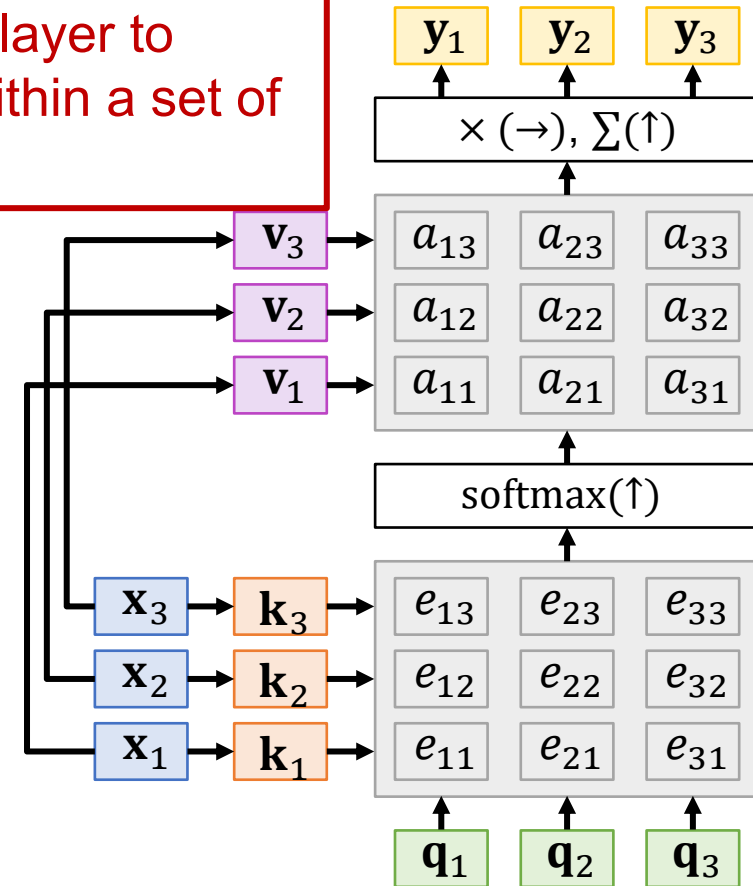
Key vectors: $K = XW_K$ (shape: $N_X \times D_Q$)

Value vectors: $V = XW_V$ (shape: $N_X \times D_V$)

Similarities: $E = QK / \sqrt{D_Q}$ (shape: $N_Q \times N_X$), $e_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D_Q}$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ (shape: $N_Q \times N_X$)

Output vector: $Y = AV$ (shape: $N_Q \times D_V$), $\mathbf{y}_i = \sum_j a_{ij} \mathbf{v}_j$



Self-Attention Layer

Inputs:

Input vectors: X (shape: $N_X \times D_X$)

Similarity function: scaled dot product

Key matrix: W_K (shape: $D_X \times D_Q$)

Value matrix: W_V (shape: $D_X \times D_V$)

Query matrix: W_Q (shape: $D_X \times D_Q$)

Computation:

Query vectors: $Q = XW_Q$ (shape: $N_X \times D_Q$)

Key vectors: $K = XW_K$ (shape: $N_X \times D_Q$)

Value vectors: $V = XW_V$ (shape: $N_X \times D_V$)

Similarities: $E = QK / \sqrt{D_Q}$ (shape: $N_Q \times N_X$), $e_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D_Q}$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ (shape: $N_Q \times N_X$)

Output vector: $Y = AV$ (shape: $N_Q \times D_V$), $\mathbf{y}_i = \sum_j a_{ij} \mathbf{v}_j$

Self-Attention Layer

Inputs:

Input vectors: X (shape: $N_X \times D_X$)

Similarity function: scaled dot product

Key matrix: W_K (shape: $D_X \times D_Q$)

Value matrix: W_V (shape: $D_X \times D_V$)

Query matrix: W_Q (shape: $D_X \times D_Q$)

Computation:

Query vectors: $Q = XW_Q$ (shape: $N_X \times D_Q$)

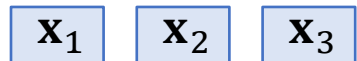
Key vectors: $K = XW_K$ (shape: $N_X \times D_Q$)

Value vectors: $V = XW_V$ (shape: $N_X \times D_V$)

Similarities: $E = QK / \sqrt{D_Q}$ (shape: $N_Q \times N_X$), $e_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D_Q}$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ (shape: $N_Q \times N_X$)

Output vector: $Y = AV$ (shape: $N_Q \times D_V$), $\mathbf{y}_i = \sum_j a_{ij} \mathbf{v}_j$



Self-Attention Layer

Inputs:

Input vectors: X (shape: $N_X \times D_X$)

Similarity function: scaled dot product

Key matrix: W_K (shape: $D_X \times D_Q$)

Value matrix: W_V (shape: $D_X \times D_V$)

Query matrix: W_Q (shape: $D_X \times D_Q$)

Computation:

Query vectors: $Q = XW_Q$ (shape: $N_X \times D_Q$)

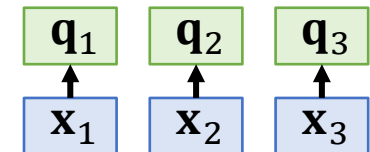
Key vectors: $K = XW_K$ (shape: $N_X \times D_Q$)

Value vectors: $V = XW_V$ (shape: $N_X \times D_V$)

Similarities: $E = QK / \sqrt{D_Q}$ (shape: $N_Q \times N_X$), $e_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D_Q}$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ (shape: $N_Q \times N_X$)

Output vector: $Y = AV$ (shape: $N_Q \times D_V$), $\mathbf{y}_i = \sum_j a_{ij} \mathbf{v}_j$



Self-Attention Layer

Inputs:

Input vectors: X (shape: $N_X \times D_X$)

Similarity function: scaled dot product

Key matrix: W_K (shape: $D_X \times D_Q$)

Value matrix: W_V (shape: $D_X \times D_V$)

Query matrix: W_Q (shape: $D_X \times D_Q$)

Computation:

Query vectors: $Q = XW_Q$ (shape: $N_X \times D_Q$)

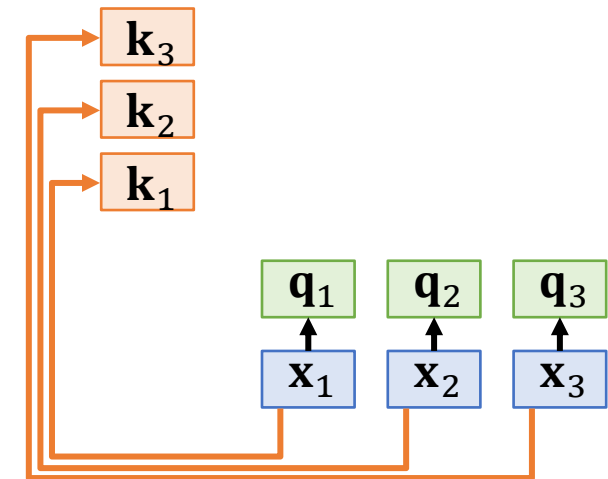
Key vectors: $K = XW_K$ (shape: $N_X \times D_Q$)

Value vectors: $V = XW_V$ (shape: $N_X \times D_V$)

Similarities: $E = QK / \sqrt{D_Q}$ (shape: $N_Q \times N_X$), $e_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D_Q}$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ (shape: $N_Q \times N_X$)

Output vector: $Y = AV$ (shape: $N_Q \times D_V$), $\mathbf{y}_i = \sum_j a_{ij} \mathbf{v}_j$



Self-Attention Layer

Inputs:

Input vectors: X (shape: $N_X \times D_X$)

Similarity function: scaled dot product

Key matrix: W_K (shape: $D_X \times D_Q$)

Value matrix: W_V (shape: $D_X \times D_V$)

Query matrix: W_Q (shape: $D_X \times D_Q$)

Computation:

Query vectors: $Q = XW_Q$ (shape: $N_X \times D_Q$)

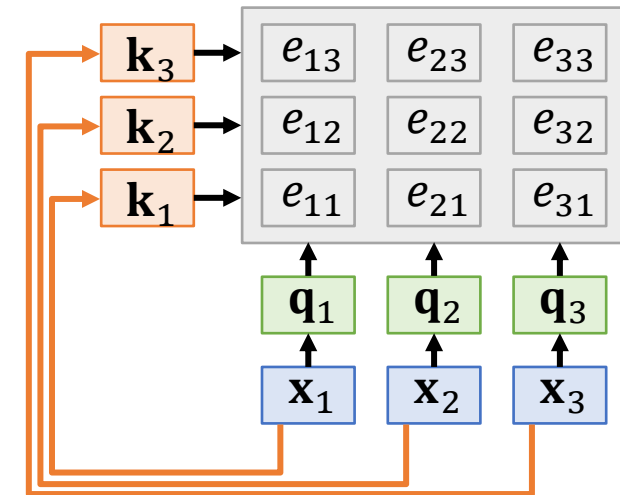
Key vectors: $K = XW_K$ (shape: $N_X \times D_Q$)

Value vectors: $V = XW_V$ (shape: $N_X \times D_V$)

Similarities: $E = QK / \sqrt{D_Q}$ (shape: $N_Q \times N_X$), $e_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D_Q}$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ (shape: $N_Q \times N_X$)

Output vector: $Y = AV$ (shape: $N_Q \times D_V$), $\mathbf{y}_i = \sum_j a_{ij} \mathbf{v}_j$



Self-Attention Layer

Inputs:

Input vectors: X (shape: $N_X \times D_X$)

Similarity function: scaled dot product

Key matrix: W_K (shape: $D_X \times D_Q$)

Value matrix: W_V (shape: $D_X \times D_V$)

Query matrix: W_Q (shape: $D_X \times D_Q$)

Computation:

Query vectors: $Q = XW_Q$ (shape: $N_X \times D_Q$)

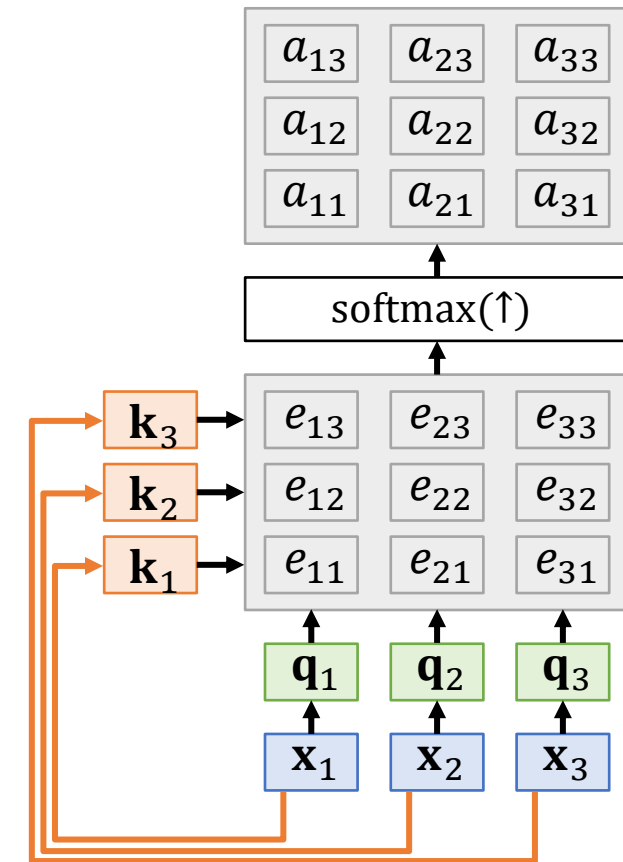
Key vectors: $K = XW_K$ (shape: $N_X \times D_Q$)

Value vectors: $V = XW_V$ (shape: $N_X \times D_V$)

Similarities: $E = QK / \sqrt{D_Q}$ (shape: $N_Q \times N_X$), $e_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D_Q}$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ (shape: $N_Q \times N_X$)

Output vector: $Y = AV$ (shape: $N_Q \times D_V$), $\mathbf{y}_i = \sum_j a_{ij} \mathbf{v}_j$



Self-Attention Layer

Inputs:

Input vectors: X (shape: $N_X \times D_X$)

Similarity function: scaled dot product

Key matrix: W_K (shape: $D_X \times D_Q$)

Value matrix: W_V (shape: $D_X \times D_V$)

Query matrix: W_Q (shape: $D_X \times D_Q$)

Computation:

Query vectors: $Q = XW_Q$ (shape: $N_X \times D_Q$)

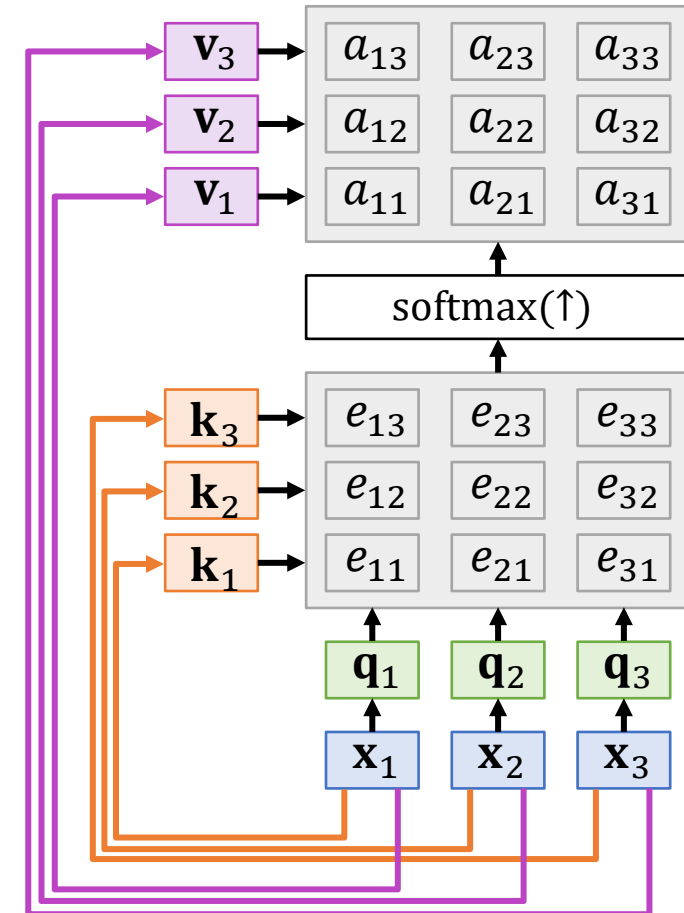
Key vectors: $K = XW_K$ (shape: $N_X \times D_Q$)

Value vectors: $V = XW_V$ (shape: $N_X \times D_V$)

Similarities: $E = QK / \sqrt{D_Q}$ (shape: $N_Q \times N_X$), $e_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D_Q}$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ (shape: $N_Q \times N_X$)

Output vector: $Y = AV$ (shape: $N_Q \times D_V$), $\mathbf{y}_i = \sum_j a_{ij} \mathbf{v}_j$



Self-Attention Layer

Inputs:

Input vectors: X (shape: $N_X \times D_X$)

Similarity function: scaled dot product

Key matrix: W_K (shape: $D_X \times D_Q$)

Value matrix: W_V (shape: $D_X \times D_V$)

Query matrix: W_Q (shape: $D_X \times D_Q$)

Computation:

Query vectors: $Q = XW_Q$ (shape: $N_X \times D_Q$)

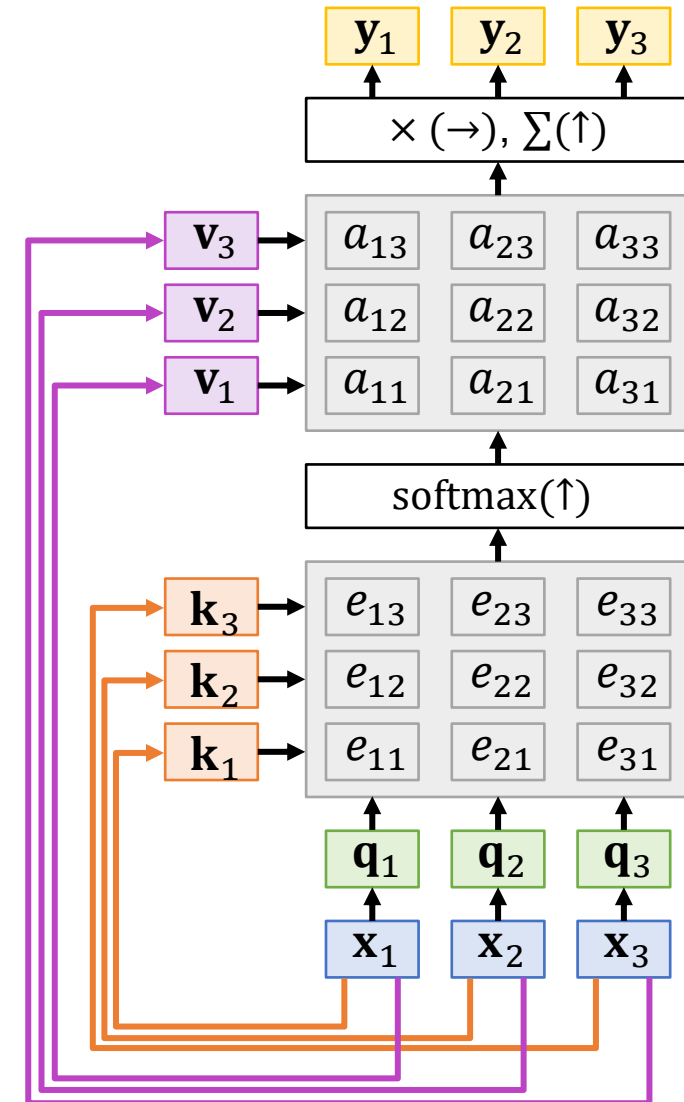
Key vectors: $K = XW_K$ (shape: $N_X \times D_Q$)

Value vectors: $V = XW_V$ (shape: $N_X \times D_V$)

Similarities: $E = QK / \sqrt{D_Q}$ (shape: $N_Q \times N_X$), $e_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D_Q}$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ (shape: $N_Q \times N_X$)

Output vector: $Y = AV$ (shape: $N_Q \times D_V$), $\mathbf{y}_i = \sum_j a_{ij} \mathbf{v}_j$



Self-Attention Layer

Inputs:

Input vectors: X (shape: $N_X \times D_X$)

Similarity function: scaled dot product

Key matrix: W_K (shape: $D_X \times D_Q$)

Value matrix: W_V (shape: $D_X \times D_V$)

Query matrix: W_Q (shape: $D_X \times D_Q$)

Computation:

Query vectors: $Q = XW_Q$ (shape: $N_X \times D_Q$)

Key vectors: $K = XW_K$ (shape: $N_X \times D_Q$)

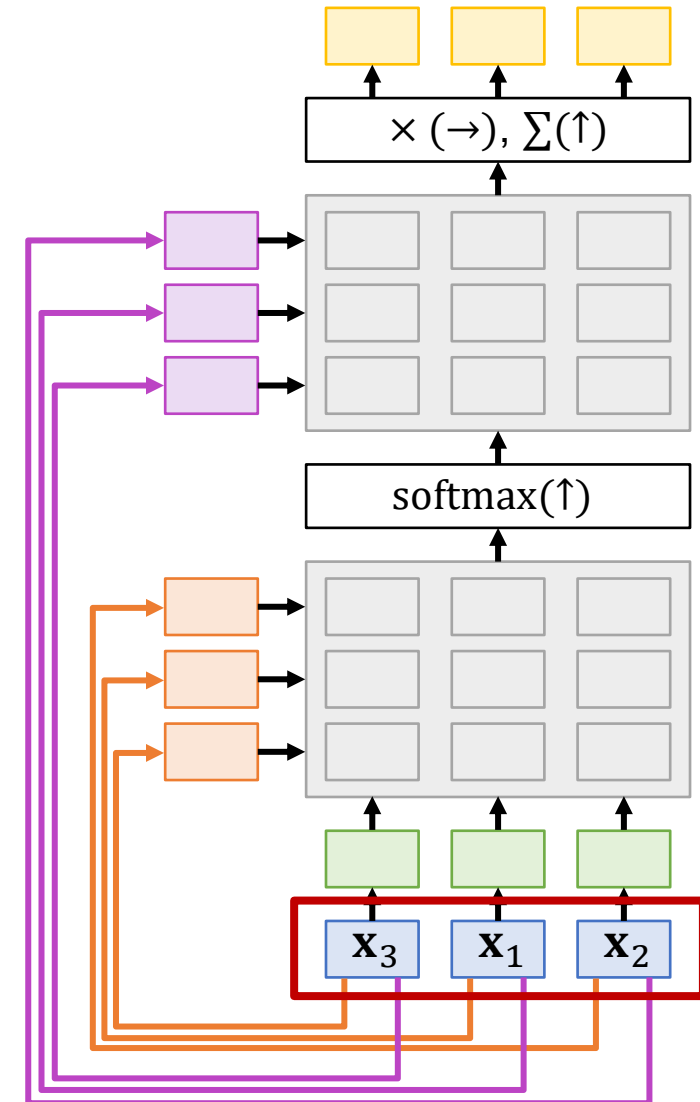
Value vectors: $V = XW_V$ (shape: $N_X \times D_V$)

Similarities: $E = QK / \sqrt{D_Q}$ (shape: $N_Q \times N_X$), $e_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D_Q}$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ (shape: $N_Q \times N_X$)

Output vector: $Y = AV$ (shape: $N_Q \times D_V$), $\mathbf{y}_i = \sum_j a_{ij} \mathbf{v}_j$

Consider *permuting* the input vectors.



Self-Attention Layer

Inputs:

Input vectors: X (shape: $N_X \times D_X$)

Similarity function: scaled dot product

Key matrix: W_K (shape: $D_X \times D_Q$)

Value matrix: W_V (shape: $D_X \times D_V$)

Query matrix: W_Q (shape: $D_X \times D_Q$)

Computation:

Query vectors: $Q = XW_Q$ (shape: $N_X \times D_Q$)

Key vectors: $K = XW_K$ (shape: $N_X \times D_Q$)

Value vectors: $V = XW_V$ (shape: $N_X \times D_V$)

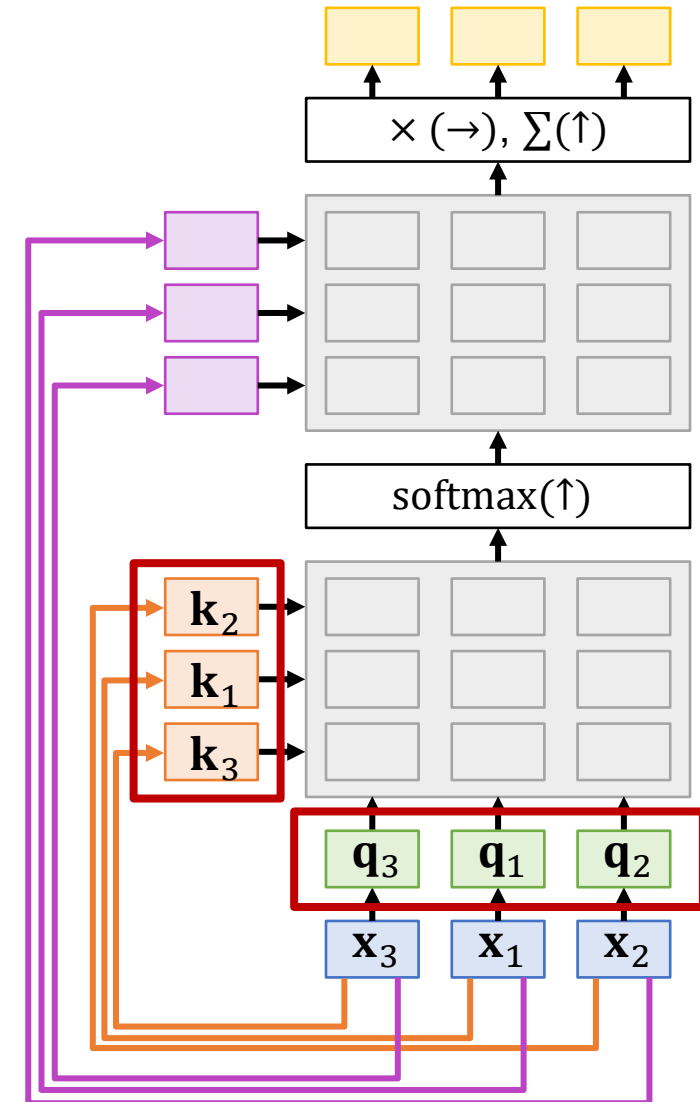
Similarities: $E = QK / \sqrt{D_Q}$ (shape: $N_Q \times N_X$), $e_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D_Q}$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ (shape: $N_Q \times N_X$)

Output vector: $Y = AV$ (shape: $N_Q \times D_V$), $\mathbf{y}_i = \sum_j a_{ij} \mathbf{v}_j$

Consider **permuting** the input vectors.

Queries and keys will be the same but **permutated**.



Self-Attention Layer

Inputs:

Input vectors: X (shape: $N_X \times D_X$)

Similarity function: scaled dot product

Key matrix: W_K (shape: $D_X \times D_Q$)

Value matrix: W_V (shape: $D_X \times D_V$)

Query matrix: W_Q (shape: $D_X \times D_Q$)

Computation:

Query vectors: $Q = XW_Q$ (shape: $N_X \times D_Q$)

Key vectors: $K = XW_K$ (shape: $N_X \times D_Q$)

Value vectors: $V = XW_V$ (shape: $N_X \times D_V$)

Similarities: $E = QK / \sqrt{D_Q}$ (shape: $N_Q \times N_X$), $e_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D_Q}$

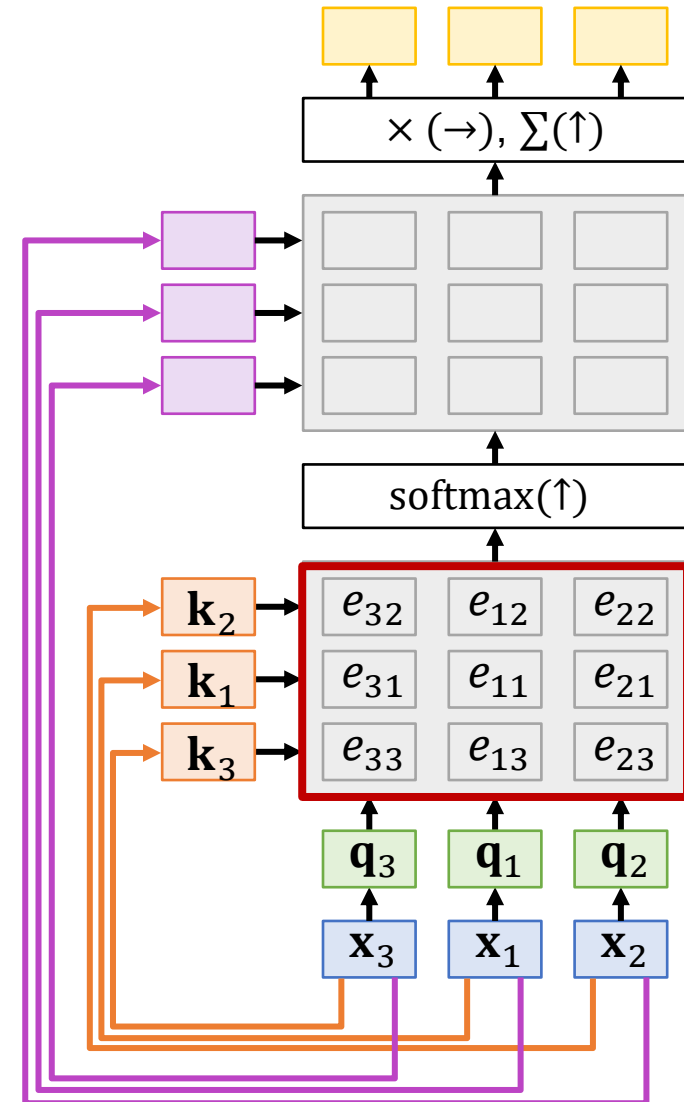
Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ (shape: $N_Q \times N_X$)

Output vector: $Y = AV$ (shape: $N_Q \times D_V$), $\mathbf{y}_i = \sum_j a_{ij} \mathbf{v}_j$

Consider **permuting** the input vectors.

Queries and keys will be the same but **permututed**.

Similarities will be the same but **permututed**.



Self-Attention Layer

Inputs:

Input vectors: X (shape: $N_X \times D_X$)

Similarity function: scaled dot product

Key matrix: W_K (shape: $D_X \times D_Q$)

Value matrix: W_V (shape: $D_X \times D_V$)

Query matrix: W_Q (shape: $D_X \times D_Q$)

Computation:

Query vectors: $Q = XW_Q$ (shape: $N_X \times D_Q$)

Key vectors: $K = XW_K$ (shape: $N_X \times D_Q$)

Value vectors: $V = XW_V$ (shape: $N_X \times D_V$)

Similarities: $E = QK / \sqrt{D_Q}$ (shape: $N_Q \times N_X$), $e_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D_Q}$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ (shape: $N_Q \times N_X$)

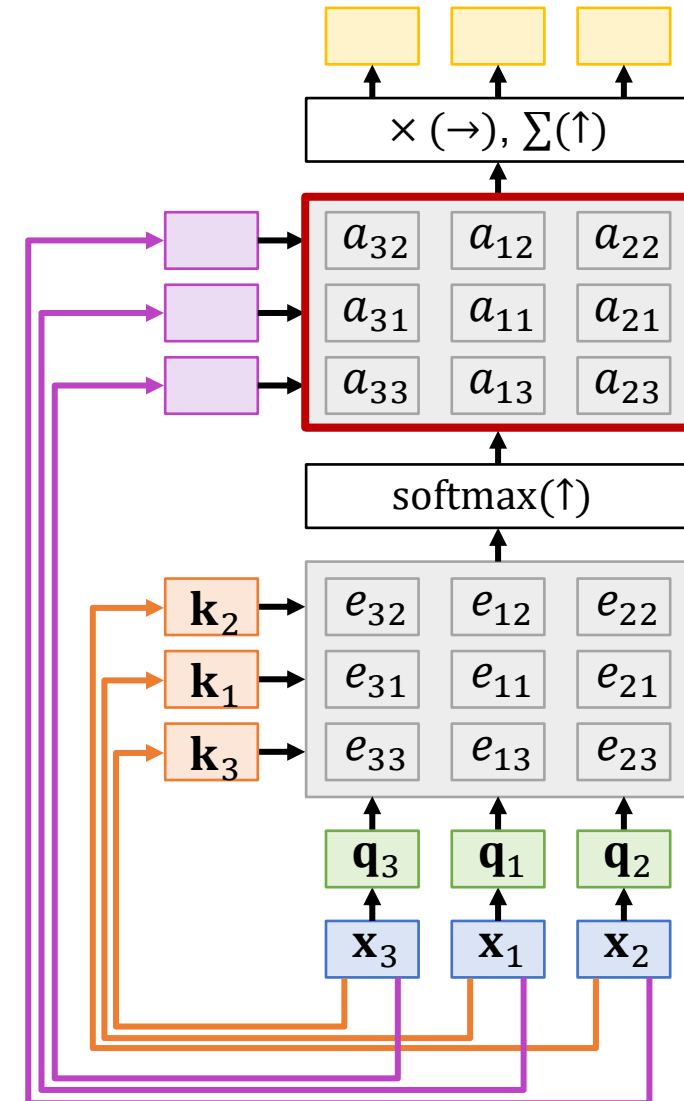
Output vector: $Y = AV$ (shape: $N_Q \times D_V$), $\mathbf{y}_i = \sum_j a_{ij} \mathbf{v}_j$

Consider **permuting** the input vectors.

Queries and keys will be the same but **permututed**.

Similarities will be the same but **permututed**.

Attention weights will be the same but **permututed**.



Self-Attention Layer

Inputs:

Input vectors: X (shape: $N_X \times D_X$)

Similarity function: scaled dot product

Key matrix: W_K (shape: $D_X \times D_Q$)

Value matrix: W_V (shape: $D_X \times D_V$)

Query matrix: W_Q (shape: $D_X \times D_Q$)

Computation:

Query vectors: $Q = XW_Q$ (shape: $N_X \times D_Q$)

Key vectors: $K = XW_K$ (shape: $N_X \times D_Q$)

Value vectors: $V = XW_V$ (shape: $N_X \times D_V$)

Similarities: $E = QK / \sqrt{D_Q}$ (shape: $N_Q \times N_X$), $e_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D_Q}$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ (shape: $N_Q \times N_X$)

Output vector: $Y = AV$ (shape: $N_Q \times D_V$), $\mathbf{y}_i = \sum_j a_{ij} \mathbf{v}_j$

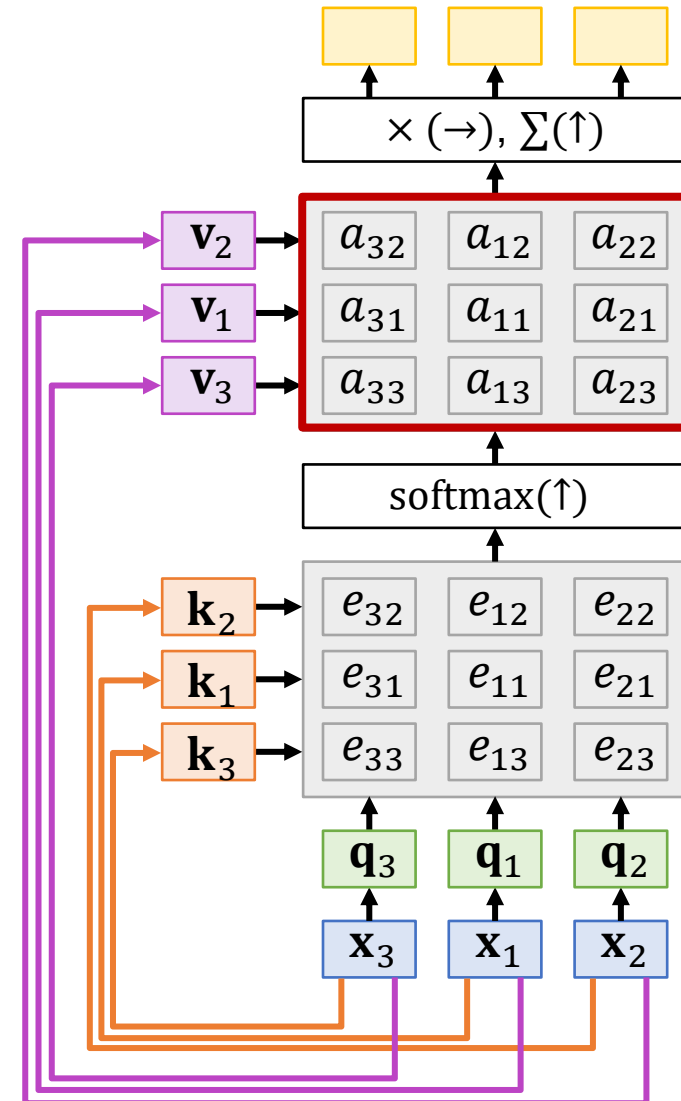
Consider **permuting** the input vectors.

Queries and keys will be the same but **permututed**.

Similarities will be the same but **permututed**.

Attention weights will be the same but **permututed**.

Values will be the same but **permututed**.



Self-Attention Layer

Inputs:

Input vectors: X (shape: $N_X \times D_X$)

Similarity function: scaled dot product

Key matrix: W_K (shape: $D_X \times D_Q$)

Value matrix: W_V (shape: $D_X \times D_V$)

Query matrix: W_Q (shape: $D_X \times D_Q$)

Computation:

Query vectors: $Q = XW_Q$ (shape: $N_X \times D_Q$)

Key vectors: $K = XW_K$ (shape: $N_X \times D_Q$)

Value vectors: $V = XW_V$ (shape: $N_X \times D_V$)

Similarities: $E = QK / \sqrt{D_Q}$ (shape: $N_Q \times N_X$), $e_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D_Q}$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ (shape: $N_Q \times N_X$)

Output vector: $Y = AV$ (shape: $N_Q \times D_V$), $\mathbf{y}_i = \sum_j a_{ij} \mathbf{v}_j$

Consider **permuting** the input vectors.

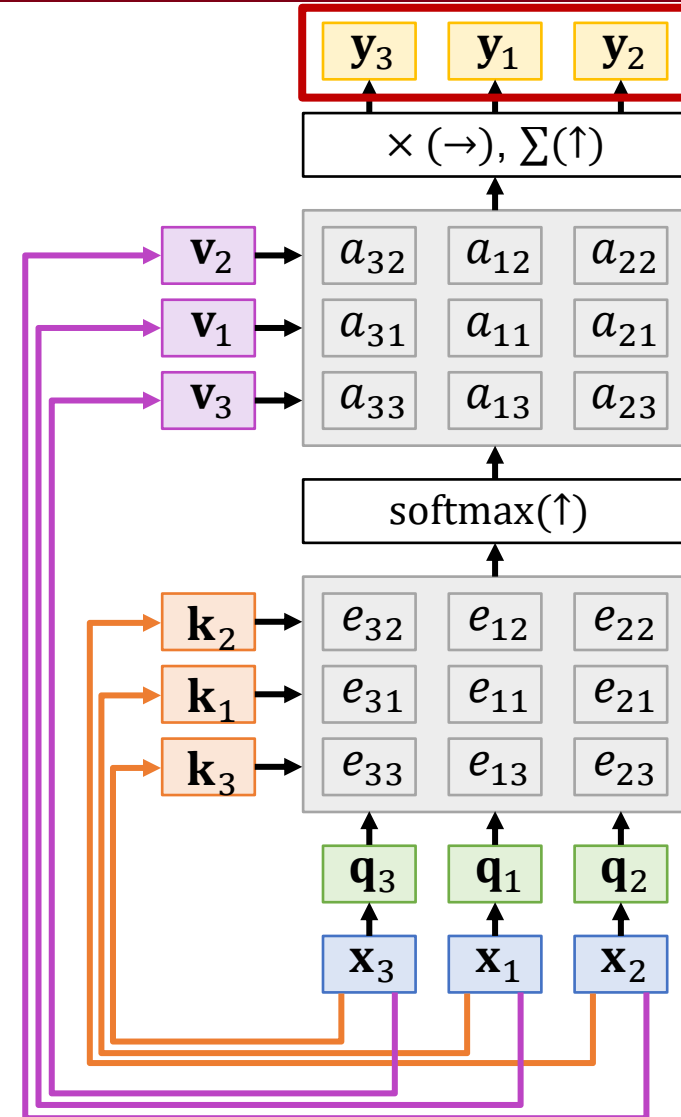
Queries and keys will be the same but **permututed**.

Similarities will be the same but **permututed**.

Attention weights will be the same but **permututed**.

Values will be the same but **permututed**.

Outputs will be the same but **permututed**.



Self-Attention Layer

Inputs:

Input vectors: X (shape: $N_X \times D_X$)

Similarity function: scaled dot product

Key matrix: W_K (shape: $D_X \times D_Q$)

Value matrix: W_V (shape: $D_X \times D_V$)

Query matrix: W_Q (shape: $D_X \times D_Q$)

Computation:

Query vectors: $Q = XW_Q$ (shape: $N_X \times D_Q$)

Key vectors: $K = XW_K$ (shape: $N_X \times D_Q$)

Value vectors: $V = XW_V$ (shape: $N_X \times D_V$)

Similarities: $E = QK / \sqrt{D_Q}$ (shape: $N_Q \times N_X$), $e_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D_Q}$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ (shape: $N_Q \times N_X$)

Output vector: $Y = AV$ (shape: $N_Q \times D_V$), $\mathbf{y}_i = \sum_j a_{ij} \mathbf{v}_j$

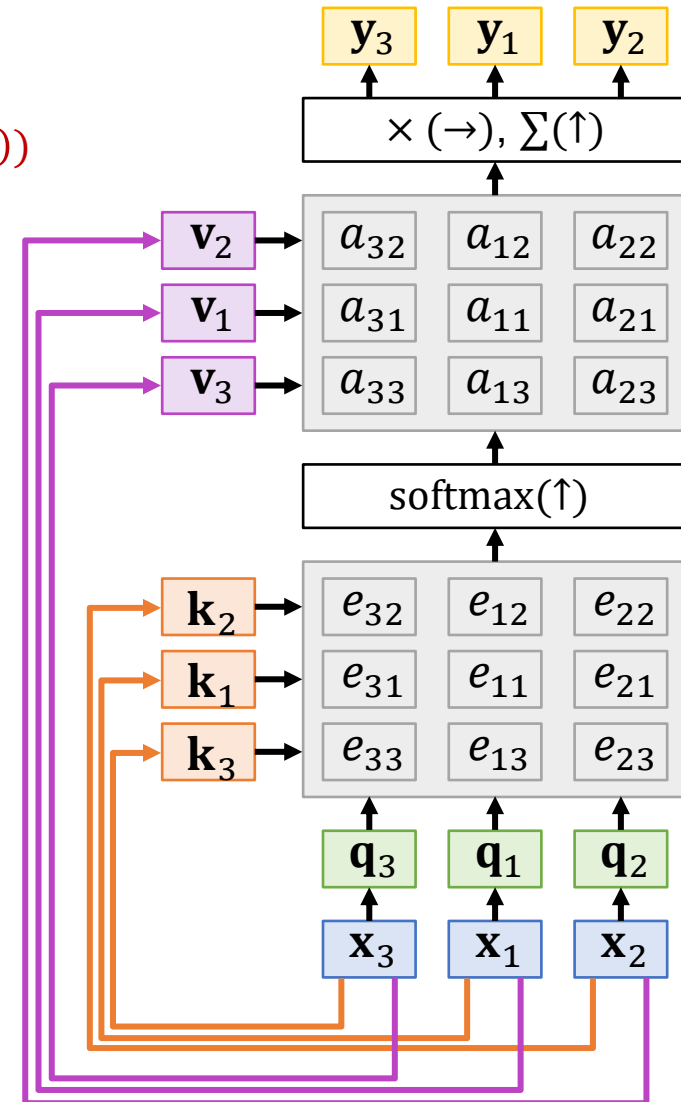
Self-attention layer is

permutation equivariant!

$$f(\text{permute}(x)) = \text{permute}(f(x))$$

Self-attention works on **sets of vectors**.

Self-attention **does not know the order** of vectors it is processing.



Self-Attention Layer

Inputs:

Input vectors: X (shape: $N_X \times D_X$)

Similarity function: scaled dot product

Key matrix: W_K (shape: $D_X \times D_Q$)

Value matrix: W_V (shape: $D_X \times D_V$)

Query matrix: W_Q (shape: $D_X \times D_Q$)

Computation:

Query vectors: $Q = XW_Q$ (shape: $N_X \times D_Q$)

Key vectors: $K = XW_K$ (shape: $N_X \times D_Q$)

Value vectors: $V = XW_V$ (shape: $N_X \times D_V$)

Similarities: $E = QK / \sqrt{D_Q}$ (shape: $N_Q \times N_X$), $e_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D_Q}$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ (shape: $N_Q \times N_X$)

Output vector: $Y = AV$ (shape: $N_Q \times D_V$), $\mathbf{y}_i = \sum_j a_{ij} \mathbf{v}_j$

Self-attention layer is

permutation equivariant!

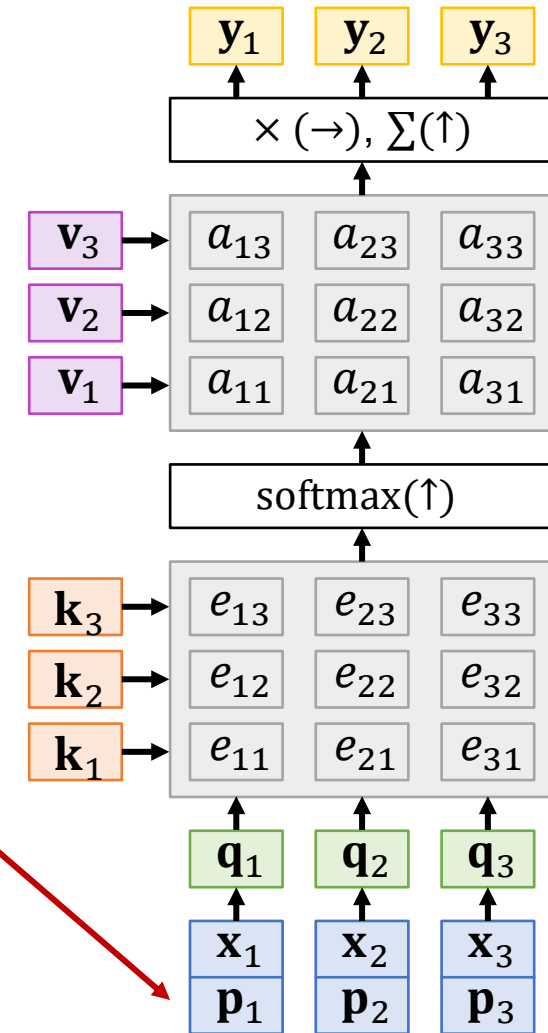
$$f(\text{permute}(x)) = \text{permute}(f(x))$$

Self-attention works on **sets of vectors**.

Self-attention **does not know the order** of vectors it is processing.

Add or concatenate

positional encoding to make the model **position-aware**



Masked Self-Attention Layer

Inputs:

Input vectors: X (shape: $N_X \times D_X$)

Similarity function: scaled dot product

Key matrix: W_K (shape: $D_X \times D_Q$)

Value matrix: W_V (shape: $D_X \times D_V$)

Query matrix: W_Q (shape: $D_X \times D_Q$)

Computation:

Query vectors: $Q = XW_Q$ (shape: $N_X \times D_Q$)

Key vectors: $K = XW_K$ (shape: $N_X \times D_Q$)

Value vectors: $V = XW_V$ (shape: $N_X \times D_V$)

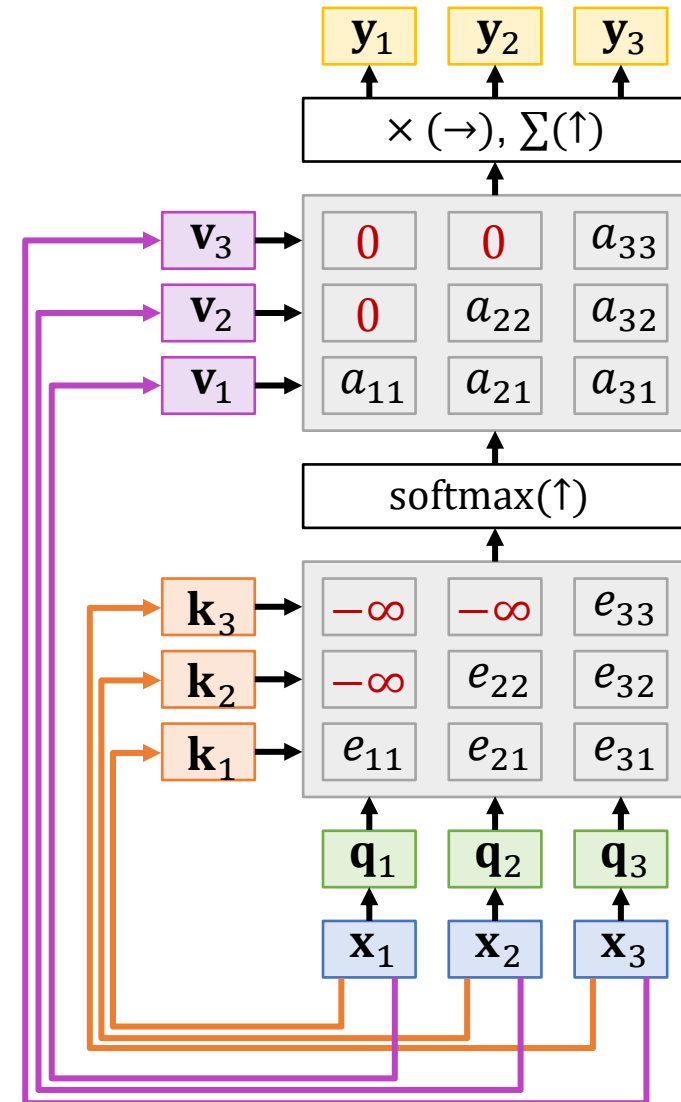
Similarities: $E = QK / \sqrt{D_Q}$ (shape: $N_Q \times N_X$), $e_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D_Q}$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ (shape: $N_Q \times N_X$)

Output vector: $Y = AV$ (shape: $N_Q \times D_V$), $\mathbf{y}_i = \sum_j a_{ij} \mathbf{v}_j$

Don't let vectors "look ahead"
in a sequence.

Used for language generation
(predict next word)



Multi-head Self-Attention Layer

Inputs:

Input vectors: X (shape: $N_X \times D_X$)

Similarity function: scaled dot product

Key matrix: W_K (shape: $D_X \times D_Q$)

Value matrix: W_V (shape: $D_X \times D_V$)

Query matrix: W_Q (shape: $D_X \times D_Q$)

Use H independent
attention heads in parallel.

Computation:

Query vectors: $Q = XW_Q$ (shape: $N_X \times D_Q$)

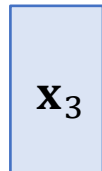
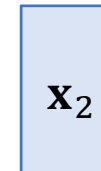
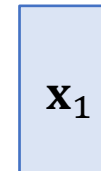
Key vectors: $K = XW_K$ (shape: $N_X \times D_Q$)

Value vectors: $V = XW_V$ (shape: $N_X \times D_V$)

Similarities: $E = QK / \sqrt{D_Q}$ (shape: $N_Q \times N_X$), $e_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D_Q}$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ (shape: $N_Q \times N_X$)

Output vector: $Y = AV$ (shape: $N_Q \times D_V$), $\mathbf{y}_i = \sum_j a_{ij} \mathbf{v}_j$



Multi-head Self-Attention Layer

Inputs:

Input vectors: X (shape: $N_X \times D_X$)

Similarity function: scaled dot product

Key matrix: W_K (shape: $D_X \times D_Q$)

Value matrix: W_V (shape: $D_X \times D_V$)

Query matrix: W_Q (shape: $D_X \times D_Q$)

Use H independent
attention heads in parallel.

Computation:

Query vectors: $Q = XW_Q$ (shape: $N_X \times D_Q$)

Key vectors: $K = XW_K$ (shape: $N_X \times D_Q$)

Value vectors: $V = XW_V$ (shape: $N_X \times D_V$)

Similarities: $E = QK / \sqrt{D_Q}$ (shape: $N_Q \times N_X$), $e_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D_Q}$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ (shape: $N_Q \times N_X$)

Output vector: $Y = AV$ (shape: $N_Q \times D_V$), $\mathbf{y}_i = \sum_j a_{ij} \mathbf{v}_j$

$\mathbf{x}_{1,1}$
$\mathbf{x}_{1,2}$
$\mathbf{x}_{1,3}$

$\mathbf{x}_{2,1}$
$\mathbf{x}_{2,2}$
$\mathbf{x}_{2,3}$

$\mathbf{x}_{3,1}$
$\mathbf{x}_{3,2}$
$\mathbf{x}_{3,3}$

Multi-head Self-Attention Layer

Inputs:

Input vectors: X (shape: $N_X \times D_X$)

Similarity function: scaled dot product

Key matrix: W_K (shape: $D_X \times D_Q$)

Value matrix: W_V (shape: $D_X \times D_V$)

Query matrix: W_Q (shape: $D_X \times D_Q$)

Use H independent
attention heads in parallel.

Computation:

Query vectors: $Q = XW_Q$ (shape: $N_X \times D_Q$)

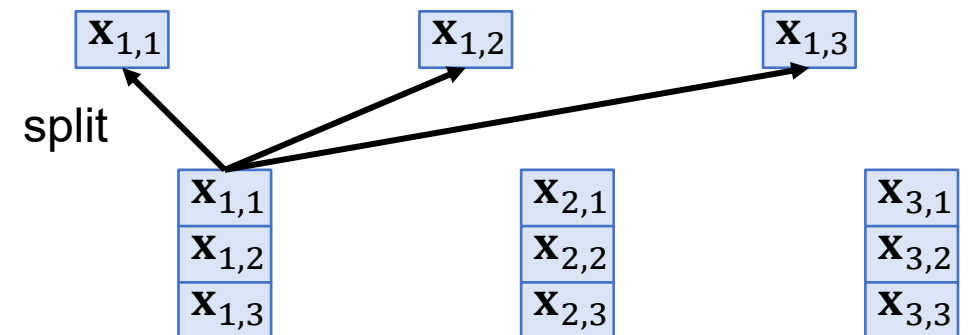
Key vectors: $K = XW_K$ (shape: $N_X \times D_Q$)

Value vectors: $V = XW_V$ (shape: $N_X \times D_V$)

Similarities: $E = QK / \sqrt{D_Q}$ (shape: $N_Q \times N_X$), $e_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D_Q}$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ (shape: $N_Q \times N_X$)

Output vector: $Y = AV$ (shape: $N_Q \times D_V$), $\mathbf{y}_i = \sum_j a_{ij} \mathbf{v}_j$



Multi-head Self-Attention Layer

Inputs:

Input vectors: X (shape: $N_X \times D_X$)

Similarity function: scaled dot product

Key matrix: W_K (shape: $D_X \times D_Q$)

Value matrix: W_V (shape: $D_X \times D_V$)

Query matrix: W_Q (shape: $D_X \times D_Q$)

Use H independent
attention heads in parallel.

Computation:

Query vectors: $Q = XW_Q$ (shape: $N_X \times D_Q$)

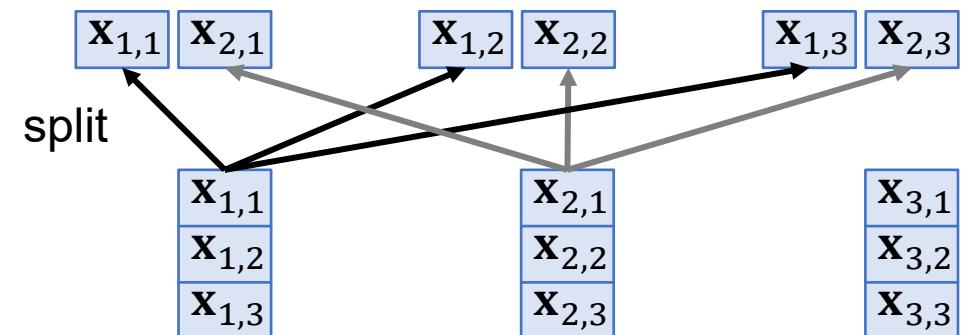
Key vectors: $K = XW_K$ (shape: $N_X \times D_Q$)

Value vectors: $V = XW_V$ (shape: $N_X \times D_V$)

Similarities: $E = QK / \sqrt{D_Q}$ (shape: $N_Q \times N_X$), $e_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D_Q}$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ (shape: $N_Q \times N_X$)

Output vector: $Y = AV$ (shape: $N_Q \times D_V$), $\mathbf{y}_i = \sum_j a_{ij} \mathbf{v}_j$



Multi-head Self-Attention Layer

Inputs:

Input vectors: X (shape: $N_X \times D_X$)

Similarity function: scaled dot product

Key matrix: W_K (shape: $D_X \times D_Q$)

Value matrix: W_V (shape: $D_X \times D_V$)

Query matrix: W_Q (shape: $D_X \times D_Q$)

Use H independent
attention heads in parallel.

Computation:

Query vectors: $Q = XW_Q$ (shape: $N_X \times D_Q$)

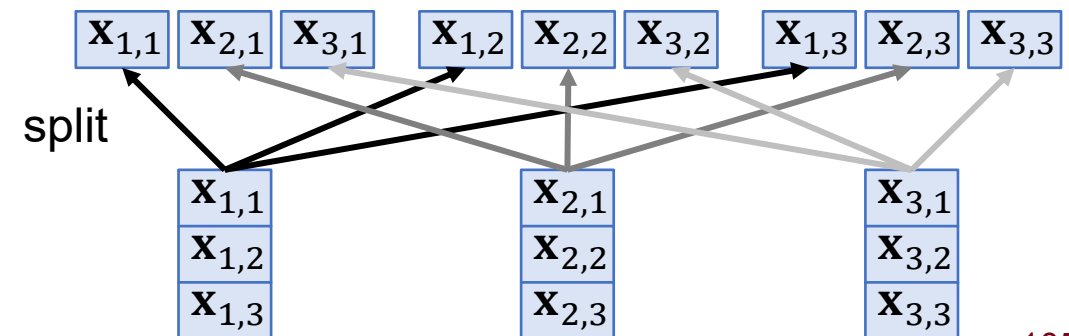
Key vectors: $K = XW_K$ (shape: $N_X \times D_Q$)

Value vectors: $V = XW_V$ (shape: $N_X \times D_V$)

Similarities: $E = QK / \sqrt{D_Q}$ (shape: $N_Q \times N_X$), $e_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D_Q}$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ (shape: $N_Q \times N_X$)

Output vector: $Y = AV$ (shape: $N_Q \times D_V$), $\mathbf{y}_i = \sum_j a_{ij} \mathbf{v}_j$



Multi-head Self-Attention Layer

Inputs:

Input vectors: X (shape: $N_X \times D_X$)

Similarity function: scaled dot product

Key matrix: W_K (shape: $D_X \times D_Q$)

Value matrix: W_V (shape: $D_X \times D_V$)

Query matrix: W_Q (shape: $D_X \times D_Q$)

Use H independent
attention heads in parallel.

Run **self-attention in parallel** on each set of input vectors with **different weights per head**.

Computation:

Query vectors: $Q = XW_Q$ (shape: $N_X \times D_Q$)

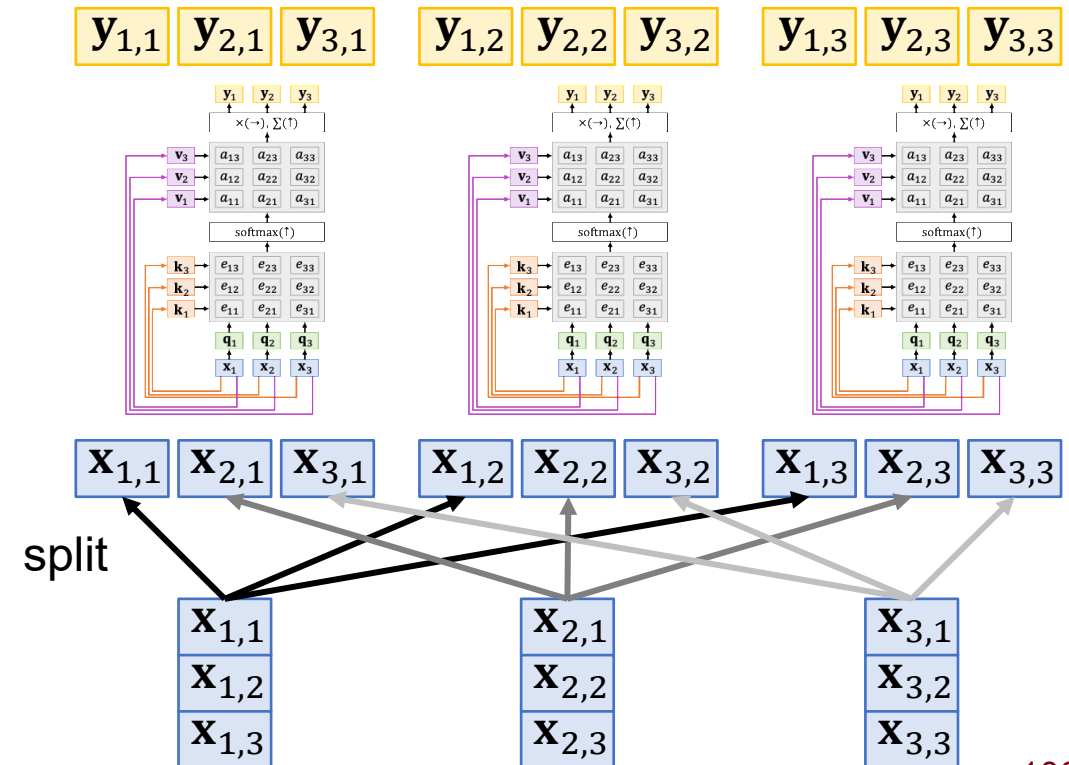
Key vectors: $K = XW_K$ (shape: $N_X \times D_Q$)

Value vectors: $V = XW_V$ (shape: $N_X \times D_V$)

Similarities: $E = QK / \sqrt{D_Q}$ (shape: $N_Q \times N_X$), $e_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D_Q}$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ (shape: $N_Q \times N_X$)

Output vector: $Y = AV$ (shape: $N_Q \times D_V$), $\mathbf{y}_i = \sum_j a_{ij} \mathbf{v}_j$



Multi-head Self-Attention Layer

Inputs:

Input vectors: X (shape: $N_X \times D_X$)

Similarity function: scaled dot product

Key matrix: W_K (shape: $D_X \times D_Q$)

Value matrix: W_V (shape: $D_X \times D_V$)

Query matrix: W_Q (shape: $D_X \times D_Q$)

Use H independent
attention heads in parallel.

Computation:

Query vectors: $Q = XW_Q$ (shape: $N_X \times D_Q$)

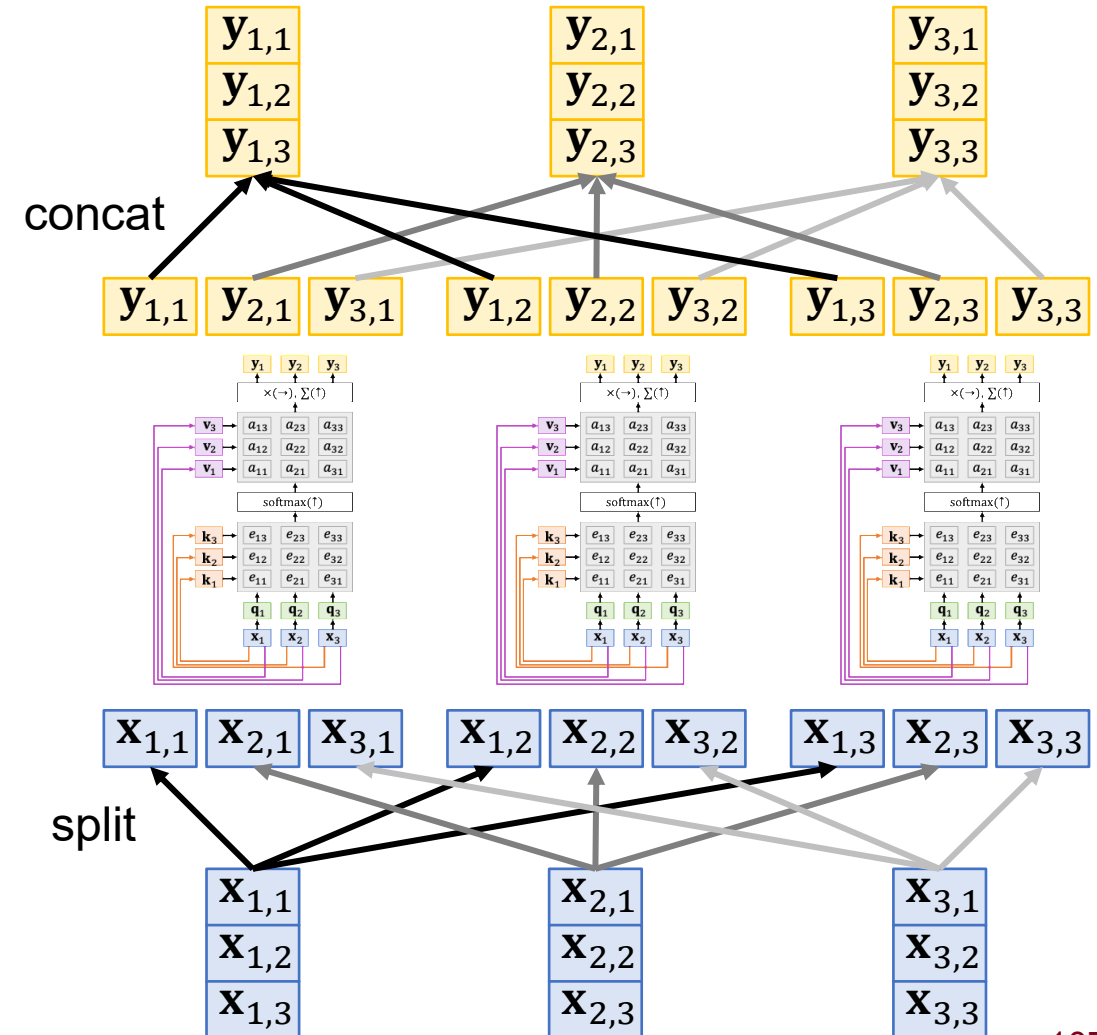
Key vectors: $K = XW_K$ (shape: $N_X \times D_Q$)

Value vectors: $V = XW_V$ (shape: $N_X \times D_V$)

Similarities: $E = QK / \sqrt{D_Q}$ (shape: $N_Q \times N_X$), $e_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D_Q}$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ (shape: $N_Q \times N_X$)

Output vector: $Y = AV$ (shape: $N_Q \times D_V$), $\mathbf{y}_i = \sum_j a_{ij} \mathbf{v}_j$



Multi-head Self-Attention Layer

Inputs:

Input vectors: X (shape: $N_X \times D_X$)

Similarity function: scaled dot product

Key matrix: W_K (shape: $D_X \times D_Q$)

Value matrix: W_V (shape: $D_X \times D_V$)

Query matrix: W_Q (shape: $D_X \times D_Q$)

Use H independent
attention heads in parallel.

Computation:

Query vectors: $Q = XW_Q$ (shape: $N_X \times D_Q$)

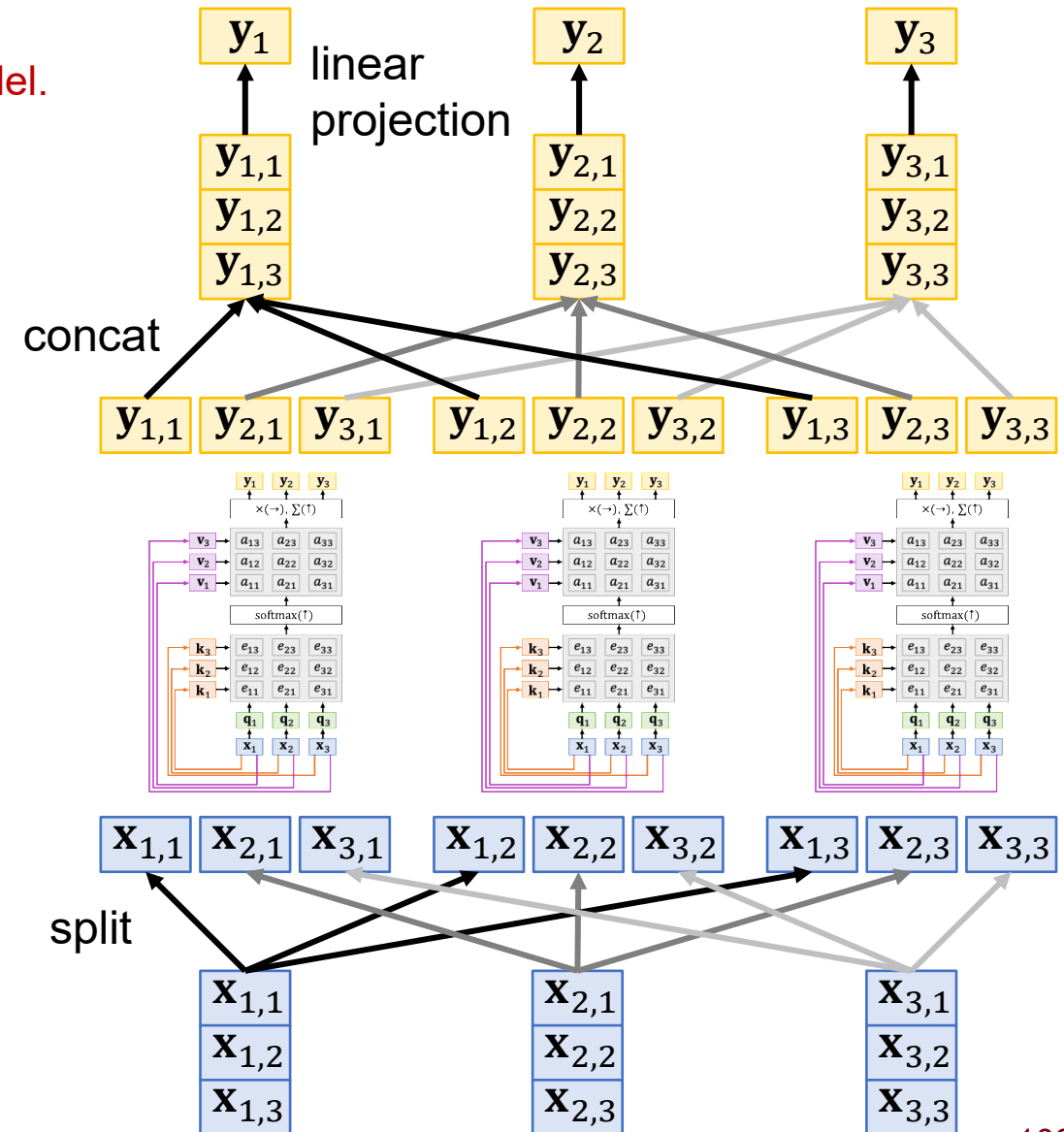
Key vectors: $K = XW_K$ (shape: $N_X \times D_Q$)

Value vectors: $V = XW_V$ (shape: $N_X \times D_V$)

Similarities: $E = QK / \sqrt{D_Q}$ (shape: $N_Q \times N_X$), $e_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D_Q}$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ (shape: $N_Q \times N_X$)

Output vector: $Y = AV$ (shape: $N_Q \times D_V$), $\mathbf{y}_i = \sum_j a_{ij} \mathbf{v}_j$



Transformers

$\mathbf{x}_1$

$\mathbf{x}_2$

$\mathbf{x}_3$

Vaswani et. al., Attention Is All You Need, NeurIPS 2017



KOREA
UNIVERSITY

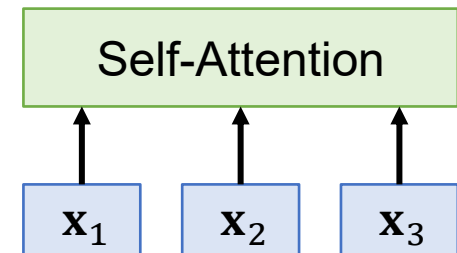


Multimodal Interactive
Intelligence Laboratory

Korea AI in Action
Large Language Models

Transformers

All vectors interact
with each other



Vaswani et. al., Attention Is All You Need, NeurIPS 2017



KOREA
UNIVERSITY



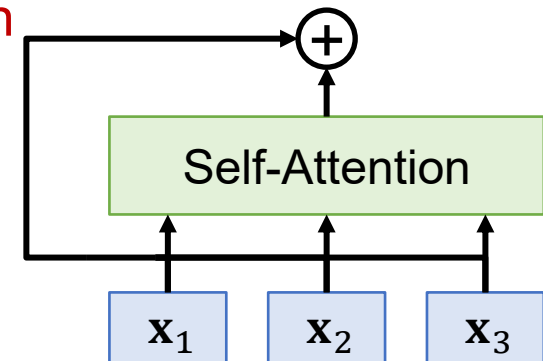
Multimodal Interactive
Intelligence Laboratory

Korea AI in Action
Large Language Models

Transformers

Residual connection

All vectors interact
with each other



Vaswani et. al., Attention Is All You Need, NeurIPS 2017

Transformers

Layer normalization:

Given $\mathbf{h}_1, \dots, \mathbf{h}_N$ (Shape: D)

Scale: γ (Shape: D)

Shift: β (Shape: D)

$$\mu_i = \frac{\sum_j h_{i,j}}{D} \quad (\text{Scalar})$$

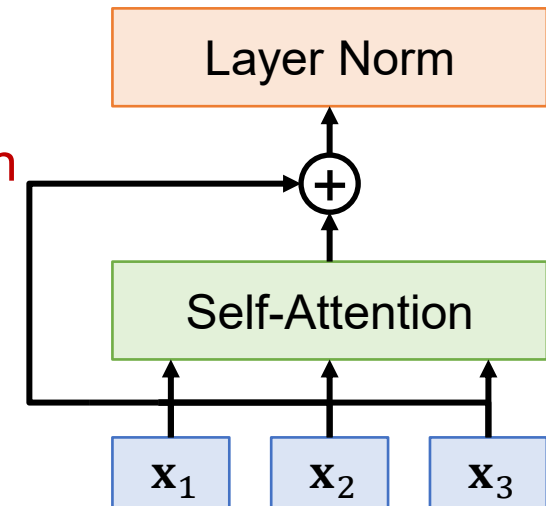
$$\sigma_i = \sqrt{\frac{\sum_j (h_{i,j} - \mu_i)^2}{D}} \quad (\text{Scalar})$$

$$\mathbf{z}_i = \frac{h_i - \mu_i}{\sigma_i}$$

$$\mathbf{y}_i = \gamma \mathbf{z}_i + \beta$$

Residual connection

All vectors interact with each other



Vaswani et. al., Attention Is All You Need, NeurIPS 2017

Transformers

Layer normalization:

Given $\mathbf{h}_1, \dots, \mathbf{h}_N$ (Shape: D)

Scale: γ (Shape: D)

Shift: β (Shape: D)

$$\mu_i = \frac{\sum_j h_{i,j}}{D} \quad (\text{Scalar})$$

$$\sigma_i = \sqrt{\frac{\sum_j (h_{i,j} - \mu_i)^2}{D}} \quad (\text{Scalar})$$

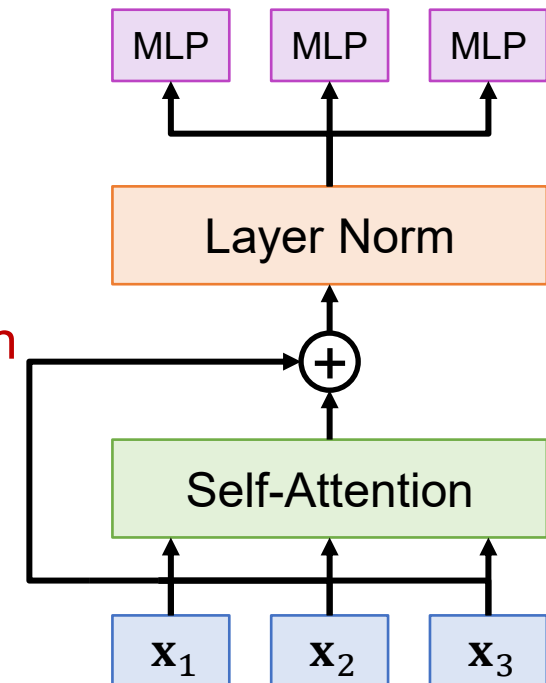
$$\mathbf{z}_i = \frac{h_i - \mu_i}{\sigma_i}$$

$$\mathbf{y}_i = \gamma \mathbf{z}_i + \beta$$

MLP independently
on each vector

Residual connection

All vectors interact
with each other



Transformers

Layer normalization:

Given $\mathbf{h}_1, \dots, \mathbf{h}_N$ (Shape: D)

Scale: γ (Shape: D)

Shift: β (Shape: D)

$$\mu_i = \frac{\sum_j h_{i,j}}{D} \quad (\text{Scalar})$$

$$\sigma_i = \sqrt{\frac{\sum_j (h_{i,j} - \mu_i)^2}{D}} \quad (\text{Scalar})$$

$$\mathbf{z}_i = \frac{h_i - \mu_i}{\sigma_i}$$

$$\mathbf{y}_i = \gamma \mathbf{z}_i + \beta$$

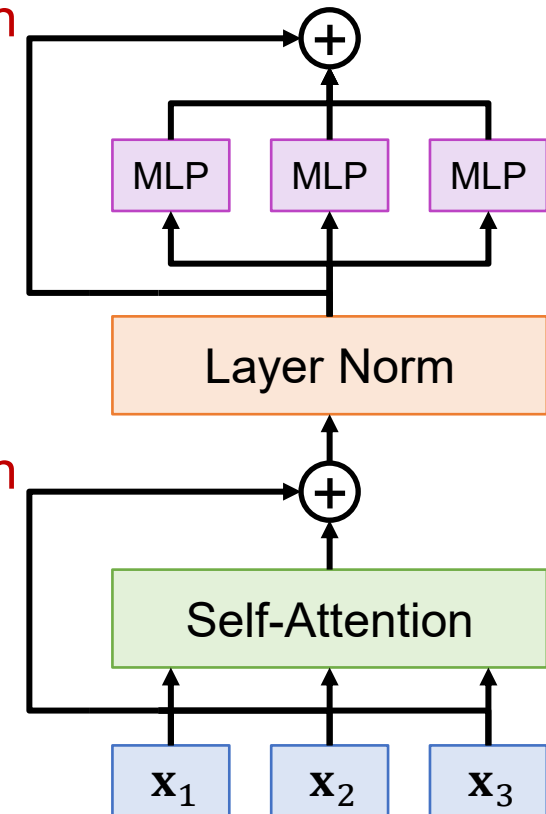
Vaswani et. al., Attention Is All You Need, NeurIPS 2017

Residual connection

MLP independently
on each vector

Residual connection

All vectors interact
with each other



Transformers

Layer normalization:

Given $\mathbf{h}_1, \dots, \mathbf{h}_N$ (Shape: D)

Scale: γ (Shape: D)

Shift: β (Shape: D)

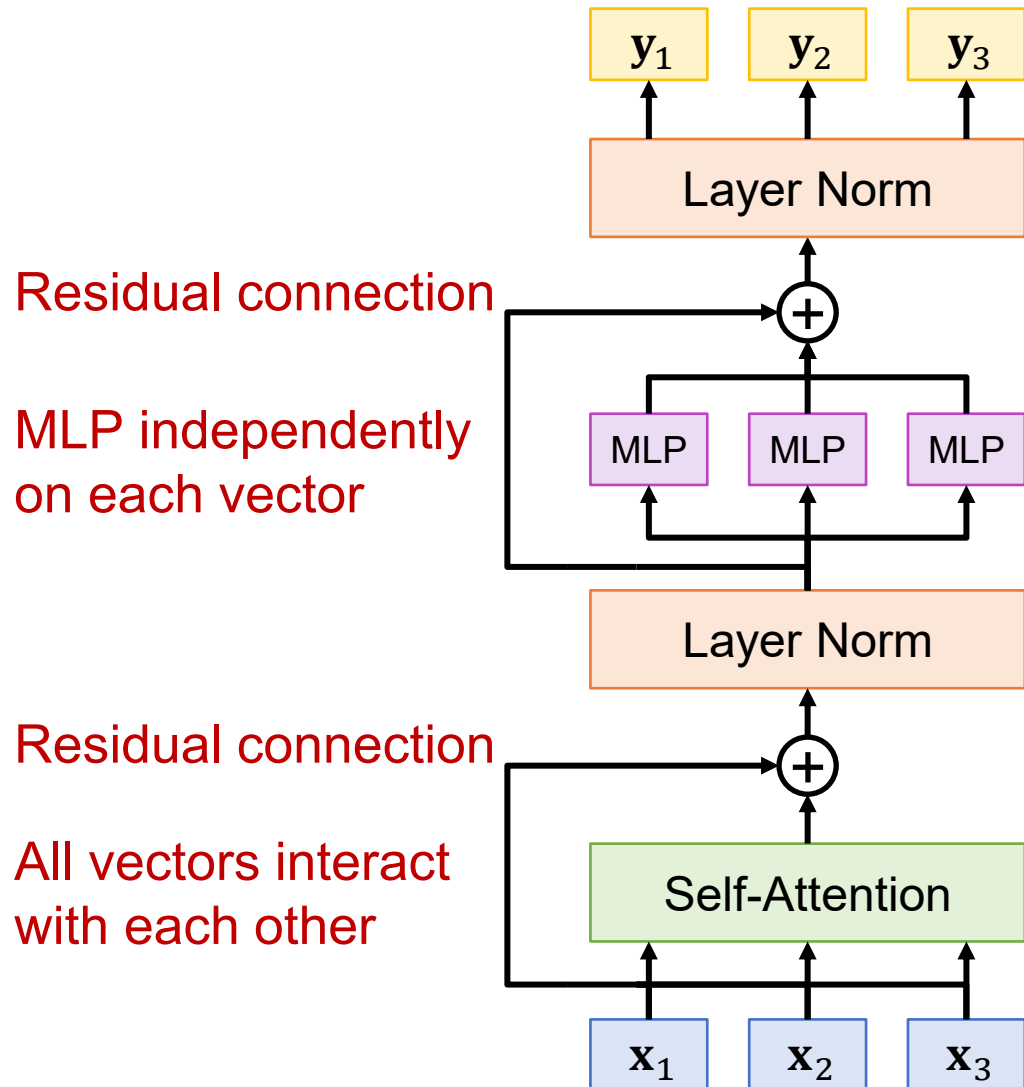
$$\mu_i = \frac{\sum_j h_{i,j}}{D} \quad (\text{Scalar})$$

$$\sigma_i = \sqrt{\frac{\sum_j (h_{i,j} - \mu_i)^2}{D}} \quad (\text{Scalar})$$

$$\mathbf{z}_i = \frac{h_i - \mu_i}{\sigma_i}$$

$$\mathbf{y}_i = \gamma \mathbf{z}_i + \beta$$

Vaswani et. al., Attention Is All You Need, NeurIPS 2017



Transformers

Transformer Block:

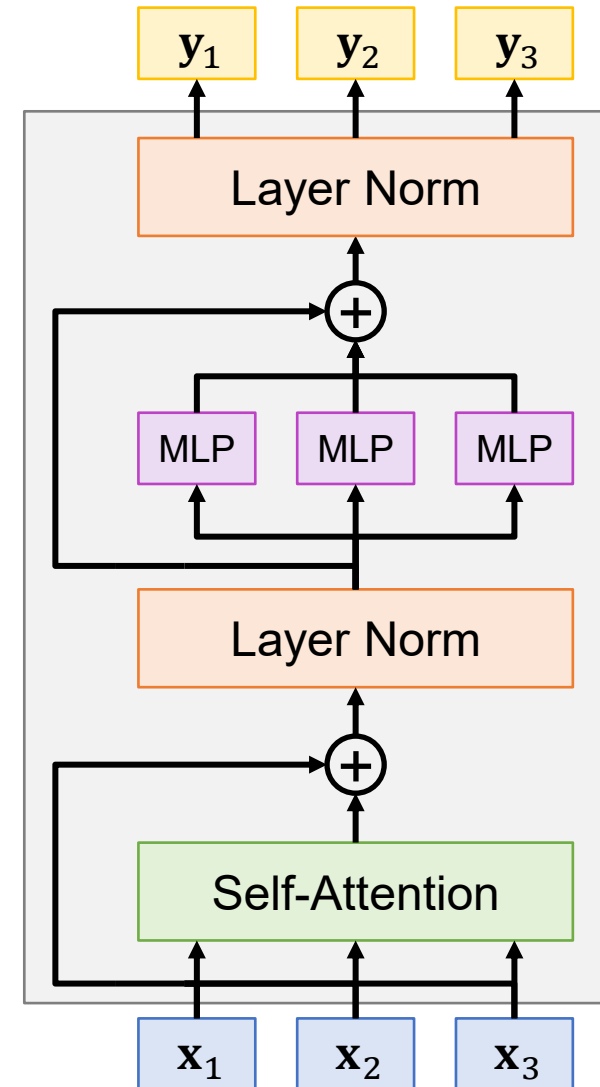
Input: a set of vectors $\mathbf{x}_1, \dots, \mathbf{x}_N$

Output: a set of vectors $\mathbf{y}_1, \dots, \mathbf{y}_N$

Attention is the only interaction between vectors!

Layer norm and MLP work independently per vector.

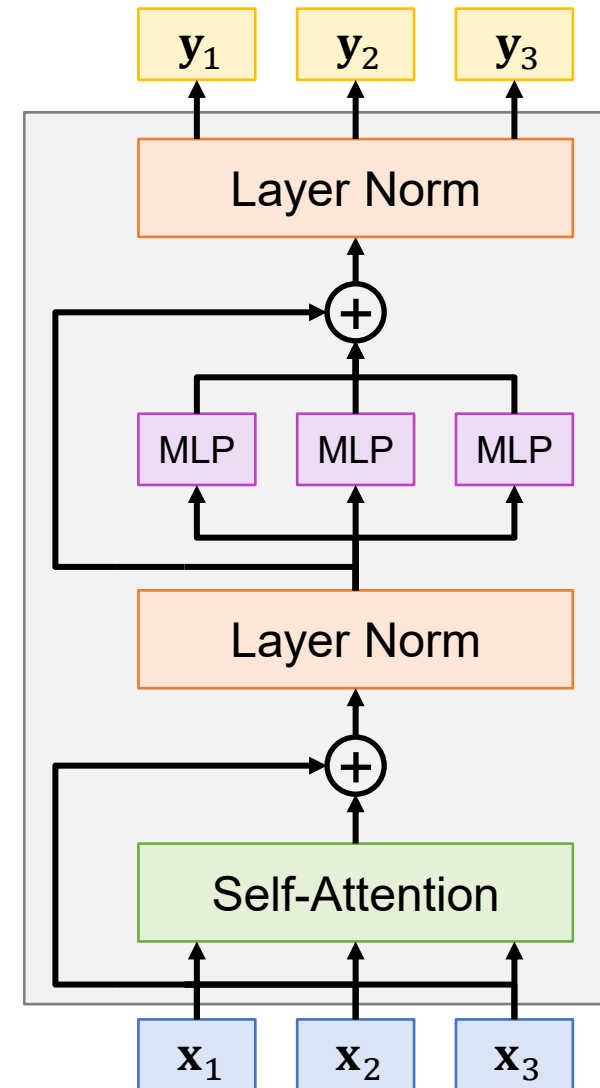
Highly scalable, highly parallelizable.



Transformers

Post-Norm Transformer:

Layer normalization is *after* residual connection



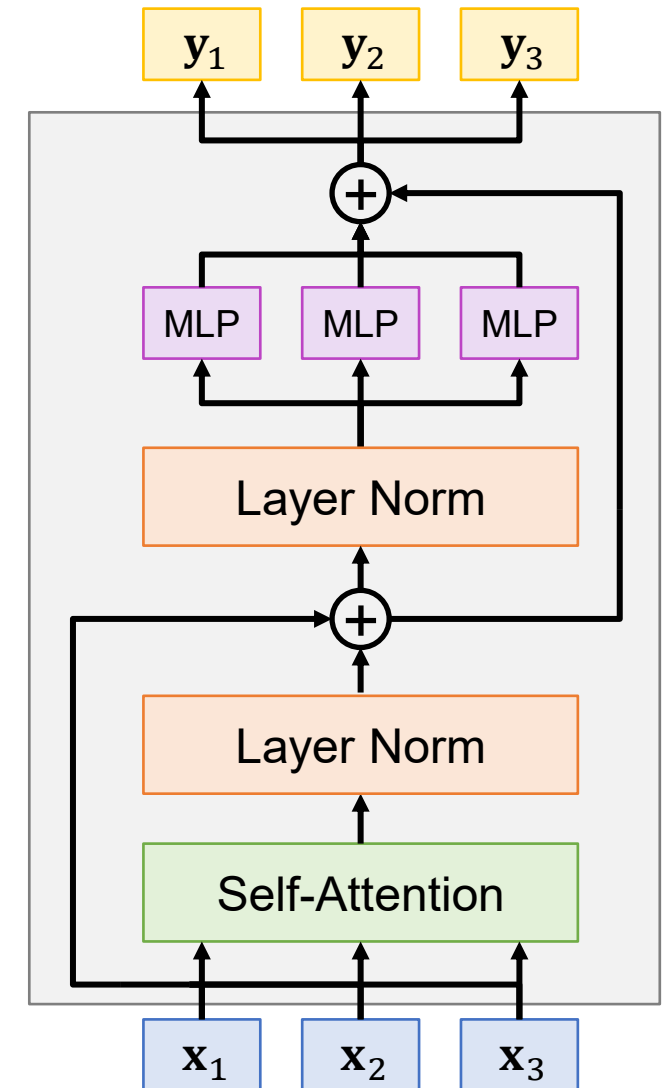
Vaswani et. al., Attention Is All You Need, NeurIPS 2017

Transformers

Pre-Norm Transformer:

Layer normalization is *inside* residual connection

Gives more stable training,
commonly used in practice!



Baevski et. al., Adaptive Input Representations for Neural Language Modeling, arXiv 2018

Transformers

Transformer Block:

Input: a set of vectors $\mathbf{x}_1, \dots, \mathbf{x}_N$

Output: a set of vectors $\mathbf{y}_1, \dots, \mathbf{y}_N$

Attention is the only interaction between vectors!

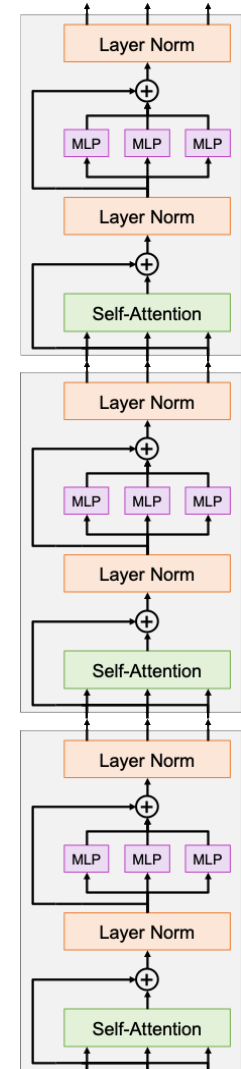
Layer norm and MLP work independently per vector.

Highly scalable, highly parallelizable.

A *transformer* is a stack of transformer blocks.

Vaswani et. al.:

12 blocks, DQ=512, 6 heads



Large Language Models

Language Modeling

- The task of predicting *the next word* in a document.
- This technique can be used to train language models.
- Language models can be applied to a wide range of tasks:
 - Text generation, text completion, question answering.

$$P(x_1, x_2, \dots, x_T) = P(x_1)P(x_2|x_1) \cdots P(x_T|x_1, \dots, x_{T-1}) = \prod_{t=1}^T P(x_t|x_{<t})$$

I ____

Web search engine / ...

I saw a cat|

I saw a cat on the chair
I saw a cat running after a dog
I saw a cat in my dream
I saw a cat book

Keyboard / mail agent / ...

I saw a catt

cat
car

Language Models are Unsupervised Multitask Learners

- GPT-2
 - Language models can perform multiple downstream tasks without task-specific finetuning.
 - As the model size gets bigger, the zero-shot capability also improves

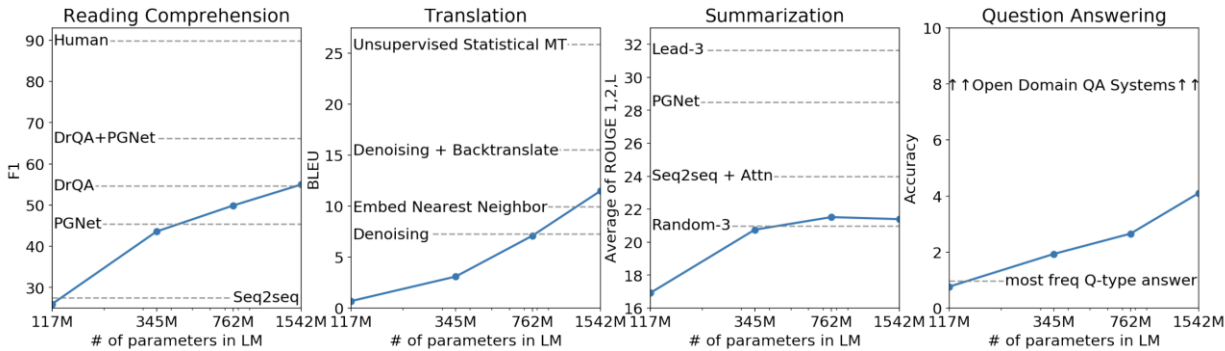


Figure 1. Zero-shot task performance of WebText LMs as a function of model size on many NLP tasks. Reading Comprehension results are on CoQA (Reddy et al., 2018), translation on WMT-14 Fr-En (Artetxe et al., 2017), summarization on CNN and Daily Mail (See et al., 2017), and Question Answering on Natural Questions (Kwiatkowski et al., 2019). Section 3 contains detailed descriptions of each result.

	LAMBADA (PPL)	LAMBADA (ACC)	CBT-CN (ACC)	CBT-NE (ACC)	WikiText2 (PPL)	PTB (PPL)	enwik8 (BPB)	text8 (BPC)	WikiText103 (PPL)	1BW (PPL)
SOTA	99.8	59.23	85.7	82.3	39.14	46.54	0.99	1.08	18.3	21.8
117M	35.13	45.99	87.65	83.4	29.41	65.85	1.16	1.17	37.50	75.20
345M	15.60	55.48	92.35	87.1	22.76	47.33	1.01	1.06	26.37	55.72
762M	10.87	60.12	93.45	88.0	19.93	40.31	0.97	1.02	22.05	44.575
1542M	8.63	63.24	93.30	89.05	18.34	35.76	0.93	0.98	17.48	42.16

Table 3. Zero-shot results on many datasets. No training or fine-tuning was performed for any of these results. PTB and WikiText-2 results are from (Gong et al., 2018). CBT results are from (Bajgar et al., 2016). LAMBADA accuracy result is from (Hoang et al., 2018) and LAMBADA perplexity result is from (Grave et al., 2016). Other results are from (Dai et al., 2019).

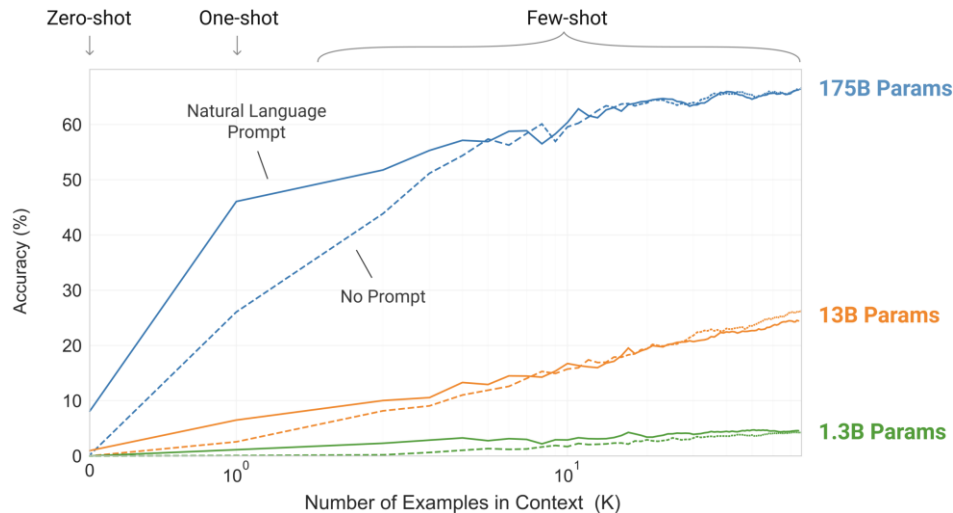
	R-1	R-2	R-L	R-AVG
Bottom-Up Sum	41.22	18.68	38.34	32.75
Lede-3	40.38	17.66	36.62	31.55
Seq2Seq + Attn	31.33	11.81	28.83	23.99
GPT-2 TL; DR:	29.34	8.27	26.58	21.40
Random-3	28.78	8.63	25.52	20.98
GPT-2 no hint	21.58	4.03	19.47	15.03

Table 4. Summarization performance as measured by ROUGE F1 metrics on the CNN and Daily Mail dataset. Bottom-Up Sum is the SOTA model from (Gehrmann et al., 2018)

Language Models are Few-Shot Learners

- GPT-3

- *In-context learning*: few shot examples given within the context improves the downstream predictions.



Brown et. al., Language Models are Few-Shot Learners, 2020

The three settings we explore for in-context learning

Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 cheese => ..... ← prompt
```

One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 sea otter => loutre de mer ← example
3 cheese => ..... ← prompt
```

Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 sea otter => loutre de mer ← examples
3 peppermint => menthe poivrée ←
4 plush girafe => girafe peluche ←
5 cheese => ..... ← prompt
```

Traditional fine-tuning (not used for GPT-3)

Fine-tuning

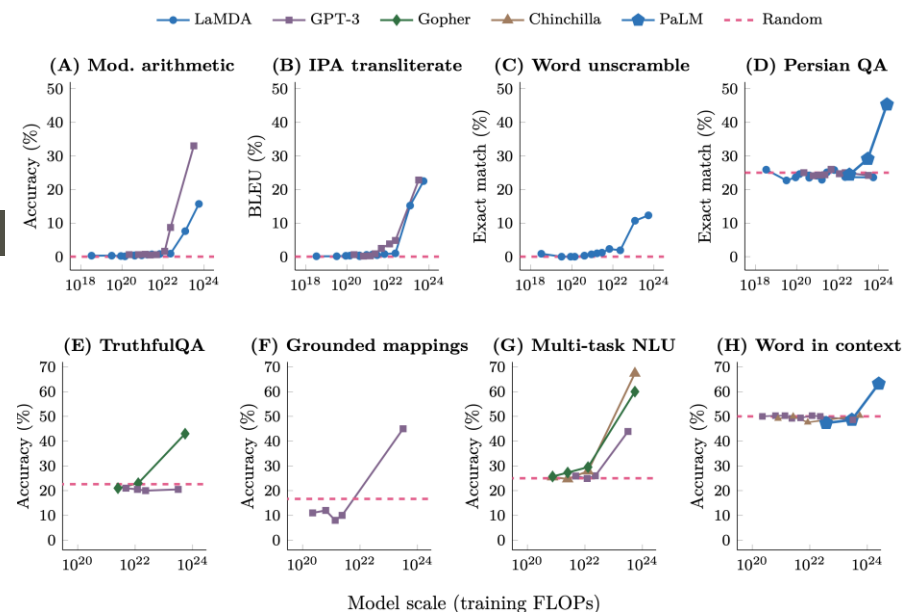
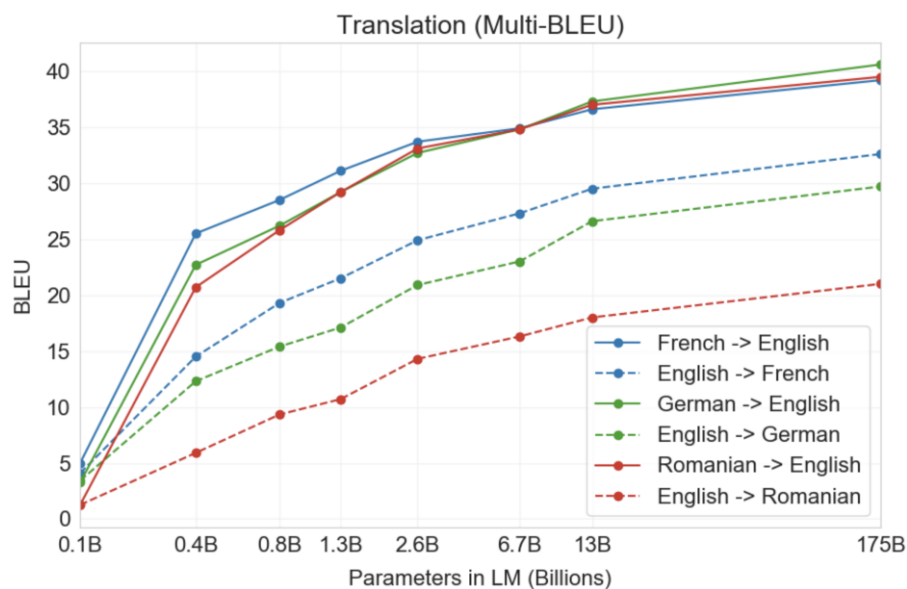
The model is trained via repeated gradient updates using a large corpus of example tasks.



Language Models are Few-Shot Learners

• GPT-3

- Model sizes matter.
- New abilities are emergent as the model size becomes bigger.

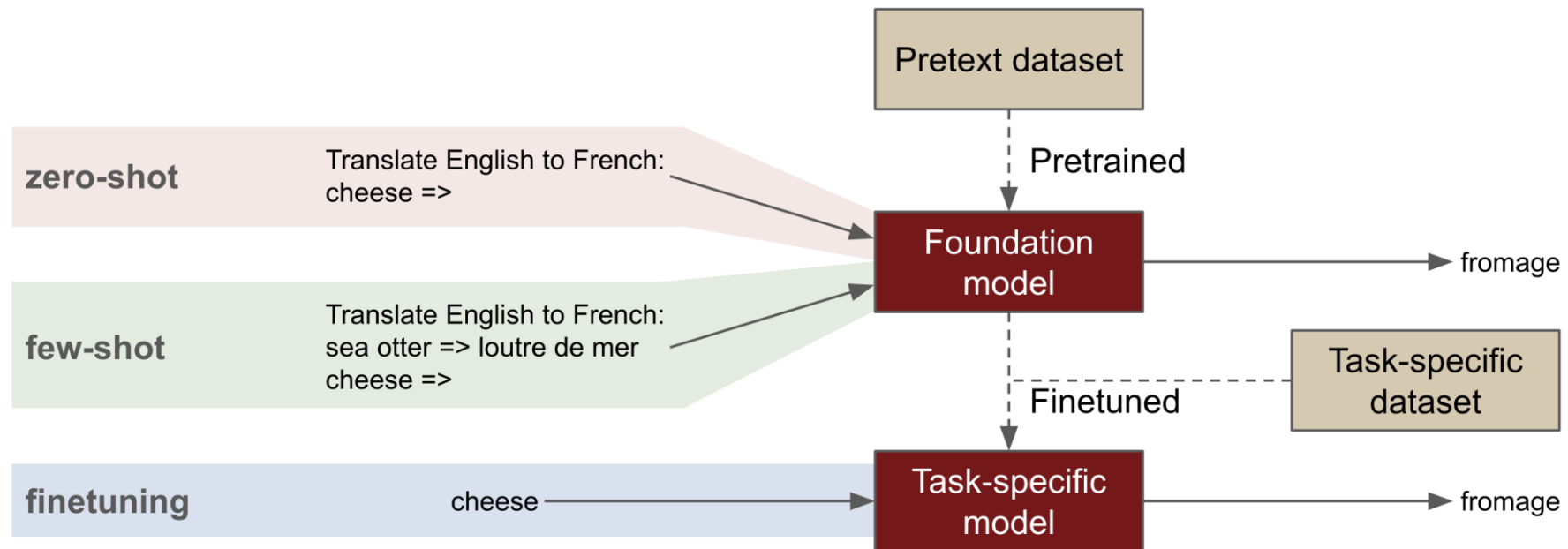


Model Name	n_{params}	n_{layers}	d_{model}	n_{heads}	d_{head}	Batch Size	Learning Rate
GPT-3 Small	125M	12	768	12	64	0.5M	6.0×10^{-4}
GPT-3 Medium	350M	24	1024	16	64	0.5M	3.0×10^{-4}
GPT-3 Large	760M	24	1536	16	96	0.5M	2.5×10^{-4}
GPT-3 XL	1.3B	24	2048	24	128	1M	2.0×10^{-4}
GPT-3 2.7B	2.7B	32	2560	32	80	1M	1.6×10^{-4}
GPT-3 6.7B	6.7B	32	4096	32	128	2M	1.2×10^{-4}
GPT-3 13B	13.0B	40	5140	40	128	2M	1.0×10^{-4}
GPT-3 175B or "GPT-3"	175.0B	96	12288	96	128	3.2M	0.6×10^{-4}

Table 2.1: Sizes, architectures, and learning hyper-parameters (batch size in tokens and learning rate) of the models which we trained. All models were trained for a total of 300 billion tokens.

Transferring Knowledge of FMs

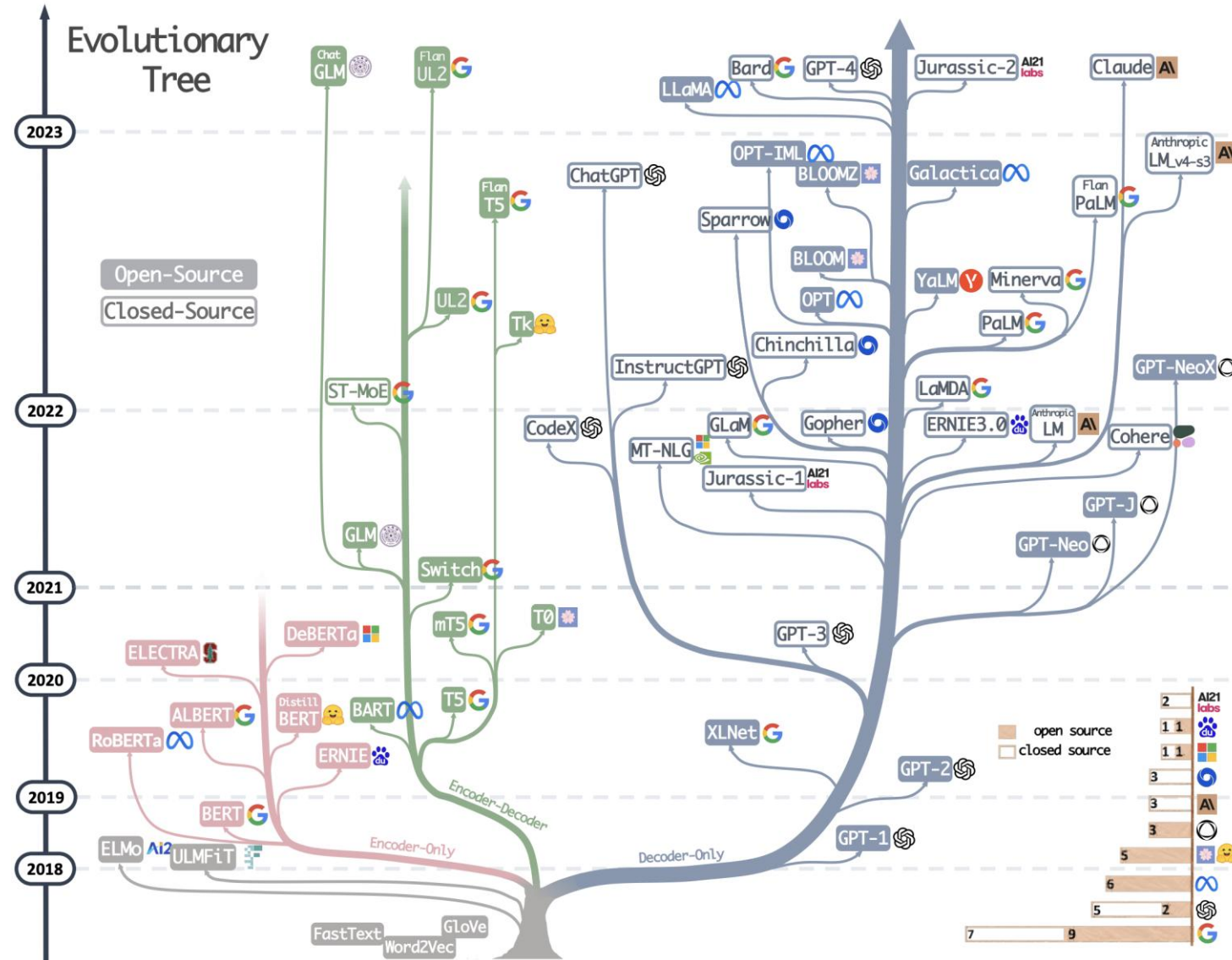
- **Data:** unannotated data, web-scale, multimodal
 - CLIP, Flamingo, SAM use (almost) annotated datasets.
- **Pretext objectives:** self-supervised learning
 - CLIP directly uses a retrieval objective.
- **Adaptation:** finetuning, prompt-engineering (zero/few-shot)



LLMs as Generalist AI Systems

- The Generalist Paradigm
 - A single LLM performs diverse tasks without task-specific training.
 - **Emergent Versatility**: Capability to perform coding, creative writing, and logical reasoning without retraining.
 - **Zero-Shot Learning**: The ability to solve novel problems purely through natural language instructions (prompting).
- Measuring Broad Intelligence
 - **Evaluation Shift**: From single-metric accuracy to Massive Multi-Task Benchmarking.

The Evolutionary Tree of Modern LLMs



Two Ways of Using LLMs

Commercial APIs

GPT-4o, Claude, Gemini — via cloud API

Pros

- No infrastructure to build or maintain
- Best-in-class quality, always updated
- Fast to integrate; scales on demand

Cons

- Data leaves your environment
- Ongoing per-token / usage costs
- Vendor lock-in, rate limits, downtime risk

On-Prem / Self-Hosted

Llama, Mistral, Qwen — on your own GPUs

Pros

- Full data privacy and control
- Fixed cost; no per-call fees
- Customizable, fine-tunable, offline-capable

Cons

- Requires GPU hardware and MLOps
- Quality below frontier models
- You own maintenance and scaling

